Bases de données avancées

Moulla Donatien K., Ph.D

Janvier 2026

moulladonatien@gmail.com

Descriptif du cours

- Acquérir des connaissances dans l'optimisation des requêtes.
- Avoir les connaissances indispensables à la maîtrise des Systèmes de Gestion de BD actuels, leur fondement et leurs limites.
- Maîtriser les concepts et techniques des bases de données de nouvelle génération (BD parallèles et réparties).
- Avoir les connaissances sur les bases de données Web et leurs intégrations.

© Copyright 2026, Moulla

objectifs du cours

- Nommer les caractéristiques et avantages du modèle relationnel de BD;
- Modéliser les relations et les contraintes d'un problème sous la forme d'un schéma relationnel efficace;
- Comprendre le problème de la redondance dans le modèle relationnel, et appliquer des méthodes de normalisation pour éliminer cette redondance;
- Définir les principales caractéristiques et technologies du paradigme de la persistance transparente;
- Décrire les caractéristiques et avantages des principales alternatives au modèle relationnel;
- Comprendre les enjeux reliés à l'optimisation de requêtes SQL, et décrire les principales stratégies permettant d'accélérer ces requêtes;
- Connaître les différences entre les entrepôts de données et les BD transactionnelles.
- Nommer les caractéristiques et avantages des BD parallèles et réparties.

© Copyright 2026, Moulla

Cours

Points importants

- Introduction + modèle relationnel;
- Conception du schéma relationnel;
- Normalisation du schéma relationnel;
- Persistance transparente;
- Intégrité et gestion des transactions;
- Gestion des données en mémoire;
- Optimisation de la performance;
- Entrepôts de données;
- BD parallèles et réparties;
- BD non-relationnelles (NoSQL) et Big Data.

© Copyright 2026, Moulla

Introduction au modèle relationnel

Concepts de base

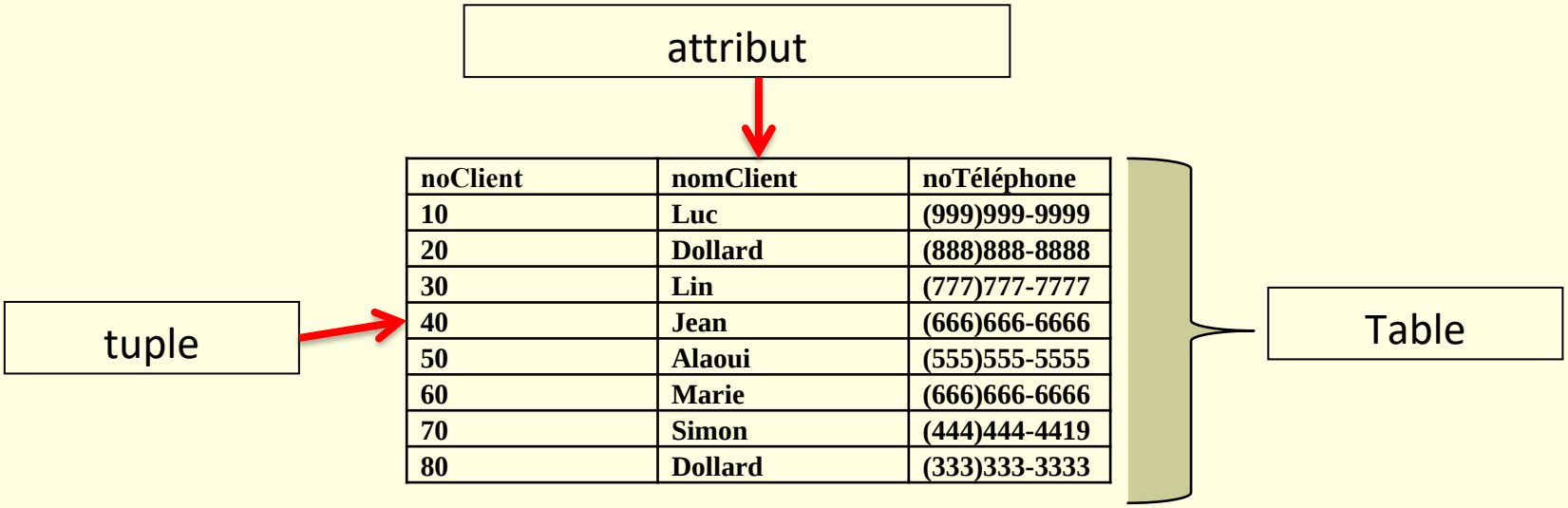
- BD : composante essentielle des systèmes informatiques modernes
  - systèmes d ’information de gestion;
  - Ingénierie;
  - contrôle de processus;
  - bibliothèques électroniques;
  - Etc.

© Copyright 2026, Moulla

Introduction au modèle relationnel

Concepts de base

- *Domaine* : ensemble de valeurs possibles pour une colonne.
- *Relation* : ensemble de tuples
- SQL : multi-ensemble



noClient	nomClient	noTéléphone
10	Luc	(999)999-9999
20	Dollard	(888)888-8888
30	Lin	(777)777-7777
40	Jean	(666)666-6666
50	Alaoui	(555)555-5555
60	Marie	(666)666-6666
70	Simon	(444)444-4419
80	Dollard	(333)333-3333

© Copyright 2026, Moulla

Introduction au modèle relationnel

Concepts de base

- **Exercice :** Donnez trois exemples de domaines pour une base de données sur une bibliothèque.
- **Correction :**

Exemple de domaines :

1. Domaine "Genre" : {Masculin, Féminin}.
2. Domaine "Statut du livre" : {Disponible, Emprunté}.
3. Domaine "Type de membre" : {Étudiant, Professeur, Externe}.

© Copyright 2026, Moulla

Introduction au modèle relationnel

- **Deux facettes du concept de table:**

  - Schéma d'une table (table schema)
    - définition de son type (intention)
      - *ex: Client(noClient, nomClient, noTéléphone)*
  - Instance ou extension d'une table
    - état de la table
    - variable qui contient un ensemble de lignes
    - modifications d 'état

- **Schéma relationnel**

  - ensemble de schémas de tables

© Copyright 2026, Moulla

Exemple d'une instance de BD

Table article			
noArticle	Description	PrixUnitaire	QuantitéEnStock
10	Cèdre en boule	10	10
20	Sapin	12	10
40	Epinette bleue	25	10
50	Chêne	22	10
60	Erable argenté	15	10
70	Herbe à puce	10	10
80	Poirier	26	10
81	Catalpa	25	10
90	Pommier	25	10
95	Genévrier	15	10

Table Commande		
noCommande	DateCommande	noClient
1	01/06/2000	10
2	02/06/2000	20
3	02/06/2000	10
4	05/07/2000	10
5	09/07/2000	30
6	09/07/2000	20
7	15/07/2000	40
8	15/07/2000	40

Table DétailLivraison	
noLivraison	noCommande
100	03/06/2000
101	04/06/2000
102	04/06/2000
103	05/06/2000
104	07/06/2000

Table Client		
noClient	nomClient	noTéléphone
10	Luc	(999)999-9999
20	Dollard	(888)888-8888
30	Lin	(777)777-7777
40	Jean	(666)666-6666
50	Alaoui	(555)555-5555
60	Marie	(666)666-6666
70	Simon	(444)444-4419
80	Dollard	(333)333-3333

Table LigneCommande		
noCommande	noArticle	quantité
1	10	10
1	70	5
1	90	1
2	40	2
2	95	3
3	20	1
4	40	1
4	50	1
5	70	3
5	10	5
5	20	5

Table DétailLivraison			
noLivraison	noCommande	noArticle	quantitéLivrée
100	1	10	
100	1	70	
101	1	10	
102	2	40	
102	2	95	
100	3	20	
103	1	90	
104	4	40	

© Copyright 2026, Moulla

Introduction au modèle relationnel

Contrainte de domaine et de valeur non Nulle

- $T(A_1 : D_1 , A_2 : D_2 , \ldots, A_n : D_n)$
  - D_1: domaine de A_1
- Valeur nulle
  - Comportement particulier
  - Valeur inconnue
  - Valeur non applicable
  - ...

© Copyright 2026, Moulla

Introduction au modèle relationnel

Clé primaire et contrainte d'entité

- *Clé unique (unique key) ou super clé (super key)*
  - Ensemble de colonnes identifiant de manière unique une ligne
- *Clé candidate (candidate key)*
  - Clé unique minimale (on ne peut pas retirer de colonnes sans perdre l'unicité)
- *Clé primaire (primary key)*
  - Clé candidate servant à référencer les lignes de la table
- *Contrainte d'entité (entity constraint)*
  - Chaque table doit avoir une clé primaire non-nulle

© Copyright 2026, Moulla

Introduction au modèle relationnel

Contrainte d'intégrité référentielle



Table Commande		
noCommande	**DateCommande**	**noClient**
1	01/06/2000	10
2	02/06/2000	20
3	02/06/2000	10
4	05/07/2000	10
5	09/07/2000	30
6	09/07/2000	20
7	15/07/2000	40
8	15/07/2000	40

Table Client		
noClient	**nomClient**	**noTéléphone**
10	Luc	(999)999-9999
20	Dollard	(888)888-8888
30	Lin	(777)777-7777
40	Jean	(666)666-6666
50	Alaoui	(555)555-5555
60	Marie	(666)666-6666
70	Simon	(444)444-4419
80	Dollard	(333)333-3333

Clé étrangère non nulle → clé primaire

© Copyright 2026, Moulla

Introduction au modèle relationnel

Algèbre relationnelle

- Opérations de manipulation de données
- Entrée
  - Une table : opération unaire
  - Deux tables : opération binaire
- Sortie
  - Une table contenant le résultat de l'opération

© Copyright 2026, Moulla

Introduction au modèle relationnel

Ensemble minimal d'opérations de l'algèbre relationnel

- Sélection (σ)
- Projection (Π)
- Produit cartésien (x)
- Jointure (⋈)
- Opérations ensemblistes (U, ∩, -)

© Copyright 2026, Moulla

Introduction au modèle relationnel

Sélection (σ)

Table article			
noArticle	**Description**	**PrixUnitaire**	**QuantitéEnStock**
10	**Cèdre en boule**	**10**	**10**
20	**Sapin**	**12**	**10**
40	**Epinette bleue**	**25**	**10**
50	**Chêne**	**22**	**10**
60	**Erable argenté**	**15**	**10**
70	**Herbe à puce**	**10**	**10**
80	**Poirier**	**26**	**10**
81	**Catalpa**	**25**	**10**
90	**Pommier**	**25**	**10**
95	**Genévrier**	**15**	**10**

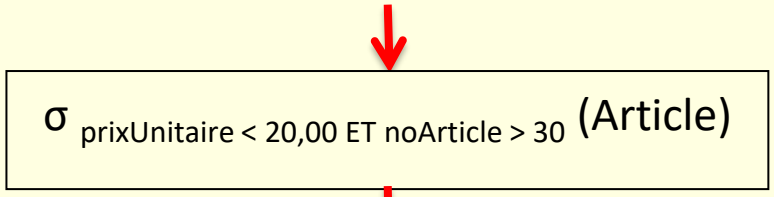


Table article			
noArticle	**Description**	**PrixUnitaire**	**QuantitéEnStock**
60	**Erable argenté**	**15**	**10**
70	**Herbe à puce**	**10**	**10**
95	**Genévrier**	**15**	**10**

© Copyright 2026, Moulla

Introduction au modèle relationnel

Sélection (σ)

- La sélection (parfois appelé restriction) est l'opérateur qui permet de sélectionner des tuples selon le prédicat donné.
  - Notation : $R' = \sigma_{condition}\ R$
- Interprétation : la relation R' correspond à la sélection des tuples de R respectant la ou les conditions données.
- Les conditions sont déterminées par des opérateurs appliqués sur les valeurs des attributs.

© Copyright 2026, Moulla

Introduction au modèle relationnel

Projection (π)

Table Commande		
noCommande	**DateCommande**	**noClient**
1	01/06/2000	10
2	02/06/2000	20
3	02/06/2000	10
4	05/07/2000	10
5	09/07/2000	30
6	09/07/2000	20
7	15/07/2000	40
8	15/07/2000	40



$\pi_{\text{noClient, DateCommande}}$ (Commande)



noClient	DateCommande
10	01/06/2000
20	02/06/2000
10	02/06/2000
10	05/07/2000
30	09/07/2000
20	09/07/2000
40	15/07/2000

© Copyright 2026, Moulla

Introduction au modèle relationnel

Projection (π)

La projection est l'opérateur qui permet de sélectionner des attributs selon le prédicat donné.

Notation : $R' = \pi_{a1,a2,a3,\ldots}\ R$

Interprétation : la relation **R'** correspond à la projection des attributs $a1, a2, a3$, ... de la relation R.

© Copyright 2026, Moulla

Introduction au modèle relationnel

Expressions complexes

Table Commande		
noCommande	**DateCommande**	**noClient**
1	01/06/2000	10
2	02/06/2000	20
3	02/06/2000	10
4	05/07/2000	10
5	09/07/2000	30
6	09/07/2000	20
7	15/07/2000	40
8	15/07/2000	40

$$\sigma_{\text{dateCommande} > 05/07/2000}(\text{Commande})$$

noCommande	DateCommande	noClient
5	09/07/2000	30
6	09/07/2000	20
7	15/07/2000	40
8	15/07/2000	40

$$\pi_{\text{noClient, DateCommande}}(\sigma_{\text{dateCommande} > 05/07/2000}(\text{Commande}))$$

noClient	DateCommande
30	09/07/2000
20	09/07/2000
40	15/07/2000

© Copyright 2026, Moulla

Introduction au modèle relationnel

Produit cartésien (x)

Table Client		
noClient	nomClient	noTéléphone
10	Luc	(999)999-9999
20	Dollard	(888)888-8888
30	Lin	(777)777-7777
40	Jean	(666)666-6666
50	Alaoui	(555)555-5555
60	Marie	(666)666-6666
70	Simon	(444)444-4419
80	Dollard	(333)333-3333

....

Table Commande		
noCommande	DateCommande	noClient
1	01/06/2000	10
2	02/06/2000	20
3	02/06/2000	10
4	05/07/2000	10
5	09/07/2000	30
6	09/07/2000	20
7	15/07/2000	40
8	15/07/2000	40

Client x Commande

Client.noClient	nomClient	noTéléphone	noCommande	dateCommande	Commande.noClient
10	Luc	(999)999-9999	1	01/06/2000	10
10	Luc	(999)999-9999	2	02/06/2000	20
10	Luc	(999)999-9999	3	02/06/2000	10
10	Luc	(999)999-9999	4	05/07/2000	10
10	Luc	(999)999-9999	5	09/07/2000	30
10	Luc	(999)999-9999	6	09/07/2000	20
10	Luc	(999)999-9999	7	15/07/2000	40
10	Luc	(999)999-9999	8	15/07/2000	40
20	Dollard	(888)888-8888	1	01/06/2000	10
20	Dollard	(888)888-8888	2	02/06/2000	20
....	.....	.....	.....	.....	.....

Introduction au modèle relationnel

Produit cartésien (x)

Le produit cartésien est un opérateur binaire permettant de produire une relation $\boldsymbol{R'}$ avec tous les tuples qui appartiennent à $\boldsymbol{R1}$ combinés à chacun des tuples de ***R2***. De plus, le schéma de $\boldsymbol{R'}$ est l'union des schémas de $\boldsymbol{R1}$ et $\boldsymbol{R2}$.

Notation: $R' = R1 \; \boldsymbol{X} \; R2$

Interprétation : produit cartésien des tuples des relations **R1** et **R2**. Cet opérateur est commutatif.

© Copyright 2026, Moulla

Introduction au modèle relationnel

Jointure (⋈)

colonnes communes appelées colonnes de jointure
ou clé de jointure

Table Client		
noClient	nomClient	noTéléphone
10	Luc	(999)999-9999
20	Dollard	(888)888-8888
30	Lin	(777)777-7777
40	Jean	(666)666-6666
50	Alaoui	(555)555-5555
60	Marie	(666)666-6666
70	Simon	(444)444-4419
80	Dollard	(333)333-3333

Table Commande		
noCommande	DateCommande	noClient
1	01/06/2000	10
2	02/06/2000	20
3	02/06/2000	10
4	05/07/2000	10
5	09/07/2000	30
6	09/07/2000	20
7	15/07/2000	40
8	15/07/2000	40

Client ⋈ Commande

noClient	nomClient	noTéléphone	noCommande	DateCommande
10	Luc	(999)999-9999	1	01/06/2000
10	Luc	(999)999-9999	3	02/06/2000
10	Luc	(999)999-9999	4	05/07/2000
20	Dollard	(888)888-8888	2	02/06/2000
20	Dollard	(888)888-8888	6	09/07/2000
30	Lin	(777)777-7777	5	09/07/2000
40	Jean	(666)666-6666	7	15/07/2000
40	Jean	(666)666-6666	8	15/07/2000

Introduction au modèle relationnel

Jointure (⋈)

La jointure est l'opération qui consiste à appliquer à la fois un produit cartésien et une sélection.

notation : $R' = R1 \bowtie_{condition} R2$

interprétation: (même sens mathématique)

$$R' = \sigma_{condition} R1 \; \boldsymbol{X} \; R2$$

© Copyright 2026, Moulla

Introduction au modèle relationnel

Opérations ensemblistes (U, ∩, -)

Table T1	
A	B
1	1
2	2

Table T2	
A	B
2	2
3	3

T1 U T2	
A	B
1	1
2	2
3	3

T1∩T2	
A	B
2	2

T1-T2	
A	B
1	1

T2-T1	
A	B
3	3

© Copyright 2026, Moulla

Conception de BD relationnelles

Modélisation des données

- ***Schéma conceptuel***
  - Modélise les classes, leurs attributs et leurs relations (ex: association, agrégation, spécialisation, etc.)
  - *Exemple*: diagramme de classe UML.

- ***Schéma relationnel (conceptuel)***
  - Traduit le schéma conceptuel sous la forme d'un modèle relationnel (ex: tables, colonnes, clés, contraintes, etc.).
  - Indépendant de la plateforme/BD utilisée.
  - *Exemple*: diagramme UML avec tables, diagramme entités - associations

© Copyright 2026, Moulla

Conception de BD relationnelles

Modélisation des données

Exemple de système de gestion de bibliothèque :

1. **Diagramme UML :**
   - **Classes :** Livre, Auteur, Emprunt.
   - **Relations :** Un auteur écrit plusieurs livres; Un livre peut être emprunté plusieurs fois.
2. **Traduction en schéma relationnel :**
   - **Table Livre :** (IDLivre, Titre, DatePublication, IDAuteur).
   - **Table Auteur :** (IDAuteur, Nom, Prénom).
   - **Table Emprunt :** (IDEmprunt, IDLivre, IDLecteur, DateEmprunt).

© Copyright 2026, Moulla

Conception de BD relationnelles

Modélisation des données

Exercice : Réalisez un diagramme UML pour un système de gestion de commandes dans un magasin et traduisez-le.

- **Correction :**
- **Diagramme UML :**
  - **Classes :** Client, Commande, Produit.
  - **Relations :** Un client passe plusieurs commandes; Une commande contient plusieurs produits.
- **Traduction :**
  - **Table Client :** (IDClient, Nom, Adresse).
  - **Table Commande :** (IDCommande, DateCommande, IDClient).
  - **Table Produit :** (IDProduit, NomProduit, PrixUnitaire).
  - **Table CommandeProduit :** (IDCommande, IDProduit, Quantité)

© Copyright 2026, Moulla

Conception de BD relationnelles

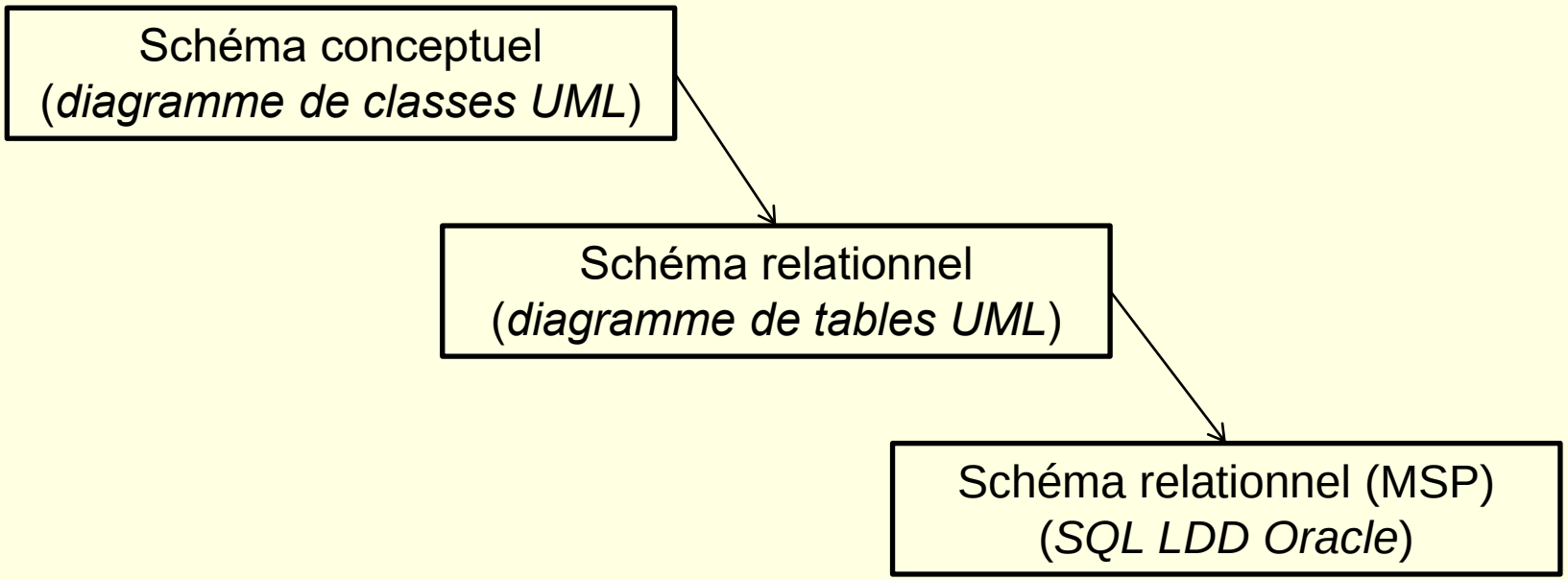
Modélisation des données

- ***Schéma relationnel***
  - Modèle spécifique à la plateforme (MSP).
  - Implémente le schéma relationnel conceptuel en considérant une plateforme spécifique.
  - Tient compte de la syntaxe spécifique à la plateforme/BD.
  - *Exemple*: SQL LDD Oracle (CREATE TABLE, VARCHAR2, etc.).

© Copyright 2026, Moulla

Conception de BD relationnelles

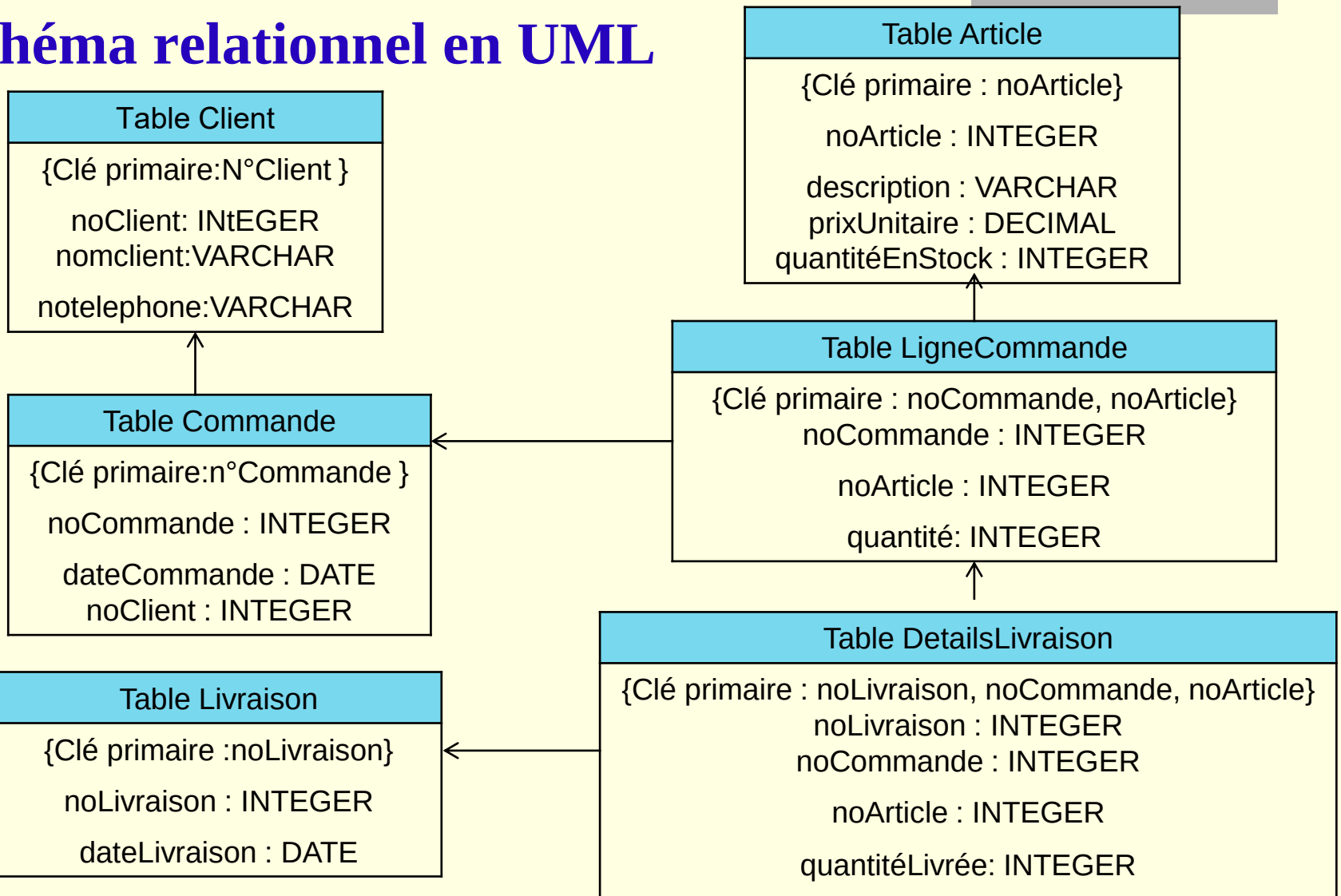
Processus de conception



© Copyright 2026, Moulla

Conception de BD relationnelles

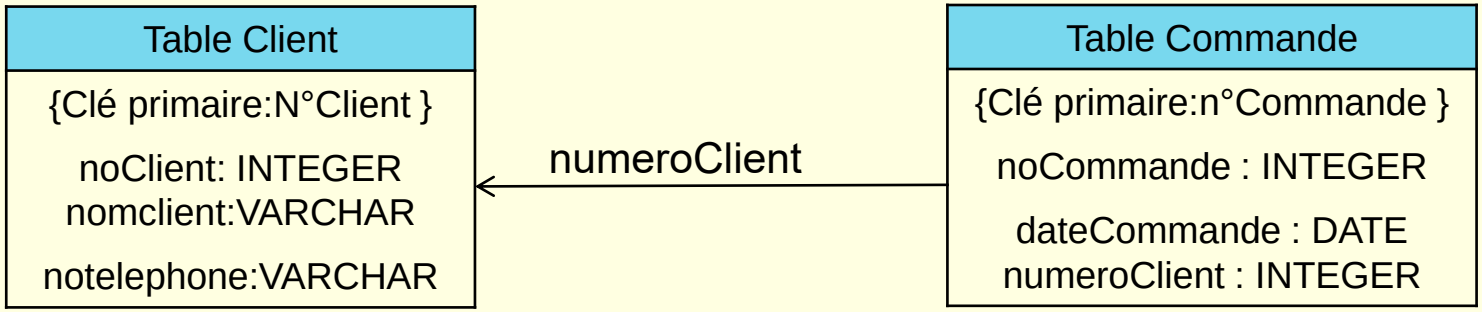
Schéma relationnel en UML



Conception de BD relationnelles

Schéma relationnel en UML

Étiquette de la relation de dépendance



© Copyright 2026, Moulla

Conception de BD relationnelles

Traduction du schéma conceptuel en schéma relationnel

- ***Etapes principales:***

1. Traduire les classes en tables;
2. Traduire les attributs et leur type;
3. Définir la clé primaire;
4. Traduire les associations.

© Copyright 2026, Moulla

Conception de BD relationnelles

Étape 2 : Traduire les attributs et leur type

- ***Cas possibles:***

1. Type simple (ex: Integer);
2. Type énuméré (ex: enum);
3. Type complexe (ex: struct en C/C++);
4. Attributs multivalués (ex: tableau Integer[0..*]);
5. Attributs de classe (ex: static).

Conception de BD relationnelles

Étape 2 : Traduire les attributs et leur type

- ***Attributs de type simple***

Attribut de la classe → colonne de la table

Livre
{UNIQUE: ISBN}
ISBN : String titre : String
annéeParution : TypeDonnéesAnnée

Table Livre
{Clé candidate: ISBN}
ISBN : CHAR(13) titre : VARCHAR(50)
annéeParution : DomaineAnnée

NB: les attributs UNIQUE deviennent des clés candidates.

© Copyright 2026, Moulla

Conception de BD relationnelles

Étape 2 : Traduire les attributs et leur type

❑ *Attributs de types énumérés*

- Petit domaine invariant
  - ✓ création d ’un domaine VARCHAR + CHECK

Exemplaire
{UNIQUE: idExemplaire}
idExemplaire : String dateAchat : Date
statut : enum (prêté, disponible, retiré)

Table Exemplaire
{Clé candidate : idExemplaire}
idExemplaire : VARCHAR(10) dateAchat : Date
statut : DomaineStatut

domain DomaineStatut
{VARCHAR(15) CHECK value IN ('prêté','disponible','retiré')}

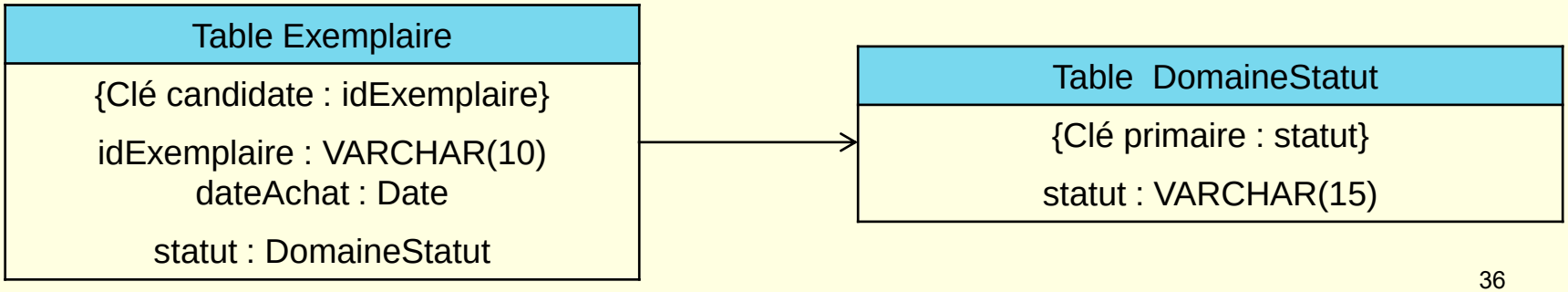
© Copyright 2026, Moulla

Conception de BD relationnelles

Étape 2 : Traduire les attributs et leur type

❑ *Attributs de types énumérés*

- Gros domaine ou extensible
  - ✓ création d ’une table à part;
  - ✓ utilisé comme liste de valeurs;
  - ✓ introduction d ’une clé primaire artificielle.



© Copyright 2026, Moulla

Conception de BD relationnelles

Étape 2 : Traduire les attributs et leur type

- ***Attributs de types complexes***

Représentation explicite des attributs du type complexe

Membre
adresse : typeDonnéesAdresse

Data type typeDonnéesAdresse
numéroCivique
numéroAppartement nomRue
nomVille
nomProvince nomPays
codePostal

Table Membre
numéroCivique
numéroAppartement nomRue
nomVille
nomProvince nomPays
codePostal

© Copyright 2026, Moulla

Conception de BD relationnelles

❑ *Attributs de types complexes*

Création d'une nouvelle table

Membre
adresse : typeDonnéesAdresse

Data type typeDonnéesAdresse
numéroCivique
numéroAppartement nomRue
nomVille
nomProvince nomPays
codePostal

Schéma conceptuel

Table Adresse
{Clé primaire : idMembre} idMembre numéroCivique
numéroAppartement nomRue
nomVille
nomProvince nomPays
codePostal

Schéma relationnel

Table Membre
{Clé primaire : idMembre} idMembre

Conception de BD relationnelles

- ***Attributs de types multivalués***

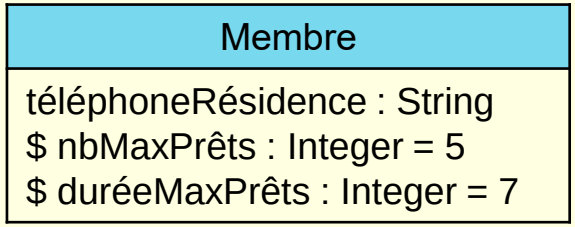
Plusieurs valeurs pour une entité donnée.

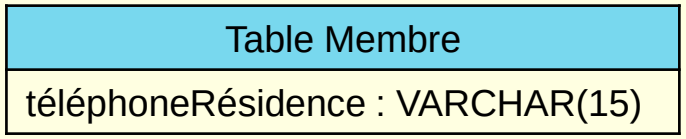
- Exemple: une personne ayant plusieurs prénoms.

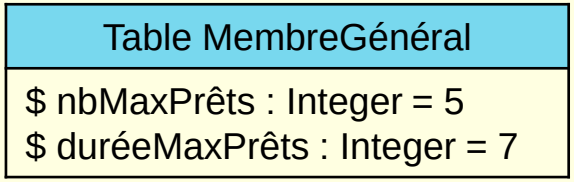
© Copyright 2026, Moulla

Conception de BD relationnelles

- ***Attributs de classes***







© Copyright 2026, Moulla

Conception de BD relationnelles

Étape 3 : Définir la clé primaire

- ***Choix possibles:***

1. Clé artificielle générée
2. Clé naturelle
3. Simple ou composée

© Copyright 2026, Moulla

Conception de BD relationnelles

Étape 4 : Traduire les relations

- ***Cas possibles:***

1. Plusieurs à plusieurs;
2. Un à plusieurs;
3. Un à un;
4. Agrégation, composition;
5. Spécialisation.

© Copyright 2026, Moulla

Normalisation du schéma relationnel

Exemple de mauvaise conception

Redondance de données

Table *Vente*

noCommande	dateCommande	noClient	nomClient	noTéléphone	noArticle	description	prixUnitaire	quantité
1	01/06/2000	10	Luc Sansom	(999)999-9999	10	Cèdre en boule	10.99	10
1	01/06/2000	10	Luc Sansom	(999)999-9999	70	Herbe à puce	10.99	5
1	01/06/2000	10	Luc Sansom	(999)999-9999	90	Pommier	25.99	1
2	02/06/2000	20	D o l l a r d Tremblay	(888)888-8888	40	Epinette bleue	25.99	2
2	02/06/2000	20	D o l l a r d Tremblay	(888)888-8888	95	Génévrier	15.99	3
3	02/06/2000	10	Luc Sansom	(999)999-9999	20	Sapin	12.99	1
4	05/06/2000	10	Luc Sansom	(999)999-9999	40	Epinette bleue	25.99	1
4	05/06/2000	10	Luc Sansom	(999)999-9999	50	Chêne	22.99	1
5	09/07/2000	30	Lin Bô	(777)777-7777	70	Herbe à puce	10.99	3
5	09/07/2000	30	Lin Bô	(777)777-7777	10	Cèdre en boule	10.99	5
5	09/07/2000	30	Lin Bô	(777)777-7777	20	Sapin	12.99	5
6	09/07/2000	20	D o l l a r d Tremblay	(888)888-8888	10	Cèdre en boule	10.99	5
6	09/07/2000	20	D o l l a r d Tremblay	(888)888-8888	40	Epinette bleue	25.99	1
7	15/07/2000	40	Jean Leconte	(666)666-6666	50	Chêne	22.99	1
7	15/07/2000	40	Jean Leconte	(666)666-6666	95	Génévrier	15.99	2
8	15/07/2000	40	Jean Leconte	(666)666-6666	20	Sapin	12.99	3

Normalisation du schéma relationnel

Solution: décomposition

Table *Vente*

noCommande	dateCommande	noClient	nomClient	noTéléphone	noArticle	description	prixUnitaire	quantité
1	01/06/2000	10	Luc Sansom	(999)999-9999	10	Cèdre en boule	10.99	10
1	01/06/2000	10	Luc Sansom	(999)999-9999	70	Herbe à puce	10.99	5
1	01/06/2000	10	Luc Sansom	(999)999-9999	90	Pommier	25.99	1
…								

Table *Client*

noClient	nomClient	noTéléphone
10	Luc Sansom	(999)999-9999
20	Dollard Tremblay	(888)888-8888
30	Lin Bô	(777)777-7777
40	Jean Leconte	(666)666-6666
50	Hafedh Lajoie	(555)555-5555
60	Comtesse Hasek	(666)666-6666
70	Coco McPoulet	(444)444-4419
80	Dollard Tremblay	(333)333-3333

Table *VenteReste*

noCommande	dateCommande	noClient	noArticle	description	prixUnitaire	quantité
1	01/06/2000	10	10	Cèdre en boule	10.99	10
1	01/06/2000	10	70	Herbe à puce	10.99	5
1	01/06/2000	10	90	Pommier	25.99	1
2	02/06/2000	20	40	Epinette bleue	25.99	2
2	02/06/2000	20	95	Génévrier	15.99	3
3	02/06/2000	10	20	Sapin	12.99	1
4	05/06/2000	10	40	Epinette bleue	25.99	1
4	05/06/2000	10	50	Chêne	22.99	1
5	09/07/2000	30	70	Herbe à puce	10.99	3
5	09/07/2000	30	10	Cèdre en boule	10.99	5
5	09/07/2000	30	20	Sapin	12.99	5
6	09/07/2000	20	10	Cèdre en boule	10.99	5
6	09/07/2000	20	40	Epinette bleue	25.99	1
7	15/07/2000	40	50	Chêne	22.99	1
7	15/07/2000	40	95	Génévrier	15.99	2
8	15/07/2000	40	20	Sapin	12.99	3

Normalisation du schéma relationnel

Théorie de la normalisation

- Résolution de problèmes de redondance de données;
- Par décomposition;
- Caractérisation des problèmes;
  - dépendances fonctionnelles, multivaluées,...
- Contexte relationnel;
- Transposable à d ’autres contextes.

© Copyright 2026, Moulla

Normalisation du schéma relationnel

Schéma normalisé versus dénormalisé

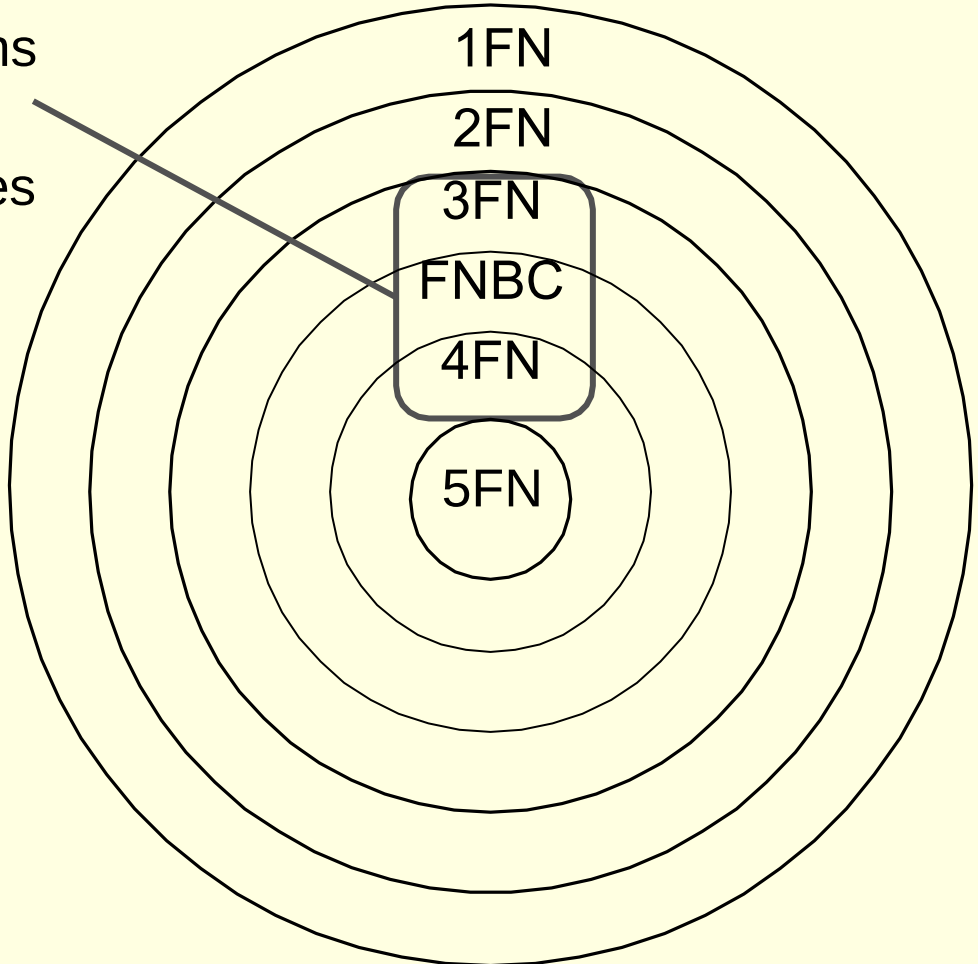
- Normalisé :
  - Évite la redondance des données;
  - Facilite les mises à jour;
  - Nécessite la jointure entre les tables normalisées;
  - Très employé dans les BD transactionnelles (3FN).
- Dénormalisé:
  - Évite la jointure des tables normalisées (meilleure performance);
  - Redondance et mises à jour complexes;
  - Très employé dans les entrepôts de données.

© Copyright 2026, Moulla

Normalisation du schéma relationnel

Différentes formes normales

Les plus employées dans les BD transactionnelles



De plus en plus restrictives

Normalisation du schéma relationnel

Décomposition sans perte

- Une décomposition binaire d'une table T en deux tables T_1 et T_2 est *sans perte* si : $T = T_1 \bowtie T_2$

- Exemple : $Vente = Client \bowtie VenteReste$

© Copyright 2026, Moulla

Normalisation du schéma relationnel

Exemple de décomposition avec perte

$$Vente \neq Client \bowtie VenteMalFoutue$$

Table *Client*

noClient	nomClient	noTéléphone
10	Luc Sansom	(999)999-9999
20	Dollard Tremblay	(888)888-8888
30	Lin Bô	(777)777-7777
40	Jean Leconte	(666)666-6666
50	Hafedh Lajoie	(555)555-5555
60	Comtesse Hasek	(666)666-6666
70	Coco McPoulet	(444)444-4419
80	Dollard Tremblay	(333)333-3333

Table *VenteMalFoutue*

noCommande	dateCommande	noArticle	description	prixUnitaire	quantité
1	01/06/2000	10	Cèdre en boule	10.99	10
1	01/06/2000	70	Herbe à puce	10.99	5
1	01/06/2000	90	Pommier	25.99	1
2	02/06/2000	40	Epinette bleue	25.99	2
2	02/06/2000	95	Génévrier	15.99	3
3	02/06/2000	20	Sapin	12.99	1
4	05/06/2000	40	Epinette bleue	25.99	1
4	05/06/2000	50	Chêne	22.99	1
5	09/07/2000	70	Herbe à puce	10.99	3
5	09/07/2000	10	Cèdre en boule	10.99	5
5	09/07/2000	20	Sapin	12.99	5
6	09/07/2000	10	Cèdre en boule	10.99	5
6	09/07/2000	40	Epinette bleue	25.99	1
7	15/07/2000	50	Chêne	22.99	1
7	15/07/2000	95	Génévrier	15.99	2
8	15/07/2000	20	Sapin	12.99	3

Normalisation du schéma relationnel

Dépendance fonctionnelle

- $A_1, A_2, \ldots, A_n \rightarrow^T B_1, B_2, \ldots, B_m$
  - pour toute instance de T
  - mêmes valeurs pour les colonnes $A_1, A_2, \ldots, A_n$ $\Rightarrow$mêmes valeurs pour les colonnes $B_1, B_2, \ldots, B_m$
- *Déterminant* : partie gauche $A_1, A_2, \ldots, A_n$
- Notation… Signifie réellement :
  - $\{A_1, A_2, \ldots, A_n\} \rightarrow^T \{B_1, B_2, \ldots, B_m\}$

Normalisation du schéma relationnel

Exemples pour Vente

Table *Vente*

noCommande	dateCommande	noClient	nomClient	noTéléphone	noArticle	description	prixUnitaire	quantité
1	01/06/2000	10	Luc Sansom	(999)999-9999	10	Cèdre en boule	10.99	10
1	01/06/2000	10	Luc Sansom	(999)999-9999	70	Herbe à puce	10.99	5
1	01/06/2000	10	Luc Sansom	(999)999-9999	90	Pommier	25.99	1
2	02/06/2000	20	D o l l a r d Tremblay	(888)888-8888	40	Epinette bleue	25.99	2
2	02/06/2000	20	D o l l a r d Tremblay	(888)888-8888	95	Génévrier	15.99	3
3	02/06/2000	10	Luc Sansom	(999)999-9999	20	Sapin	12.99	1
4	05/06/2000	10	Luc Sansom	(999)999-9999	40	Epinette bleue	25.99	1
4	05/06/2000	10	Luc Sansom	(999)999-9999	50	Chêne	22.99	1
5	09/07/2000	30	Lin Bô	(777)777-7777	70	Herbe à puce	10.99	3
5	09/07/2000	30	Lin Bô	(777)777-7777	10	Cèdre en boule	10.99	5
5	09/07/2000	30	Lin Bô	(777)777-7777	20	Sapin	12.99	5
6	09/07/2000	20	D o l l a r d Tremblay	(888)888-8888	10	Cèdre en boule	10.99	5
6	09/07/2000	20	D o l l a r d Tremblay	(888)888-8888	40	Epinette bleue	25.99	1
7	15/07/2000	40	Jean Leconte	(666)666-6666	50	Chêne	22.99	1
7	15/07/2000	40	Jean Leconte	(666)666-6666	95	Génévrier	15.99	2
8	15/07/2000	40	Jean Leconte	(666)666-6666	20	Sapin	12.99	3

$$noClient \rightarrow nomClient, noTéléphone$$

$$noCommande \rightarrow noClient, dateCommande$$

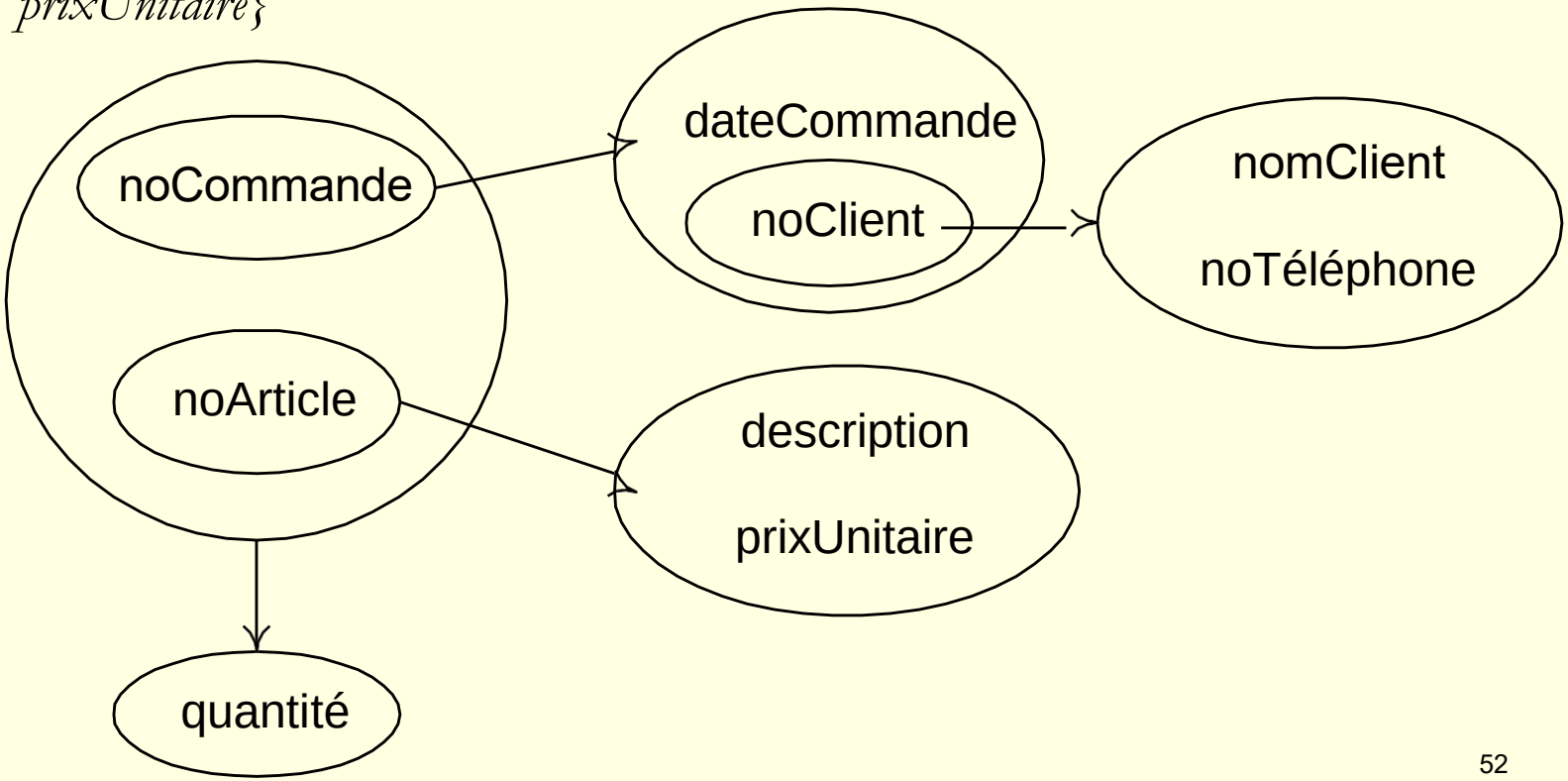
$$noCommande, noArticle \rightarrow quantité$$

$$noArticle \rightarrow description, prixUnitaire$$

Normalisation du schéma relationnel

Représentation par diagramme à bulles

- D = {*noClient* → *nomClient*, *noTéléphone* ; *noCommande* → *noClient*, *dateCommande* ; *noCommande*, *noArticle* → *quantité* ; *noArticle* → *description*, *prixUnitaire*}

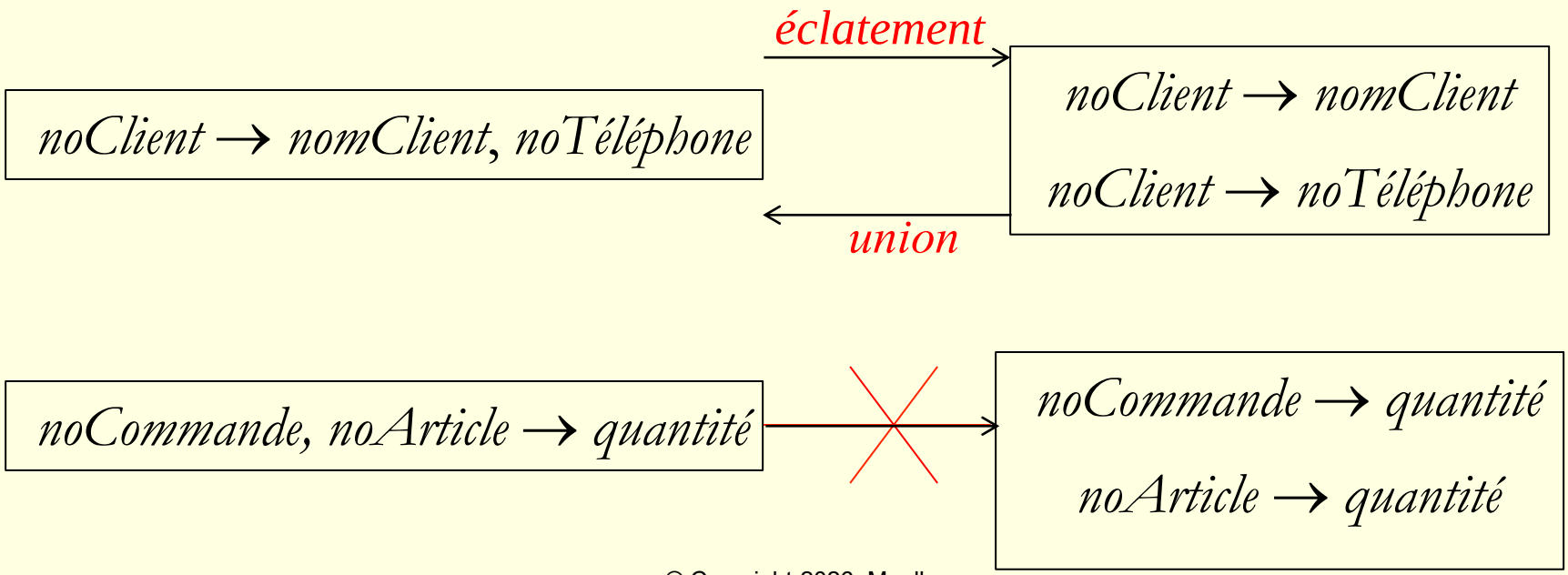


© Copyright 2026, Moulla

Normalisation du schéma relationnel

Représentation par diagramme à bulles

- Équivalences logiques



© Copyright 2026, Moulla

Normalisation du schéma relationnel

Dépendance élémentaire

- **Dépendance fonctionnelle** *complètement non triviale* **:**
  - Aucune colonne du déterminant est répétée à droite

$$noClient \rightarrow noClient, nomClient$$

- **Dépendance fonctionnelle** *pleine* **(« *full* »):**
  - Impossible de retirer une colonne du déterminant sans briser la dépendance

$$noClient, noCommande \rightarrow nomClient$$

- **Dépendance fonctionnelle** *élémentaire* **:**
  - Complètement non-trivial ET pleine

Normalisation du schéma relationnel

Dépendances superflues (redondantes)

$$noClient \rightarrow nomClient, noTéléphone$$

$$noCommande \rightarrow noClient, dateCommande$$

~~$$noCommande \rightarrow nomClient$$~~

Par transitivité

© Copyright 2026, Moulla

Normalisation du schéma relationnel

Axiomes d'Armstrong

- Ensemble minimal d'opérations permettant de construire toute dépendance valide à partir de dépendances connues
- A_1. *Réflexivité*
  - Si $\{B_1, B_2, \ldots, B_m\} \subseteq \{A_1, A_2, \ldots, A_n\}$ alors
  - $A_1, A_2, \ldots, A_n \rightarrow B_1, B_2, \ldots, B_m$
- A_2. *Augmentation*
  - Si $A_1, A_2, \ldots, A_n \rightarrow B_1, B_2, \ldots, B_m$ et
  - $\{D_1, D_2, \ldots, D_p\} \subseteq \{C_1, C_2, \ldots, C_r\}$ alors
  - $A_1, A_2, \ldots, A_n, C_1, C_2, \ldots, C_r \rightarrow B_1, B_2, \ldots, B_m, D_1, D_2, \ldots, D_p$
- A_3. *Transitivité*
  - Si $A_1, A_2, \ldots, A_n \rightarrow B_1, B_2, \ldots, B_m$ et
  - $B_1, B_2, \ldots, B_m \rightarrow C_1, C_2, \ldots, C_r$ alors
  - $A_1, A_2, \ldots, A_n \rightarrow C_1, C_2, \ldots, C_r$

Normalisation du schéma relationnel

Fermeture de X (dénotée X^+)

- Les colonnes pouvant être déduites à partir des colonnes de X, c.-à-dire $X^+ = \{x \mid X \rightarrow x\}$
- Exemples:
  - $\{noClient\}^+ = \{noClient, nomClient, noTéléphone\}$
  - $\{noArticle\}^+ = \{noArticle, description, prixUnitaire\}$
  - $\{prixUnitaire\}^+ = \{prixUnitaire\}$
  - $\{noCommande\}^+ = \{noCommande, noClient, dateCommande, noTéléphone\}$
  - $\{noCommande, noArticle\}^+ = \{noClient, nomClient, noTéléphone, noCommande, dateCommande, noArticle, description, prixUnitaire, quantité\}$

© Copyright 2026, Moulla

Normalisation du schéma relationnel

Calcul de X^+ à l'aide du diagramme à bulles

1. Marquer les colonnes de X dans le diagramme
2. Tant qu'il existe un dépendance $A \rightarrow B$ telle que:
   a) Toutes les colonnes de A sont marquées
   b) Au moins une colonne de B n'est pas marquée

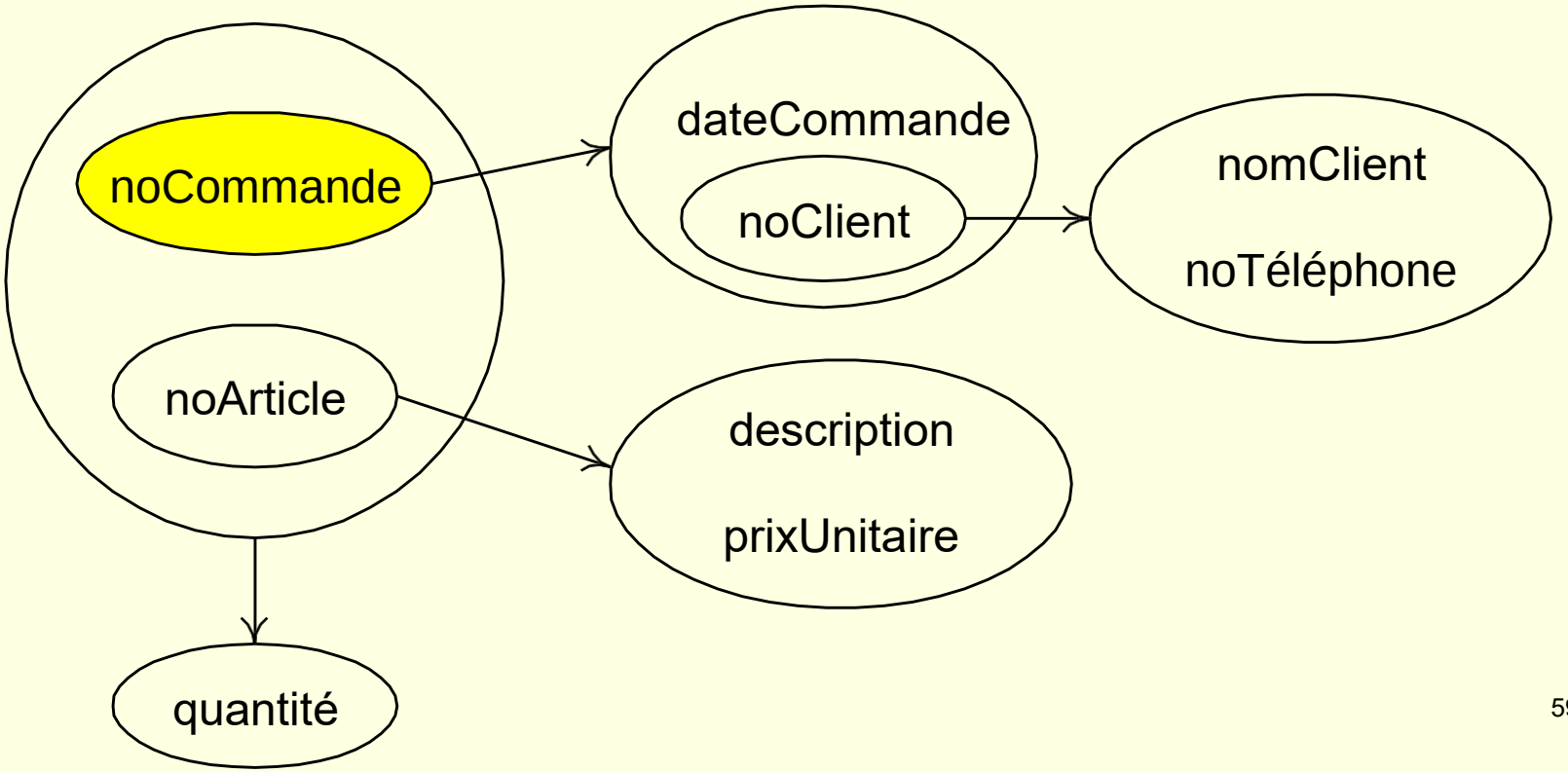
   $\Longrightarrow$ marquer les colonnes de B non déjà marquées
3. X^+ correspond aux colonnes de marquées

© Copyright 2026, Moulla

Normalisation du schéma relationnel

Fonction Fermeture sur diagramme à bulles

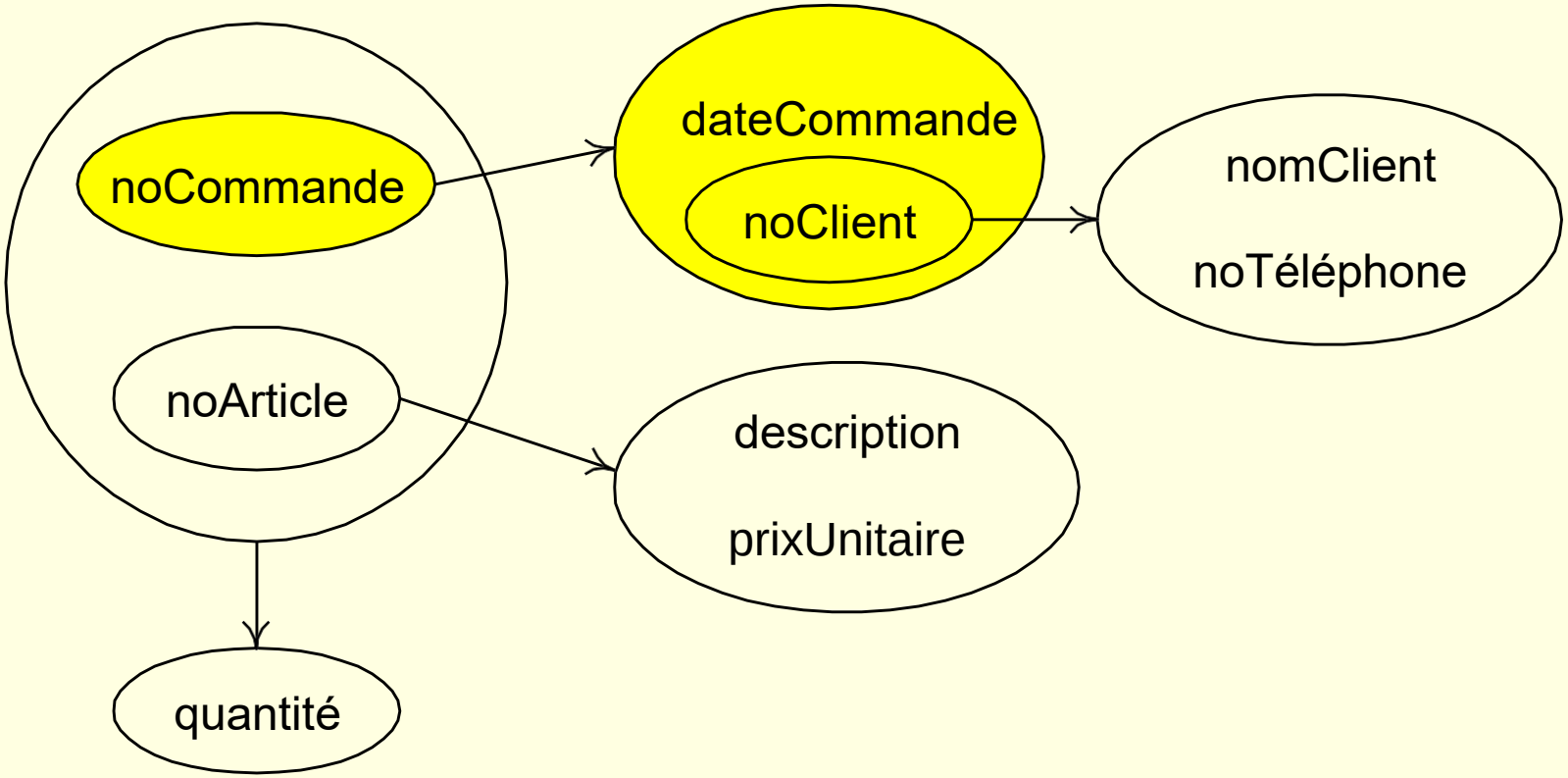
- Exemple : *Fermeture*(*B*, *noCommande*)



© Copyright 2026, Moulla

Normalisation du schéma relationnel

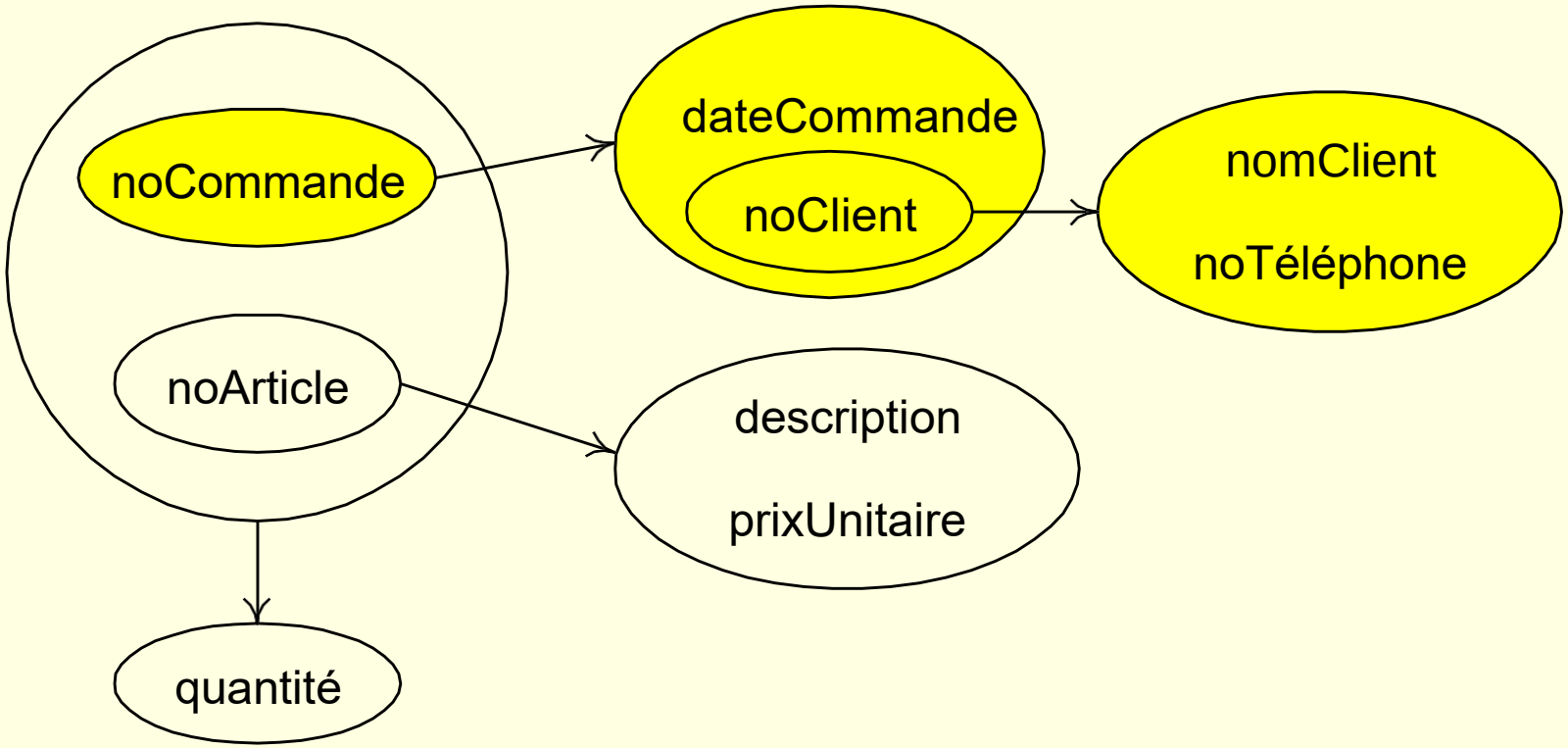
Fonction Fermeture ...



© Copyright 2026, Moulla

Normalisation du schéma relationnel

Fonction Fermeture ...



© Copyright 2026, Moulla

Normalisation du schéma relationnel

1èré forme normale (1NF):

Une relation est en première forme normale si elle ne comporte pas d'attribut multivalué. La représentation des données sous forme de relation implique cette première forme normale.

- Exemple de relation qui n'est pas en 1NF:

LIVRE(CodeL, Titre, Auteurs) n'est pas en première forme normale car un livre pourrait avoir plusieurs auteurs.

Solution: LIVRE(CodeL, Titre), AUTEURS(CodeL, Auteur)

Normalisation du schéma relationnel

2ème forme normale(2NF):

Une relation est en deuxième forme normale si la relation est en première forme normale et si chaque attribut non-clé (c'est-à-dire qui n'appartient pas à une clé candidate) dépend pleinement des clés candidates.

- Exemple de relation qui n'est pas en 2NF:

EMP(Matr, CodeP, NomE, FonctionP, Dept) n'est pas en 2NF puisque le nom d'un employé ou son département ne dépend que de son matricule et pas du projet auquel il appartient.

© Copyright 2026, Moulla

Normalisation du schéma relationnel

2ème forme normale(2NF):

Exemple de relation qui n'est pas en 2NF: **EMP(Matr, CodeP, NomE, FonctionP, Dept)** n'est pas en 2NF puisque le nom d'un employé ou son département ne dépend que de son matricule et pas du projet auquel il appartient.

- Problèmes occasionnés lors de la mise à jour des données : si on modifie le nom d'un employé, on doit le modifier dans toutes les lignes qui correspondent aux divers projets.
- on ne peut insérer un nouvel employé s'il ne participe pas à un projet.
- on risque de supprimer les renseignements sur un employé s'il ne participe plus à aucun projet.
- On peut décomposer cette relation en deux relations qui seront en 2NF

EMP(Matr, NomE, Dept)

PARTICIPATION(Matr, CodeP, FonctionP)

Normalisation du schéma relationnel

3ème forme normale(3NF):

Une relation est en troisième forme normale si tout attribut non-clé (n'appartenant pas à une clé candidate) ne peut dépendre que d'une clé candidate.

- Exemple **EMP(Matr, NomE, Dept, NomD)** où Dept est le numéro du Département n'est pas en 3FN car le nom du département dépend du numéro du département.
- Comme pour la 2NF, on rencontrera des anomalies lors des mises à jour de relations qui ne sont pas en 3NF. On sera par exemple obligé de dupliquer les renseignements concernant les départements.

- On peut décomposer cette relation en deux relations qui seront en 3NF

EMP(Matr, NomE, Dept) et DEPT(Dept, NomD)

© Copyright 2026, Moulla

Normalisation du schéma relationnel

Forme normale de Boyce-Codd (BCNF):

une relation est en forme normale de Boyce-Codd si les seules sources de dépendances fonctionnelles sont les clés candidates.

- Exemple: Si on suppose que le nom d'un employé peut être une clé candidate, la relation **EMP_PROJET(Matr, CodeP, NomE, FonctionP)** est en 3NF. En effet les 2 clés candidates sont (Matr, CodeP) et (NomE, CodeP)
- Le seul attribut qui n'appartient pas à une clé candidate est FonctionP qui ne dépend pas d'un sous-ensemble strict d'une des clés candidates.
- Cette relation n'est pas en BCNF car Matr -->NomE est une dépendance fonctionnelle dont la source n'est pas une clé candidate. Elle va occasionner des redondances et des problèmes de mises à jour. Elle peut être décomposées dans les 2 relations suivantes qui sont en BCNF: **EMP(Matr, NomE) et PROJET(Matr, CodeP, FonctionP)**

© Copyright 2026, Moulla

Gestion de la persistance transparente

Persistance

- Définition: Le stockage d'un objet ou d'une entité du domaine d'application sur disque ou tout autre dispositif de stockage, de manière à être utilisé d'une session à l'autre.
- Assure les propriétés de *durabilité* (ne pas perdre les données en cas de panne) et d'*intégrité* (préserver la cohérence des données malgré des accès/mises à jour concurrents)
- Normalement réalisée à l'aide d'une base de données relationnelle.
- Problème: le domaine d'application est modélisé à l'aide d'objets alors que la BD emploie un modèle relationnel.

© Copyright 2026, Moulla

Gestion de la persistance transparente

Incompatibilité objet vs relationnel

- Granularité:
  - Ex: Une classe adresse ne correspond pas forcément à une table adresse (afin d'éviter la jointure).
- Sous-types (relation de spécialisation):
  - Concept de polymorphisme absent en relationnel;
  - Plusieurs solutions possibles (délégation/fusion/concaténation);
- Identité (équivalence):
  - Valeur ou référence (objet) VS clé primaire (relationnel)
- Associations:
  - Unidirectionnel (objet) VS bidirectionnel (relationnel)
  - Plusieurs à plusieurs nécessite une table de jointure.

© Copyright 2026, Moulla

Gestion de la persistance transparente

Gestion de la persistance

- Deux approches:
  - Matérialisation/dématérialisation à l'aide de JDBC (Java Database Connectivity)
  - Framework de persistance transparente:
    - Object / relationnal mapping ORM (ex: JDO (Java Data Object), Hibernate, EJB3 (Entreprise Java Bean 3))
    - Object / SQL mapping (ex: iBatis/myBatis)

© Copyright 2026, Moulla

Gestion de la persistance transparente

Persistance avec JDBC

- Matérialisation:
  - Rendre/mettre à jour un objet persistant dans la BD
  - Se fait à l'aide d'un ou plusieurs INSERT/ UPDATE
- Dématérialisation:
  - Reconstruire un objet persistant à partir des données de la BD
  - Se fait à l'aide d'un ou plusieurs SELECT

© Copyright 2026, Moulla

Gestion de la persistance transparente

Problèmes d'utiliser JDBC

- Le développement et le maintient de code SQL peut être complexe;
- La portabilité est réduite par le code SQL spécifique au SGBD (ex: séquences Oracle);
- Demande de gérer soi-même le contrôle d'accès, la concurrence des transactions, la mémoire tampon (*cache*), le chargement hâtif, etc.

© Copyright 2026, Moulla

Gestion de la persistance transparente

Mappage Objet - Relationnel (ORM)

- Mappage du domaine d'application à BD relationnelle
  - Classes ↔ tables
  - Attributs ↔ colonnes
  - Relations inter-objets (1-1, 1-plusieurs, etc.)
  - Héritage
- Gestion du cycle de vie des objets;
- Gestion de l'identité des objets.

© Copyright 2026, Moulla

Gestion de la persistance transparente

Mappage Objet - Relationnel (ORM)

- Services des frameworks ORM:
  - Mappage déclaratif entre modèle objet et schéma de BD relationnelle;
  - API pour création, lecture, modification et destruction d'objets;
  - Langage de requête (objet) pour trouver efficacement les objets basé sur des critères;
  - Support pour la gestion de transactions;
  - Chargement hâtif ou tardif (eager/lazy loading);
  - Gestion de cache pour limiter les accès à la BD;
  - Manipulation d'objets "détachés".

Gestion de la persistance transparente

Mappage Objet - Relationnel (ORM)

- Avantages:
  - Productivité:
    - Aucun code SQL / JDBC à écrire;
    - Le découplage de la BD facilite les tests et la maintenance
  - Performance:
    - Gestion avancée de la cache, chargement hâtif, etc.
  - Portabilité:
    - Code SQL généré spécifique au SGDB employé (ex: Oracle)
- Limitations:
  - Pas toujours évident de mapper le domaine d'application au schéma relationnel (ex: systèmes legacy, dénormalisation, etc.);
  - Pas toujours le contrôle sur la définition et l'accès au schéma relationnel (ex: accès par procédures stockées seulement).

Gestion de la persistance transparente

Mappage Objet - SQL

- Utilisé dans les cas où on ne contrôle pas le schéma de la BD (ex: code SQL ultra-optimisé, accès restreint);
- Élimine la dépendance de l'application à la couche de persistance en encapsulant les accès à la BD dans un fichier de configuration externe (XML);
- Le code JDBC est géré par le framework;
- Similaire au preparedStatement de JDBC, mais les paramètres et valeurs de retour sont des objets du domaine d'application.

© Copyright 2026, Moulla

Gestion de la persistance transparente

Framework ORM : Hibernate

- Outil de persistance transparente Java
- Open source
- Améliorations vs EJB 2 et JDO 1
  - Plain Old Java Objects (POJO)
    - Héritage, associations par attributs Java
  - Langage de requête HQL plus proche de SQL
  - Pas de manipulation de code (introspection Java)
- Support de l'API de persistance Java de la nouvelle norme EJB3 (JSR220)

© Copyright 2026, Moulla

Gestion de la persistance transparente

Optimisation de la performance

- Verrouillage optimiste (*optimistic locking*)
  - Accélère le traitement de transactions concurrentes en évitant de bloquer les ressources;
  - En cas de conflit, la transaction gagnante est celle qui a écrit en premier;
  - Verrouillage à l'aide d'une version;
  - Peut également se faire à l'aide d'une estampille (*timestamp*);
  - Le verrouillage d'une table, d'un attribut ou d'une relation peut être contrôlé à l'aide du paramètre *optimistic-lock*.

Paradigme relationnel-objet

Le modèle relationnel-objet (OR)

- Ajoute quelques notions au modèle relationnel, qui comblent ses plus grosses lacunes;
- Compatibilité ascendante : les applications relationnelles fonctionnent dans le monde OR;
- La norme SQL:1999 (SQL3) reprend beaucoup d'idées du modèle OR;
- Diffère des bases de données objet (SGBDO)
  - Le monde de l'application est distinct de celui de la BD;
  - Il faut quand même créer l'interface entre ces deux mondes (ex: JDBC).

© Copyright 2026, Moulla

Paradigme relationnel-objet

Pourquoi le relationnel-objet ?

- Simplicité et performance
  - La reconstitution d'objets complexes éclatés sur plusieurs tables relationnelles est coûteuse car elle occasionne de nombreuses jointures.
  - Pour échapper aux éclatements-jointures, l'OR introduit
    - Les références, qui permettent d'implanter des structures complexes;
    - Les attributs multivalués (tableaux, ensembles ou listes).

© Copyright 2026, Moulla

Paradigme relationnel-objet

Pourquoi le relationnel-objet ?

- Rapprochement au monde objet (applications)
  - Permet de définir de nouveaux types utilisateur simples ou complexes (*User data type*)
  - Permet d'encapsuler les opérations sur les données avec les données (fonctions, procédures)
  - Supporte l'héritage de type pour profiter du polymorphisme et faciliter la réutilisation
- Données multimédia volumineuses
  - Facilite le stockage et la manipulation de données multimédia volumineuses en permettant leur partage simplement et à moindre coût (sans jointure)

Paradigme relationnel-objet

Pourquoi ne pas passer directement aux SGBD Objet ?

- Inertie de l'existant: de très nombreuses bases relationnelles en fonctionnement;
- Manque de normalisation pour les SGBDO / trop de solutions propriétaires;
- Moins souple que le relationnel pour s'adapter à plusieurs applications et à l'augmentation de charge;
- Peu d'informaticiens formés aux SGBDO;
- Le modèle OR peut permettre un passage en douceur.

© Copyright 2026, Moulla

Paradigme relationnel-objet

Nouvelles possibilités de l'OR

- Définir de nouveaux types complexes avec des fonctions pour les manipuler;
- Une colonne peut contenir une collection (ensemble, sac, liste);
- Ligne considérée comme un objet, avec un identificateur (Object Identifier OID);
- Utilisation de références aux objets;
- Extensions du langage SQL (SQL3 ou SQL99) pour la recherche et la modification des données;

Paradigme relationnel-objet

Les problèmes de l'OR

- Ne s'appuie pas sur une théorie solide comme le modèle relationnel;
- Manque de standard de fait : implantations différentes, et encore partielles, dans les divers SGBDs.

© Copyright 2026, Moulla

Paradigme relationnel-objet

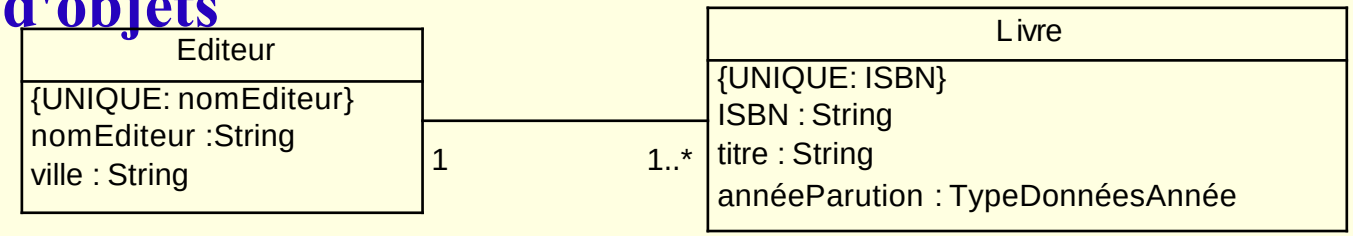
Types définis par l'utilisateur (User-Defined-Type)

- Équivalent à une classe en OO;
- Un UDT possède
  - Des attributs (simples ou autres types);
  - Des méthodes;
- Utilisations d'un UDT
  - Comme colonne d'une table ordinaire;
  - Comme attribut d'un autre type;
  - Comme une ligne d'une table d'objets.

© Copyright 2026, Moulla

Paradigme relationnel-objet

Traduction d'une classe d'objets UML persistante par un type et une TABLE d'objets




```
  CREATE TYPE EditeurType  AS OBJECT
(nomEditeur              VARCHAR(20),
   ville                 VARCHAR(20)
    )

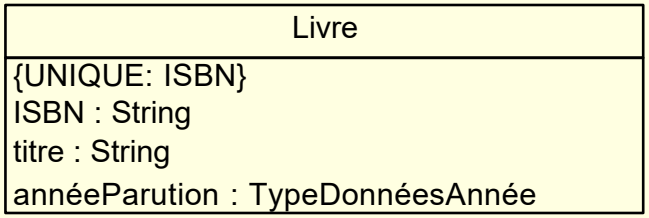
CREATE TYPE LivreType AS OBJECT
 (ISBN                  CHAR(13),
 titre                 VARCHAR(50),
 anneeParution         TypeDonnéesAnnée,
  éditeur               REF EditeurType
                              )

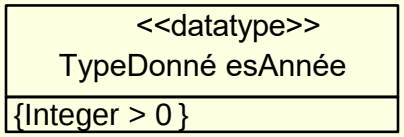
CREATE TABLE Editeur OF EditeurType
     (PRIMARY KEY(nomEditeur))

CREATE TABLE Livre OF LivreType
      (PRIMARY KEY(ISBN),
CONSTRAINT anneeSup0 CHECK(anneeParution.valeurAnnée > 0),
CONSTRAINT referenceTableEditeur éditeur SCOPE IS Editeur)
```

Paradigme relationnel-objet

Type de valeurs d'un attribut ou colonne






```
    CREATE TYPE TypeDonnéesAnnée AS
     OBJECT  (valeurAnnée INTEGER)

    CREATE TYPE LivreType AS OBJECT
     (ISBN                CHAR(13),
      titre          VARCHAR(50),
 anneeParution            TypeDonnéesAnnée,
    éditeur          REF EditeurType
       )

CREATE TABLE Livre OF LivreType
      (PRIMARY KEY(ISBN),
CONSTRAINT anneeSup0 CHECK(anneeParution.valeurAnnée > 0),
CONSTRAINT referenceTableEditeur éditeur SCOPE IS Editeur)
```

Paradigme relationnel-objet

Les références (REF)

- Pointe vers un objet du type correspondant;
- Renferme un identifiant objet (OID) généré soit
  1. Automatiquement (défaut)
     - indexation efficace
     - potentiellement moins lourd qu'avec la clé primaire
  2. À partir de la clé primaire

```
CREATE TABLE NomTable OF NomType (
                    (
                        …
 PRIMARY KEY(…)
) OBJECT IDENTIFIER IS PRIMARY KEY
```

Obligatoire

Paradigme relationnel-objet

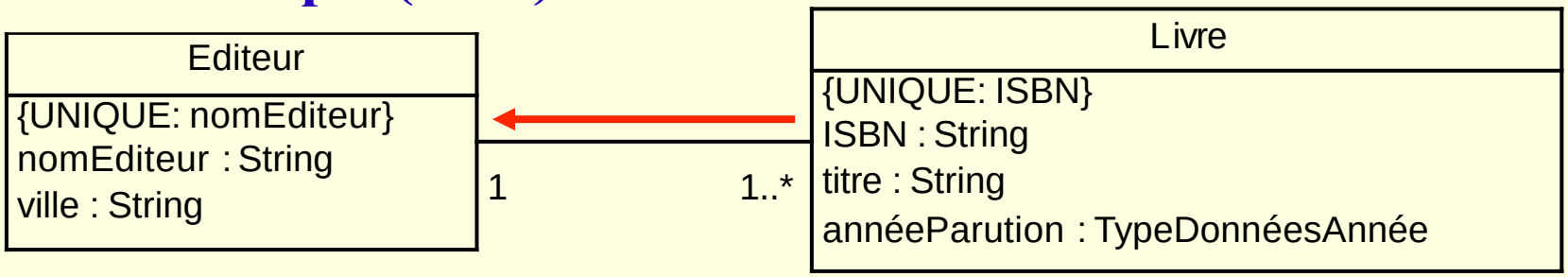
Les références (REF)

- REF vs clé étrangère:
  - Permet de représenter une relation 1 à plusieurs à l'aide d'une collection de références (jointure si clé étrangère);
  - Peut pointer dans le vide si l'objet est supprimé;
  - Peut pointer sur des objets stockés dans des tables différentes.

© Copyright 2026, Moulla

Paradigme relationnel-objet

Traduction d'une association un à plusieurs UML par référence simple (REF)

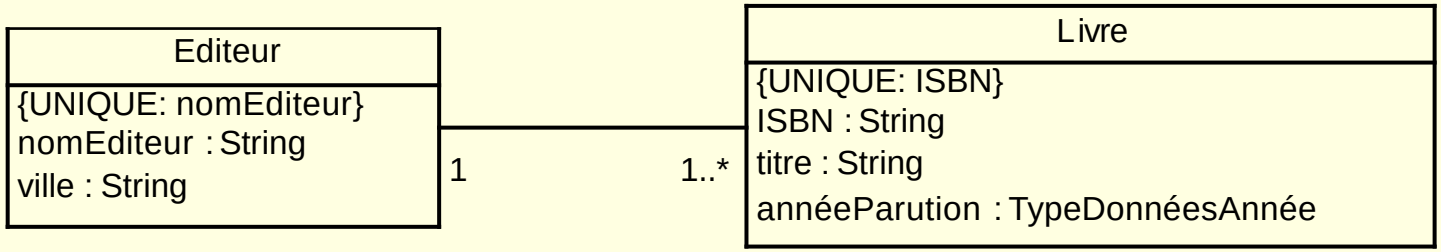



```
 CREATE TYPE LivreType AS OBJECT
(ISBN                          CHAR(13),
 titre                         VARCHAR(50),
 annéeParution                 DomaineAnnée,
 éditeur                       REF EditeurType,
                              )
```

© Copyright 2026, Moulla

Paradigme relationnel-objet

Contrainte SCOPE IS

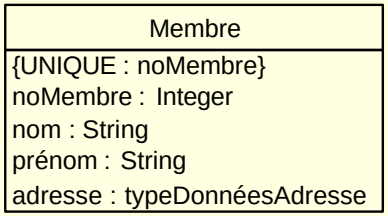


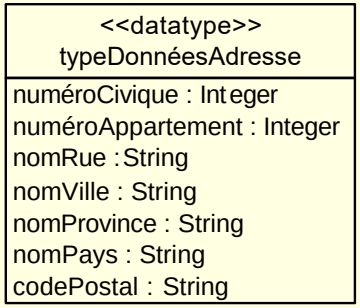

```
CREATE TYPE LivreType AS OBJECT
(ISBN                       CHAR(13),
 titre                  VARCHAR(50),
 anneeParution          DomaineAnnée,
 éditeur                REF EditeurType,
)
```

```
CREATE TABLE Livre OF LivreType
(PRIMARY KEY(ISBN),
CONSTRAINT anneeSup0 CHECK(anneeParution.valeurAnnée > 0),
CONSTRAINT referenceTableEditeur éditeur SCOPE IS Editeur)
```

Paradigme relationnel-objet

Représentation d'un type de données complexe ou d'une composition UML par un UDT






```
CREATE TYPE TypeDonnéesAdresse AS OBJECT
(noCivique              INTEGER,
 numéroAppartement      INTEGER,
 nomRue                 VARCHAR(20),
 nomVille               VARCHAR(20),
 nomProvince            VARCHAR(20),
 nomPays                VARCHAR(20),
 codePostal             VARCHAR(10)
        )

 CREATE TYPE MembreType AS OBJECT
     (noMembre          INTEGER,
    nom               VARCHAR(20),
    prénom            VARCHAR(20),
 adresse          typeDonnéesAdresse
                   )
```

Intégrité et gestion des transactions

L'intégrité et la gestion des transactions

- 1. Intégrité sémantique
  - respect des contraintes d'intégrité;
- 2. Fiabilité (sûreté)
  - fautes (pannes) matérielles ou logicielles;
- 3. Contrôle de concurrence
  - concurrence sans interférence indésirable;
- 4. Sécurité (confidentialité)
  - accès autorisés.

© Copyright 2026, Moulla

Intégrité et gestion des transactions

Pseudo-code d'une transaction de transfert bancaire

```
DébutTransaction ;
Lire(SoldeCompte1, s1);
s1:= s1-100;
Écrire(s1, SoldeCompte1);
SI s1 < 0
      AnnulerTransaction;
SINON
      Lire(SoldeCompte2, s2);
      s2:= s2+100;
      Écrire(s2, SoldeCompte2);
      ConfirmerTransaction;
FINSI;
```

© Copyright 2026, Moulla

Intégrité et gestion des transactions

Propriétés d'une transaction (ACID)

- Atomicité (atomicity)
  - tout ou rien

Scénario de transfert interrompu

Opération	**État de la mémoire centrale**	**État de la BD**
DébutTransaction		$SoldeCompte_1 = \$300$ $SoldeCompte_2 = \$500$
Lire($SoldeCompte_1$, s_1)	s_1=300	
$s_1 := s_1$-100	s_1=200	
Écrire(s_1, $SoldeCompte_1$)		$SoldeCompte_1 = \$200$ $SoldeCompte_2 = \$500$
Interruption		**Olala !**

Intégrité et gestion des transactions

Propriétés d'une transaction (ACID)

- Consistance (consistency) ou cohérence
  - Chaque transaction doit laisser la BD dans un état cohérent
  - Les contraintes d'intégrité doivent être respectées à la fin

© Copyright 2026, Moulla

Intégrité et gestion des transactions

Propriétés d'une transaction (ACID)

- Isolation
  - Chaque transaction doit être isolée des effets indésirables des transactions concurrentes
  - *Exemple de perte de mise à jour* (*lost update*)

Temps	Transaction T_1	Transaction T_2	État de la BD
t_1	**DébutTransaction**		*SoldeCompte* = 1000
t_2	**Lire**(*SoldeCompte*, *s*)		
t_3		**DébutTransaction**	
t_4		**Lire**(*SoldeCompte*, *s*)	
t_5	*s* = *s* + *700*		
t_6	**Écrire**(*s*, *SoldeCompte*)		*SoldeCompte* = 1700
t_7	**ConfirmerTransaction**		
t_8		*s* = *s* + *200*	
t_9		**Écrire**(*s*, *SoldeCompte*)	*SoldeCompte* = 1200
t_{10}		**ConfirmerTransaction**	**Saperlipopette!**

Intégrité et gestion des transactions

Exemple de lecture impropre (dirty read) : cas de sommaire inconsistant

Temps	Transaction T_1	Transaction T_2	État de la BD	Environnement
t_1	**DébutTransaction**		$SoldeCompte_1 = \$300$ $SoldeCompte_2 = \$500$	
t_2	**Lire**($SoldeCompte_1$, s_1)			
t_3	$s_1 := s_1 - 100$			
t_4	**Écrire**(s_1, $SoldeCompte_1$)		$SoldeCompte_1 = \$200$ $SoldeCompte_2 = \$500$	
t_5		**DébutTransaction**		
t_6		**Lire**($SoldeCompte_1$, s_1)		
t_7		**Lire**($SoldeCompte_2$, s_2)		
t_8		$total = s_1 + s_2$		
t_9		*Afficher*(s_1, s_2, *total*)		Affiche \$200, \$500, \$700 **Oups!**
t_{10}		**ConfirmerTransaction**		
t_{11}	**Lire**($SoldeCompte_2$, s_2)			
t_{12}	$s_2 := s_2 + 100$			
t_{12}	**Écrire**(s_2, $SoldeCompte_2$)		$SoldeCompte_1 = \$200$ $SoldeCompte_2 = \$600$	
t_{14}	**ConfirmerTransaction**			

Lecture impropre : lecture d 'une donnée « non confirmée »

Intégrité et gestion des transactions

Propriétés d'une transaction (ACID)

- Durabilité (durability)
  - Les effets d'une transaction confirmée doivent durer malgré les pannes;
  - Le SGBD doit pouvoir *rétablir* les effets des transactions qui se sont terminées normalement (commit).
- Légalité
  - Respecter les privilèges.

© Copyright 2026, Moulla

Intégrité et gestion des transactions

Contrôle de concurrence

- Méthode de contrôle de concurrence
  - Assure la propriété d'isolation des transactions
  - Empêche les effets indésirables des transactions concurrentes
- Ordonnancement, historique, ou exécution (schedule, history, log)
  - Exemple d'un ordonnancement transactionnel:

Temps	Transaction T_1	Transaction T_2	État de la BD
t_1	**DébutTransaction**		*SoldeCompte* = 1000
t_2	**Lire**(*SoldeCompte*, *s*)		
t_3		**DébutTransaction**	
t_4		**Lire**(*SoldeCompte*, *s*)	
t_5	*s* = *s* + *700*		
t_6	**Écrire**(*s*, *SoldeCompte*)		*SoldeCompte* = 1700
t_7	**ConfirmerTransaction**		
t_8		*s* = *s* + *200*	
t_9		**Écrire**(*s*, *SoldeCompte*)	*SoldeCompte* = 1200
t_{10}		**ConfirmerTransaction**	**Saperlipopette!**

Intégrité et gestion des transactions

Ordonnancement séquentiel (serial schedule)

- L'une après l'autre

Temps	Transaction T_1	Transaction T_2	État de la BD
t_1	**DébutTransaction**		*SoldeCompte* = 1000
t_2	**Lire**(*SoldeCompte*, *s*)		
t_3	*s* = *s* + *700*		
t_4	**Écrire**(*s*, *SoldeCompte*)		*SoldeCompte* = 1700
t_5	**ConfirmerTransaction**		
t_6		**DébutTransaction**	
t_7		**Lire**(*SoldeCompte*, *s*)	
t_8		*s* = *s* + *200*	
t_9		**Écrire**(*s*, *SoldeCompte*)	*SoldeCompte* = 1900
t_{10}		**ConfirmerTransaction**	

- Garantit la propriété d'isolation
- Mais pas pratique:
  - Plusieurs petites transactions peuvent être « bloquées » par une très longue transaction

Intégrité et gestion des transactions

Ordonnancement sérialisable (serializable)

- Ordonnancement qui n'est pas forcément séquentiel, mais est *équivalent* à un ordonnancement séquentiel

Temps	Transaction T_1	Transaction T_2	État de la BD	Environnement
t_1	**DébutTransaction**		$SoldeCompte_1$ = \$300 $SoldeCompte_2$ = \$500	
t_2	**Lire**($SoldeCompte_1$, s_1)			
t_3	s_1:= s_1-100			
t_4		**DébutTransaction**		
t_5		**Lire**($SoldeCompte_1$, s_1)		
t_6		**Lire**($SoldeCompte_2$, s_2)		
t_7		$total = s_1 + s_2$		
t_8		*Afficher*(s_1, s_2, *total*)		Affiche \$300, \$500, \$800
t_9		**ConfirmerTransaction**		
t_{10}	**Écrire**(s_1, $SoldeCompte_1$)		$SoldeCompte_1$ = \$200 $SoldeCompte_2$ = \$500	
t_{11}	**Lire**($SoldeCompte_2$, s_2)			
t_{12}	s_2:= s_2+100			
t_{13}	**Écrire**(s_2, $SoldeCompte_2$)		$SoldeCompte_1$ = \$200 $SoldeCompte_2$ = \$600	
t_{14}	**ConfirmerTransaction**			

Intégrité et gestion des transactions

Exemple non-sérialisable: Perte de mise à jour

- Pas équivalent ni à T_1, T_2 ou T_2, T_1
  - solde : $1900

Temps	Transaction T_1	Transaction T_2	État de la BD
t_1	**DébutTransaction**		*SoldeCompte* = 1000
t_2	**Lire**(*SoldeCompte*, *s*)		
t_3		**DébutTransaction**	
t_4		**Lire**(*SoldeCompte*, *s*)	
t_5	*s* = *s* + *700*		
t_6	**Écrire**(*s*, *SoldeCompte*)		*SoldeCompte* = 1700
t_7	**ConfirmerTransaction**		
t_8		*s* = *s* + *200*	
t_9		**Écrire**(*s*, *SoldeCompte*)	*SoldeCompte* = 1200
t_{10}		**ConfirmerTransaction**	**Saperlipopette!**

Intégrité et gestion des transactions

Enjeux du contrôle de concurrence

- Solution bestiale
  - Interdire les ordonnancements concurrents
    - mécanisme de section critique;
  - Une transaction longue bloque toutes les autres…
- Objectifs
  - Permettre ordonnancements concurrents sans nuire à la performance;
  - Gérer les accès concurrents aux mêmes données.

© Copyright 2026, Moulla

Intégrité et gestion des transactions

Sérialisabilité par permutation

- Sérialisabilité difficile à vérifier en pratique
- Exemple : ordonnancement chanceux !

Temps	Transaction T_1	Transaction T_2	État de la BD
t_1	**DébutTransaction**		$A = 1$
t_2	**Lire**(A, a)		
t_3	$a := a^2$		
t_4		**DébutTransaction**	
t_5		**Lire**(A, a)	
t_6	**Écrire**(a, A)		$A = 1$
t_7		$a := a + 10$	
t_8		**Écrire**(a, A)	$A = 11$
t_9	**ConfirmerTransaction**		
t_{10}		**ConfirmerTransaction**	

© Copyright 2026, Moulla

Intégrité et gestion des transactions

Sérialisabilité par permutation

- Ne considère que les opérations de lecture et d'écriture
- Paire d'opérations non permutables (*conflicting*):
  1. Les opérations sont dans deux transactions différentes
  2. Les opérations portent sur la même donnée (ex: même ligne d'une table)
  3. Au moins une des deux opérations est une écriture (ex: lire-écrire ou écrire-lire ou écrire-écrire)
- L'ordre d'une paire d'opérations correspond à leur ordre d'exécution
  - Ex: $O_1 \rightarrow O_2$ signifie que O_1 s'exécute avant O_2

Intégrité et gestion des transactions

Sérialisabilité par permutation

- Ordonnancement sérialisable par permutation (*conflict serializable schedule*):
  - Toutes les opérations non permutables sont effectuées dans le même ordre que dans un ordonnancement séquentiel
- Méthode pour déterminer la sérialisabilité par permutation:
  1. Créer un graphe contenant un sommet par transaction
  2. Pour chaque paire d'opérations non-permutables $O_1 \rightarrow O_2$ dessiner un arc allant de la transaction contenant O_1 vers la transaction contenant O_2
  3. Si le graphe résultant ne comporte **aucun** cycle:
     - La transaction est sérialisable par permutation
     - L'ordre séquentiel équivalent correspond à un chemin dans le graphe

Intégrité et gestion des transactions

Sérialisable par permutation : flèches dans la même direction

Temps	Transaction T_1	Transaction T_2	État de la BD	Environnement
t_1	**DébutTransaction**		$SoldeCompte_1 = \$300$ $SoldeCompte_2 = \$500$	
t_2	**Lire**($SoldeCompte_1$, s_1)			
t_3	$s_1 := s_1 - 100$			
t_4		**DébutTransaction**		
t_5		**Lire**($SoldeCompte_1$, s_1)		
t_6		**Lire**($SoldeCompte_2$, s_2)		
t_7		$total = s_1 + s_2$		
t_8		*Afficher*(s_1, s_2, *total*)		Affiche \$300, \$500, \$800
t_9		**ConfirmerTransaction**		
t_{10}	**Écrire**(s_1, $SoldeCompte_1$)		$SoldeCompte_1 = \$200$ $SoldeCompte_2 = \$500$	
t_{11}	**Lire**($SoldeCompte_2$, s_2)			
t_{12}	$s_2 := s_2 + 100$			
t_{13}	**Écrire**(s_2, $SoldeCompte_2$)		$SoldeCompte_1 = \$200$ $SoldeCompte_2 = \$600$	
t_{14}	**ConfirmerTransaction**			

Intégrité et gestion des transactions

Principales stratégies de contrôle de concurrence

- Verrouillage (*locking*)
  - Bloque temporairement l'accès à une ressource durant son utilisation par une transaction;
  - Politique du premier arrivé, premier servi;
  - Approche pessimiste (souvent plus contraignant que nécessaire);
  - Garantit la sérialisabilité (sauf en cas d'inter-blocages).

- Estampillage (timestamping)
  - Impose aux opérations non-permutables de suivre l'ordre des estampilles des transactions (numéro de version ou temps de début des transactions).
  - Annulation d'une transaction en cas de conflit;
  - Approche moins pessimiste que verrouillage;
  - Peut entraîner l'annulation de transactions non dangereuses.

Intégrité et gestion des transactions

Principales stratégies de contrôle de concurrence

- Certification (validation)
  - On laisse les transactions s'exécuter (écritures faites localement) et, avant de confirmer, on vérifie les conflits à l'aide de l'ordre des estampilles;
  - Si aucun conflit, on propage les écritures dans la BD. Sinon, on annule la dernière transaction impliquée dans le conflit;
  - Approche ultra-optimiste.
- Multi-version
  - Chaque transaction écrit dans sa propre version des données;
  - Les lectures sont faites dans des versions récentes;
  - Moins restrictif: évite le blocage des ressources, réduit les annulations;
  - Plus difficile à gérer.

© Copyright 2026, Moulla

Intégrité et gestion des transactions

Contrôle de concurrence par verrouillage

- *Modes de verrouillage*
  - *Verrouillage exclusif* (*X, exclusive lock*)
    - Une transaction à la fois
    - En écriture
  - *Verrouillage partagé* (*P, share lock* (*S*))
    - En lecture

© Copyright 2026, Moulla

Intégrité et gestion des transactions

Verrouillage en deux phases

- Le verrouillage simple des ressources **ne garantit pas** la sérialisabilité;
- Protocole de verrouillage en deux phases (V2P, *two phase locking*)
  - *Ver*(*B*,*P*) ou *Ver*(*B*,*X*) avant *Lire*(*B*);
  - *Ver*(*B*,*X*) avant *Écrire*(*B*);
  - Aucun verrouillage (*Ver*) après un déverrouillage (*Dev*).
- V2P $\Rightarrow$ sérialisable par permutation.

© Copyright 2026, Moulla

Intégrité et gestion des transactions

Contrôle de concurrence par estampillage

- Permet des ordonnancements impossibles avec le V2P
- Principe
  - Si $O_1 \rightarrow O_2$ non permutables (O_1 dans T_1 et O_2 dans T_2) alors
    - Opérations exécutées dans l'ordre correspondant aux estampilles des transactions: OK, si Estampille(T_1) < Estampille(T_2)
  - Sinon, zigouiller requérant (transaction T_2 dans ce cas)
- Ordre séquentiel correspond à l'ordre des estampilles
- Information à maintenir par SGBD
  - *Estampille-L*(*B*): Estampille maximale des transactions ayant lu *B*
  - *Estampille-E*(*B*): Estampille maximale des transactions ayant écrit *B*
  - Seulement les plus récentes

Intégrité et gestion des transactions

Contrôle de concurrence par estampillage

- Lecture d'un objet B par une transaction T:

```
SI Estampille(T) < Estampille-E(B) ALORS
     Annuler la transaction T
SINON
     Effectuer la lecture
     Estampille-L(B) := Estampille(T)
FIN SI
```

- Écriture d'un objet B par une transaction T:

```
SI  Estampille(T)  <  Estampille-L(B)  OU
   Estampille(T) < Estampille-E(B) ALORS
     Annuler la transaction T
SINON
     Effectuer l'écriture
     Estampille-E(B) := Estampille(T)
FIN SI
```

Intégrité et gestion des transactions

Contrôle de concurrence par certification

- Phase 1: lecture
  - Écritures locales sans interruption
- Phase 2: certification
  - Lorsque transaction se termine
    - Vérifie conflit par rapport à l'ordre des estampilles de fin
- Phase 3: confirmation
  - Si certifiée: les écritures sont faites
  - Sinon: transaction annulée

© Copyright 2026, Moulla

Intégrité et gestion des transactions

Méthode multi-version

- Principe
  - Chaque opération d'écriture produit une nouvelle version de l'objet écrit, ou une annulation;
  - En lecture, il faut choisir la bonne version.
- Avantage:
  - Aucun blocage, peu d'annulations;
- Inconvénients:
  - Données peuvent être légèrement décalées par rapport à l'état courant;
  - Doit gérer possiblement plusieurs version d'une même donnée (quoi que déjà fait pour la récupération).

Intégrité et gestion des transactions

Contrôle de concurrence en SQL

- Les niveaux d'isolation d 'une transaction en SQL
  - 0. Lecture impropre (READ UNCOMMITTED)
    - Permet lecture impropre (ex: Ver(X) en écriture mais non strict)
  - 1. Lecture propre (READ COMMITTED)
    - Interdit lecture impropre (ex: V2P strict)
  - 2. Lecture reproductible (REPEATABLE READ)
    - Interdit lecture impropre
    - Garantit relecture sans modifications visibles à l'intérieur d'une même transaction (V2P rigoureux)
    - Permet fantômes (lignes insérées par d'autres transactions, pendant l'exécution)
  - 3. Sérialisable (SERIALIZABLE)

Intégrité et gestion des transactions

Contrôle de concurrence dans Oracle

- Combine les approches de verrouillage et d'estampillage multi-version
- Permet uniquement READ COMMITTED (défaut), et SERIALIZABLE et READ ONLY
- READ COMMITTED
  - Lorsque le nombre de transactions concurrentes est élevé et que le temps de réponse doit être court;
  - Lorsqu'il n'est pas nécessaire de relire la même valeur au cours de la transaction;
  - Lorsque le risque de données fantômes est faible.

Intégrité et gestion des transactions

Contrôle de concurrence dans Oracle

- SERIALIZABLE:
  - Lorsque le risque d'avoir des enregistrements fantômes est important (ex: transaction qui fait un rapport d'une table lorsque de nouvelles lignes sont ajoutées à la table);
  - Lorsqu'il est nécessaire de relire plusieurs fois certaines lignes, sans changement;
  - Demande de faire la gestion des exceptions: « *Cannot serialize access* ».
  - Problématique pour les longues transactions qui doivent mettre à jour des lignes accédées par un grand nombre de courtes transactions (annulations répétées).

Intégrité et gestion des transactions

Types de fautes

- Faute de transaction (*transaction failure*)
  - Faute de logique;
  - Faute attribuable au SGBD;
  - Annulation volontaire;
- Panne système (*system crash*);
- Panne de mémoire secondaire (*secondary memory failure*)

© Copyright 2026, Moulla

Organisation des données en mémoire

Critères d'évaluation des méthodes d'organisation

- Temps d'accès aux données par rapport à différentes méthodes d'accès (balayage, sélection, ...);
- Délai d'insertion et de suppression;
- Occupation mémoire;
- Les critères sont en compétition:
  - Ajouter un index peut accélérer la sélection mais il est souvent plus rapide de faire un balayage;
  - Un index doit être mis à jour lors d'une insertion ou d'une suppression;
  - Les index d'un schéma peuvent prendre autant d'espace que les données du schéma.

© Copyright 2026, Moulla

Organisation des données en mémoire

Conception physique

- Facteurs à considérer:
  - Taille des données et leur croissance;
  - Taille disponible de stockage;
  - Requêtes fréquentes;
  - ...
- Choix de conception:
  - Paramètres de stockage;
  - Index de données;
  - Partitionnement;
  - Compression;
  - ...

© Copyright 2026, Moulla

Organisation des données en mémoire

Organisation physique des enregistrements

- Organisation séquentielle
  - Les enregistrements sont ordonnés physiquement selon l'ordre d'insertion ou par rapport à une clé de tri;
  - Facilite l'accès séquentiel des enregistrements selon la clé;
  - Gestion complexe de la mémoire en cas de mises-à-jours (ex: suppressions).

© Copyright 2026, Moulla

Organisation des données en mémoire

Organisation physique des enregistrements

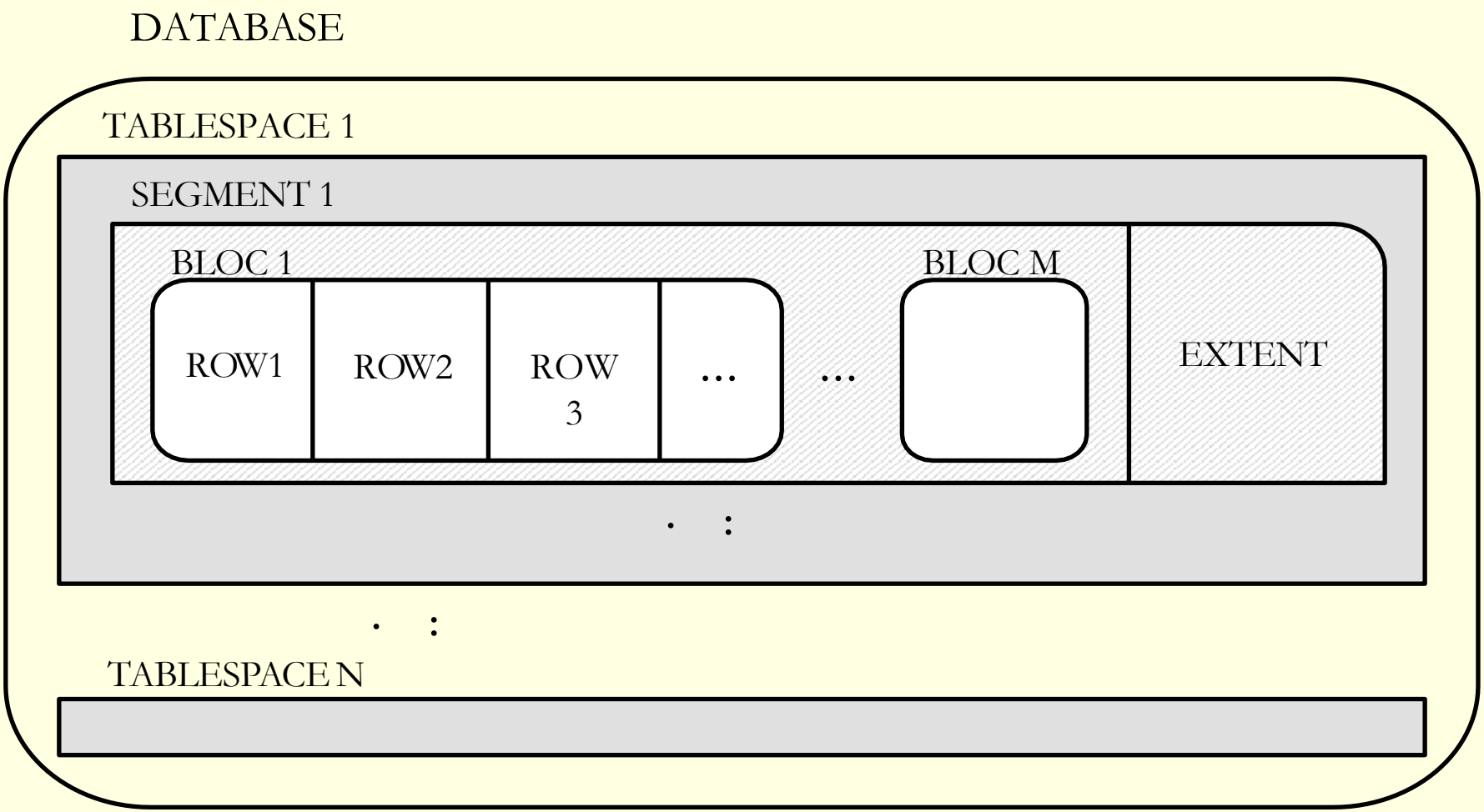
- Organisation sérielle (*heap*):
  - Les enregistrements sont placés sans ordre particulier, où il y a de l'espace;
  - Adressage à l'aide d'un identifiant d'enregistrement (IDE)

    Ex: ROWID Oracle: (#fichier, #bloc, #enregistrement)
  - Gestion simplifiée de la mémoire;
  - Organisation par défaut dans la plupart des BD (ex: Oracle)

© Copyright 2026, Moulla

Organisation des données en mémoire

Structure de stockage en Oracle



Organisation des données en mémoire

Structure de stockage en Oracle

- DATABASE:
  - Données, journalisation, fichiers de contrôle (*control file*), etc.
  - Ensemble de TABLESPACE

```
CREATE DATABASE nomDatabase  ...
```

- TABLESPACE:
  - Ensemble de DATAFILE
  - Définition des paramètres de stockage
  - Exemples:
    - USERS : Contient les tables des utilisateurs (créé par défaut)
    - SYSTEM : Contient le dictionnaire de données

© Copyright 2026, Moulla

Organisation des données en mémoire

Structure de stockage en Oracle

- SEGMENT
  - Associé à une table, index, etc.
  - Peut être réparti sur plusieurs fichiers.
- EXTENT
  - Unité d'allocation d'espace composée de blocs contigus;
  - Un EXTENT est ajouté si un SEGMENT manque d'espace.
- BLOC
  - Unité minimale de transfert entre la mémoire secondaire (disque dur) et la mémoire centrale;
  - Contient un ensemble d'enregistrements.
- Enregistrement
  - Ligne d'une table, clé d'un index, etc.

Organisation des données en mémoire

Compression des tables

- Syntaxe:

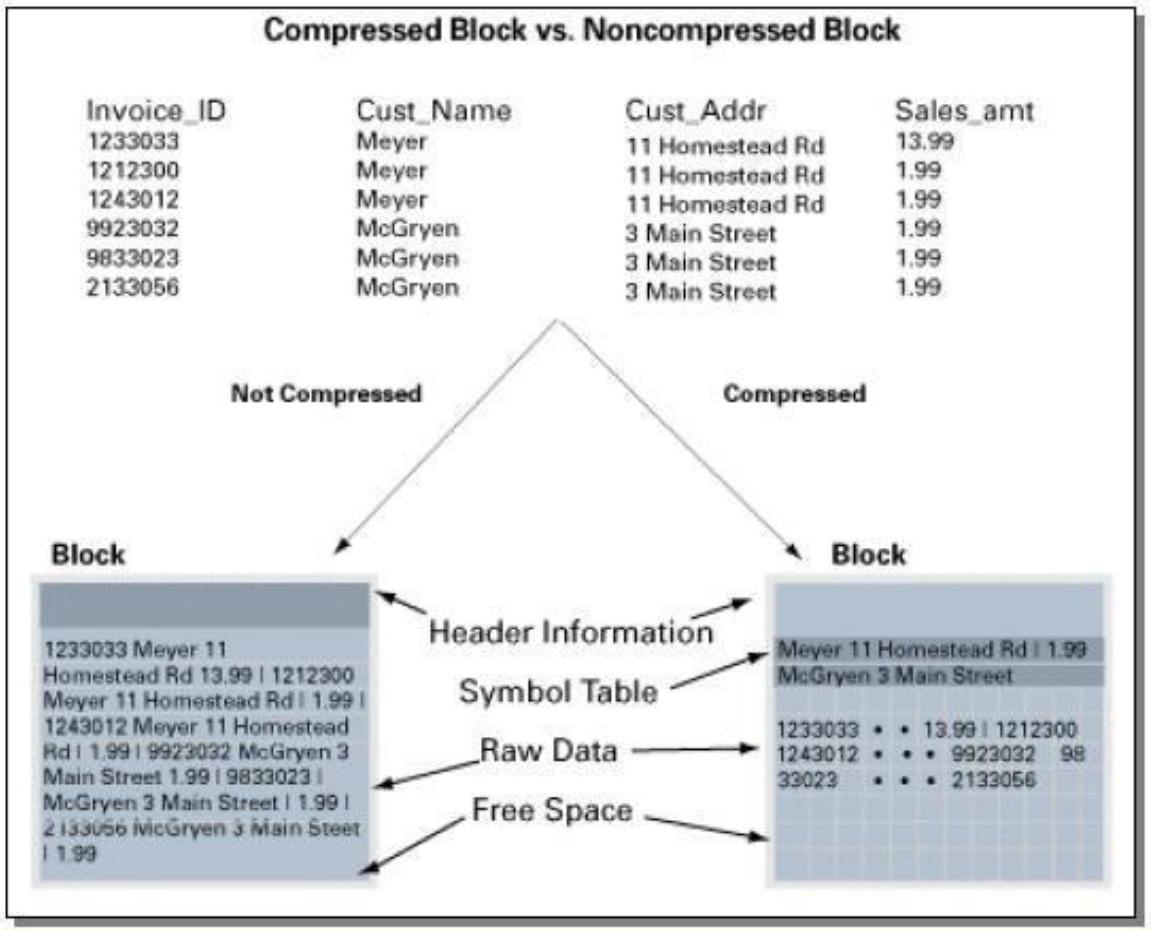
```
CREATE TABLE TableCompressee
            (...)
    COMPRESS [BASIC|FOR OLTP] AS
  SELECT * FROM TableNonCompresee
```

- Compression des enregistrements *doublons* dans les blocs
- Permet de sauver de l'espace disque
- Balayage de tables (FULL TABLE SCAN) accéléré
- Mises à jour (UPDATE) plus coûteuses.
- BASIC : Applique la compression lors de l'insertion en lot
- FOR OLTP: Applique la compression à toutes les opérations LMD

Organisation des données en mémoire

Compression des tables

- Exemple:



Organisation des données en mémoire

Index de données

- Structure physique utilisée pour faciliter la recherche d'enregistrements d'une table (ex: éviter le balayage de la table);
- Est ordonné selon une ou plusieurs colonnes composant la clé de l'index;
- Accélère les opérations de sélection, jointure, tri, etc.
- Nécessite de l'espace sur le disque et une mise à jour;
- Peut être *primaire* ou *secondaire*

© Copyright 2026, Moulla

Organisation des données en mémoire

Index primaire vs secondaire

- Index primaire:
  - Organisation indexée qui inclut les enregistrements;
  - Les enregistrements sont ordonnés physiquement selon la clé de l'index;
  - Limite d'un seul index primaire par table;
  - Surtout avantageux lors de sélections par intervalle sur la clé de l'index.
  - Exemple: INDEX ORGANIZED TABLE en Oracle

© Copyright 2026, Moulla

Organisation des données en mémoire

Index primaire vs secondaire

- Index secondaire:
  - Organisation indexée contenant des références (ex: ROWID) aux enregistrements physiques;
  - Les enregistrements ne sont pas ordonnés physiquement
  - Plusieurs index secondaires possibles pour une même table
  - Très avantageux pour les sélections jointures par valeur et les jointures.
  - Exemple: Simple INDEX en Oracle

© Copyright 2026, Moulla

Organisation des données en mémoire

Index primaire dans Oracle

- Avantages:
  - Pas nécessaire de stocker la clé primaire dans les données (car contenue dans l'index) ni le ROWID (remplacé par la clé primaire)
  - Accélère surtout la sélection par intervalle sur la clé primaire
  - Peut également accélérer la jointure (algorithme de tri-fusion)
- Inconvénients:
  - L'adresse logique d'une ligne change lorsque l'index est mis à jour
  - Pas recommandé de combiner avec des index secondaires sur d'autres colonnes

© Copyright 2026, Moulla

Organisation des données en mémoire

Index secondaire dans Oracle

- On peut avoir plusieurs index par table, mais un seul par ensemble de colonnes;
- Créer des index coûte de la mémoire;
- Consignes:
  - Ne pas créer d'index si plus de 15% de la table peut être retournée (sinon balayage est plus efficace);
  - Ne pas créer d'index sur les petites tables;
  - Ne pas créer d'index sur les colonnes qui sont souvent nulles;
  - Créer des index sur les clés étrangères et colonnes utilisées pour filtrer;
  - Clés composées: mettre d'abord les colonnes les plus utilisées car l'index sert également pour les préfixes.

Évaluation et optimisation de requêtes

Question

- Laquelle de ces requêtes est la plus performante?

Requête 1:

```
SELECT SUM(LC.quantite)
FROM LigneCommande LC, Produit P
WHERE LC.idProduit = P.idProduit AND
P.categorie = 'imprimante'
```

Requête 2:

```
SELECT SUM(LC.quantite)
FROM LigneCommande  LC
                    IN
  (SELECT idProduit FROM Produit P
   WHERE P.categorie ='imprimante')
```

Évaluation et optimisation de requêtes

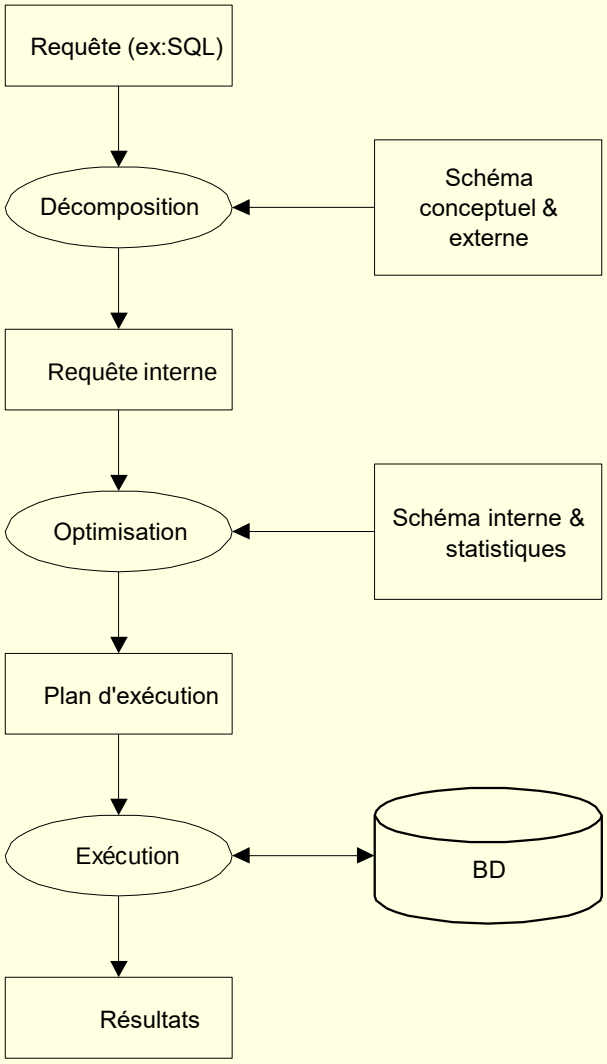
Évaluation des requêtes relationnelles

- SQL
  - QUOI
- Évaluateur de requêtes du SGBD
  - COMMENT
  - en fonction du schéma interne

© Copyright 2026, Moulla

Évaluation et optimisation de requêtes

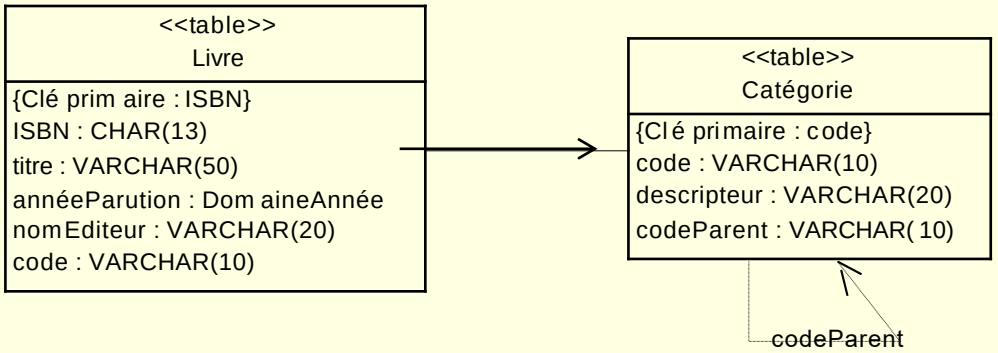
Traitement des requêtes



Évaluation et optimisation de requêtes

Requête interne

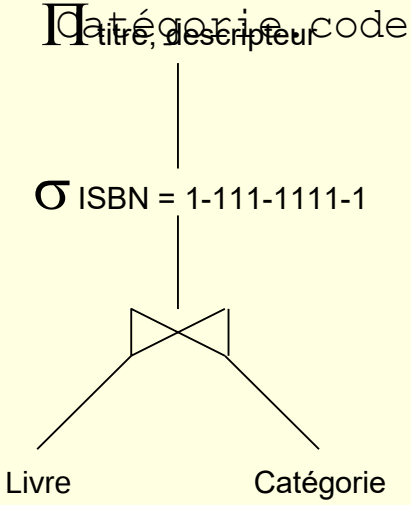
Schéma relationnel:



Requête SQL:

```
SELECT     titre, descripteur
 FROM       Livre, Catégorie
 WHERE             ISBN = 1-111-1111-1 AND Livre.code =
                                    Catégorie.code
```

Requête interne:



Évaluation et optimisation de requêtes

Coût des opérations physiques

- La performance d'une requête interne est évaluée en fonction:

Métrique	Description
TempsES :	Temps d'accès (lectures et écritures) à la mémoire secondaire
TempsUCT :	Temps de traitement de l'unité centrale (souvent négligeable par rapport au temps d'accès)
TailleMC :	Espace requis en mémoire centrale
TailleMS :	Espace requis en mémoire secondaire

- Dans les systèmes transactionnels:
  - Le principal soucis est de pouvoir traiter les requêtes le plus rapidement possible;
  - Les accès disque sont les opérations les plus coûteuses;
  - Donc, la principale métrique de performance est *TempsES*

Évaluation et optimisation de requêtes

Statistiques d'évaluation

- L'estimation du coût des opérations se base sur des statistiques
- Statistiques d'une table T:

Statistique	Description
N_T	Nombre de lignes de la table T
$TailleLigne_T$	La taille d'une ligne de la table T
FB_T	Facteur de blocage moyen de T (nombre moyen de lignes contenues dans un bloc)
FBM_T	Facteur de blocage maximal de T **Estimation** : $\lfloor (TailleBloc\text{-}TailleDescripteurBloc) / TailleLigne_T \rfloor$
B_T	Nombre de blocs de la table T **Estimation** : $\lceil N_T / FB_T \rceil$
$Card_T(col)$:	Nombre de valeurs distinctes (cardinalité) de la colonne *col* pour la table T Ex : $Card_T(sexe) = 2$
$Min_T(col)$, $Max_T(col)$	Valeurs minimale et maximale que peut prendre une colonne *col*

Évaluation et optimisation de requêtes

Statistiques d'évaluation

- Statistiques d'une expression de sélection par valeur ($col = val$) ou par intervalle ($col \in [val_1, val_2]$) :

Statistique	Description
$FacteurSélectivité_T(col = val)$	Pourcentage de lignes pour lesquelles la colonne *col* contient la valeur *val* **Estimation** : $1/Card_T(col)$
$FacteurSélectivité_T(col \in [val_1,val_2])$	Pourcentage de lignes pour lesquelles la colonne *col* contient une valeur comprise entre val_1 et val_2 **Estimation** : $(val_2 - val_1)/(Max_T(col) - Min_T(col))$
$Sel_T(col = val)$	Nombre de lignes de *T* sélectionnées par l'expression de sélection. **Estimation** : $FacteurSélectivité_T(col) * N_T$

- Le facteur de sélectivité est important car il détermine le nombre de blocs à transférer (*TempsES*)

Évaluation et optimisation de requêtes

Maintien des statistiques par le SGBD

- Dilemme entre la précision des statistiques (menant à une meilleure estimation des coûts) et l'effort (temps, cpu) requis pour les générer.
- Types de mise à jour des statistiques:
  - Incrémentale
    - À chaque mise à jour de la table ou de l'index
  - Mise à jour en différé
    - Durant les périodes de faible activité (ex: la nuit)
    - Déclenchée par un script
  - Estimation par échantillonnage
    - Pour les tables très volumineuses (ex: pourcentage des lignes)

Évaluation et optimisation de requêtes

Maintien des statistiques par le SGBD

- Clause ANALYZE (Oracle)
  - Statistiques complètes:

```
ANALYZE TABLE NomTable COMPUTE STATISTICS
```

  - Échantillonnage (10 pourcent des lignes):

```
ANALYZE TABLE NomTable ESTIMATE STATISTICS
SAMPLE 10 PERCENT
```

© Copyright 2026, Moulla

Évaluation et optimisation de requêtes

Opérations à optimiser

1) Sélection de lignes selon une clé
   - Ex: balayage, index, etc.
2) Tri des lignes d'une table
   - Ex: algorithme tri-fusion
3) Jointure de deux tables
   - Ex: boucles imbriquées, jointure tri-fusion, etc.

© Copyright 2026, Moulla

Évaluation et optimisation de requêtes

Sélection d'enregistrements

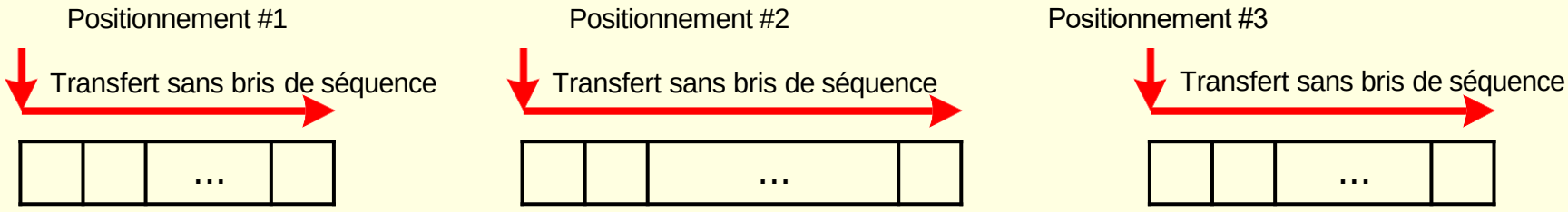
- Sélection par balayage:
  - On parcourt la table jusqu'à ce qu'on trouve l'enregistrement désiré
  - Lire toute la table dans le pire cas: **O**(N)
- Sélection par indexage:
  - On traverse l'index selon la clé de recherche
  - Complexité: **O**(log(N))
  - Permet également la sélection par intervalle

© Copyright 2026, Moulla

Évaluation et optimisation de requêtes

Balayage de tables (FULL TABLE SCAN)

- On lit séquentiellement tous les blocs de la table
- Facteurs à considérer:
  - Le nombre de blocs dans la table
  - Les temps de positionnement de la tête de lecture, dus aux bris de séquence (ex: 10ms)
- Temps estimé:
  - $TempsES\ (BAL) = B_T \times TempsTrans + \quad NombrePos \times TempsPosDébut$



Évaluation et optimisation de requêtes

Balayage de tables (FULL TABLE SCAN)

- Situations où il peut être acceptable de balayer une table:
  - Petites tables:
    - Plus avantageux de lire tous les blocs de la table en mémoire centrale et de faire les opérations (sélection, tri, jointure, etc.) en mémoire centrale.
  - Tables intermédiaires:
    - Ex: table retournée par un SELECT imbriqué
    - On ne possède pas d'index ou autres structures d'optimisation pour ces tables
    - Le balayage est souvent la seule option

Évaluation et optimisation de requêtes

Exemple : balayage de la table Livre

N_{Livre}	1 000 000
FBM_{Livre}	20

- $FB_{Livre} = FBM_{Livre} = 20$
- $B_{Livre} = \lceil N_{Livre} / FB_{Livre} \rceil = \lceil 1000000/20 \rceil = 50\,000$ blocs
- Meilleur cas (aucun bris de séquence):
  - $TempsES\ (BAL_{Livre})$ = 50 000*1 ms + 1*10 ms = 50,01 *secs*
- Pire cas (bris de séquence après chaque lecture de bloc) :
  - $TempsES\ (BAL_{Livre})$ = 50 000*1 ms + 50 000*10 ms = 9,16 *minutes*
- Observations:
  - Le balayage de grandes tables est à éviter si possible.
  - Le temps de repositionnement peut avoir un impact significatif sur la performance de la requête.

Évaluation et optimisation de requêtes

Balayage de l'index (FULL INDEX SCAN)

- Dans les cas où toutes les colonnes souhaitées se trouvent dans la clé de l'index;
- Ex: (index sur la colonne *idClient* d'une table *Transaction*)

```
SELECT idClient, COUNT(*) as nbTransactions
FROM Transaction
GROUP BY idClient
```

- On balaye les blocs feuilles de l'index au lieu de balayer les blocs de la table;
- Comme l'information est plus compacte dans les feuilles de l'index, il y a moins de blocs à lire.

Évaluation et optimisation de requêtes

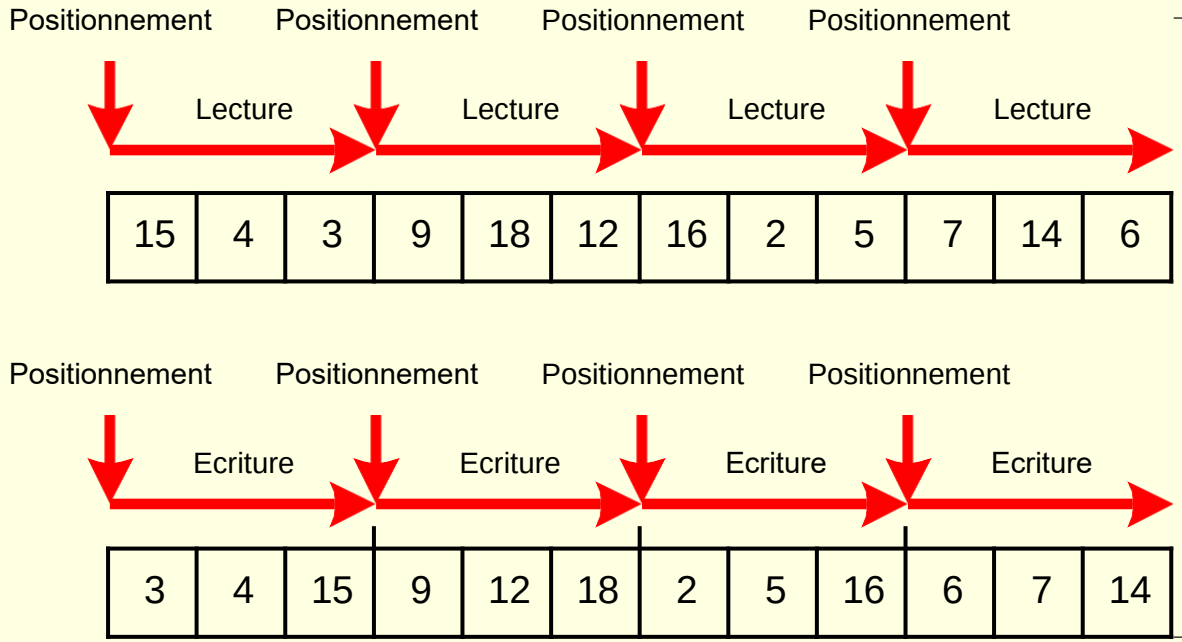
Tri d'une table

- Plusieurs opérations nécessitent de trier une table:
  - Résultats triés (ORDER BY)
  - Élimination des doubles (DISTINCT)
  - Opérations d'agrégation (GROUP BY)
  - Jointure par tri-fusion
- Si le nombre de blocs en mémoire centrale (*M*) est grand
  - On lit la table et la trie en mémoire centrale
- Sinon, si *M* est petit
  - On doit faire un tri externe de la table (ex: algorithme de tri fusion)

Évaluation et optimisation de requêtes

Algorithme de tri fusion

- Etape tri
  - On trie la table en mémoire centrale par groupes de M blocs
  - On doit lire et écrire tous les blocs de la table, avec un repositionnement pour chaque groupe de M blocs
  - Coût : $2 \times (\lceil B_T/M \rceil \times TempsPosDébut + B_T \times TempsTrans)$



Création de 12/3 = 4 groupes

Évaluation et optimisation de requêtes

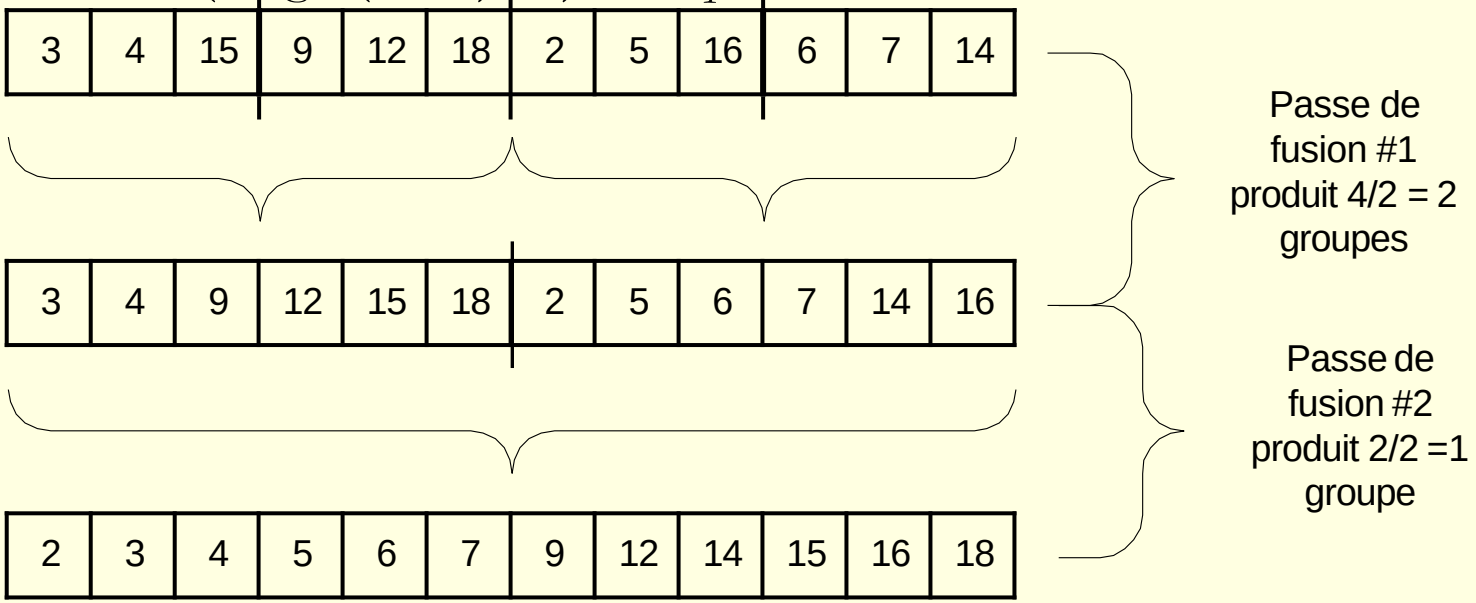
Algorithme de tri fusion

- Étape fusion

– On fusionne récursivement les groupes voisins (triés) jusqu'à obtenir un seul groupe contenant tous les blocs

– Environ $\lceil log_{M-1}(B_T/M) \rceil$ passes de fusion, chacune demandant un balayage en lecture et écriture de la table

– Coût : $B_T \times (2\lceil log_{M-1}(B_T/M) \rceil - 1) \times TempsESBloc$



Évaluation et optimisation de requêtes

Exemple: Tri de la table Livre

N_{Livre}	1 000 000
FBM_{Livre}	20
B_{Livre}	50 000

- $M = 50$ (*TempsESBloc = 11ms)*
- *TempsES* (tri-fusion) = coût_tri + coût_fusion = 29,5 *mins*
- Observations:
  - Inefficace de trier une grande table lorsqu'on possède peu de mémoire centrale (ex: seulement M=50 blocs)
  - Solution alternative: créer un index primaire ou CLUSTER INDEX sur la clé de tri afin que les lignes soient physiquement ordonnées selon cette clé, en mémoire secondaire

Évaluation et optimisation de requêtes

Jointure de deux tables ($R \bowtie_\theta S$)

- Une des opérations ayant le plus d'impact sur la performance des requêtes
- L'optimisation des jointure est donc cruciale
- Principales approches de jointure:
  - Jointure par boucles imbriquées par blocs (BIB)
  - Jointure par boucles imbriquées avec index (BII)
  - Jointure par tri-fusion (JTF)
  - Jointure par hachage (JH)
  - Pré-jointure (PJ)

© Copyright 2026, Moulla

Évaluation et optimisation de requêtes

Boucles imbriquées par blocs

```
POUR chaque bloc b_R de R
    POUR chaque bloc b_S de S
        POUR chaque ligne l_R de b_R
            POUR chaque ligne l_S de b_S
                SI θ sur l_R et l_S est satisfait
                    Produire la ligne concaténée à partir de l_R et l_S
                FINSI
            FINPOUR
        FINPOUR
    FINPOUR
FINPOUR
```

- $TempsES\ (BIB) = B_R \times (TempsESBloc + B_S \times TempsTrans + TempsPosDébut)$
- Variante : Boucles imbriquées multiblocs (BIM)
  - On lit M blocs de R à la fois (au lieu de 1 bloc)
  - Réduit le nombre de repositionnements de la tête de lecture

Évaluation et optimisation de requêtes

Exemple: BIB sur Livre⋈Catégorie

$N_{Catégorie}$	4 000
$FB_{Catégorie}$	40
$B_{Catégorie}$	100

N_{Livre}	1 000 000
FB_{Livre}	20
B_{Livre}	50 000

- *R*=*Livre* et *S*=*Catégorie*:
  - *TempsES* (*BIB*) = 100,83 *mins*
- *R* = *Catégorie* et *S* = *Livre*:
  - *TempsES* (*BIB*) = 83,37 *mins*
- *S* lue une seule fois en mémoire centrale:
  - *TempsES* (*BIB*) = 50,1 *sec*
- Observations:
  - Légèrement avantageux de mettre la plus petite table dans la boucle externe (83,37 *mins* vs 100,83 *mins*)
  - Gain important de performance si la table de la boucle interne peut être mise en mémoire (aucune

Évaluation et optimisation de requêtes

Boucles imbriquées avec index

- Comme la BIB, à la différence que les lignes de la table interne sont obtenues à l'aide d'un index (au lieu d'un balayage):

```
POUR chaque ligne l_R de R
        POUR chaque ligne l_S de S satisfaisant θ (sélection en utilisant un index)
                Produire la ligne concaténée à partir de l_R et l_S
        FINPOUR
FINPOUR
```

- $TempsES\ (BII) = B_R \times TempsESBloc + N_R \times TempsES$ (index secondaire)

- Exige d'avoir un index sur la colonne de jointure de la table située dans la boucle interne

© Copyright 2026, Moulla

Évaluation et optimisation de requêtes

Boucles imbriquées avec index

- Observations:
  - Sélection par index n'est pas tellement affectée par la taille de la table puisqu'on ne parcours pas toutes les lignes de la table.
  - Donc mettre la plus grande table dans la boucle **interne** de la jointure BII.

© Copyright 2026, Moulla

Évaluation et optimisation de requêtes

- On trie les tables *R* et *S*, et on les fusionne en faisant un balayage synchronisé dans les tables triées;
- On joint les groupes de lignes ayant la même valeur pour la clé.

Trier *R* et *S* par tri externe et réécrire dans des fichiers temporaires

Lire groupe de lignes $G_R(c_R)$ de *R* pour la première valeur c_R de clé de jointure Lire groupe de lignes $G_S(c_S)$ de *S* pour la première valeur c_S de clé de jointure

TANT QUE il reste des lignes de *R* et *S* à traiter

SI $c_R = c_S$
Joindre chaque paire de lignes $G_R(c_R)$ et $G_S(c_S)$;
Lire les groupes suivants $G_R(c_R)$ de *R* et $G_S(c_S)$ de *S*;
SINON SI $c_R < c_S$
Lire le groupe suivant $G_R(c_R)$ de R;
SINON SI $c_R > c_S$
Lire le groupe $G_S(c_S)$ suivant dans S;
FINSI
FIN TANT QUE

- $TempsES\ (JTF) = TempsES\ (TRI_R) + TempsES\ (TRI_S) + 2 \times (B_R + B_S) \times TempsESBloc$

Évaluation et optimisation de requêtes

Comparaison des méthodes de jointure

- BIB / BIM
  - Une des deux tables à joindre (ou les deux) est petite et peut être lue en mémoire centrale.
- BII
  - Une des tables est beaucoup plus petite que l'autre et il y a un index secondaire sur la clé de jointure de la **plus grande** table;
  - La **plus grande** table est mise dans la boucle **interne**, et le coût de sélection de ses lignes est amorti par l'index.
- JTF
  - Pour joindre deux grandes tables;
  - Permet également les jointures sur des conditions d'**inégalité**;
  - Avantageux lorsqu'une des tables (ou les deux) est déjà triée selon la clé de jointure (ex: index primaire ou tri d'une opération en amont).

Évaluation et optimisation de requêtes

Optimisation

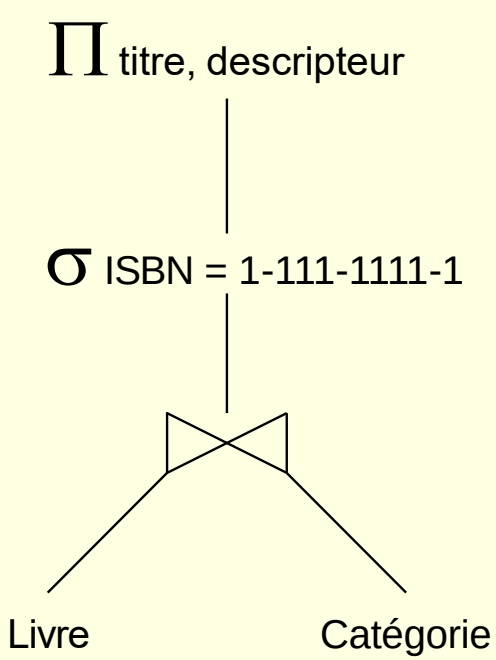
- Chercher le meilleur plan d'exécution.
  - Beaucoup trop de possibilités en pratique
- Solution approchée à coût raisonnable
  - Générer un nombre limité d'alternatives prometteuses
    - Heuristiques (ex: ordre des sélections, projection, jointures, etc.)
  - Choisir la meilleure
    - Estimation approximative du coût d'exécution

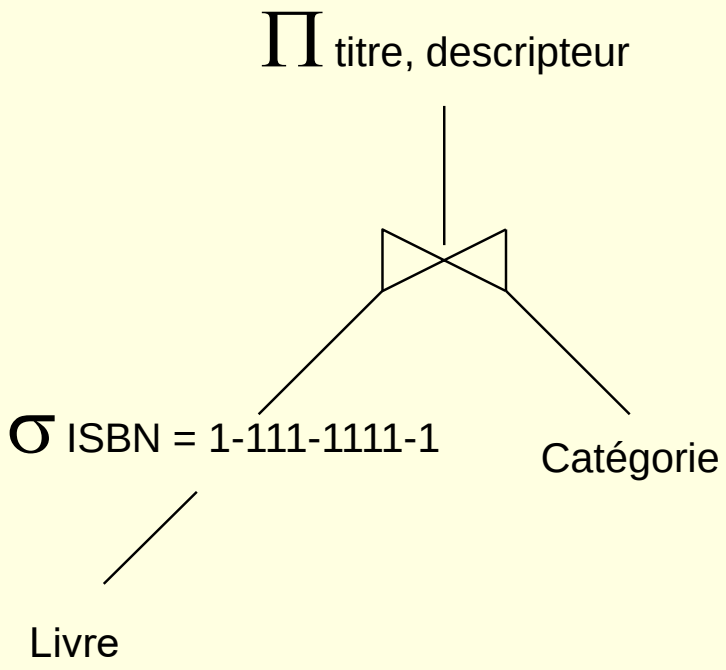
© Copyright 2026, Moulla

Évaluation et optimisation de requêtes

Plans d'exécutions équivalents

- Il peut exister plusieurs arbres algébriques équivalents





© Copyright 2026, Moulla

Évaluation et optimisation de requêtes

Règles d'équivalences d'algèbre relationnelle

- Eclatement d'une sélection conjonctive (SE)
  - $\sigma_{e1 \text{ ET } e2}(T) = \sigma_{e1}(\sigma_{e2}(T))$
- Commutativité de la sélection (SC)
  - $\sigma_{e1}(\sigma_{e2}(T)) = \sigma_{e2}(\sigma_{e1}(T))$
- Elimination des projections en cascades (PE)
  - $\pi_{liste1}(\pi_{liste2}(T)) = \pi_{liste1}(T)$
- Commutativité de la jointure (JC)
  - $T_1 \bowtie T_2 = T_2 \bowtie T_1$
- Associativité de la jointure (JA)
  - $T_1 \bowtie (T_2 \bowtie T_3) = (T_1 \bowtie T_2) \bowtie T_3$

Évaluation et optimisation de requêtes

Règles d'équivalences d'algèbre relationnelle

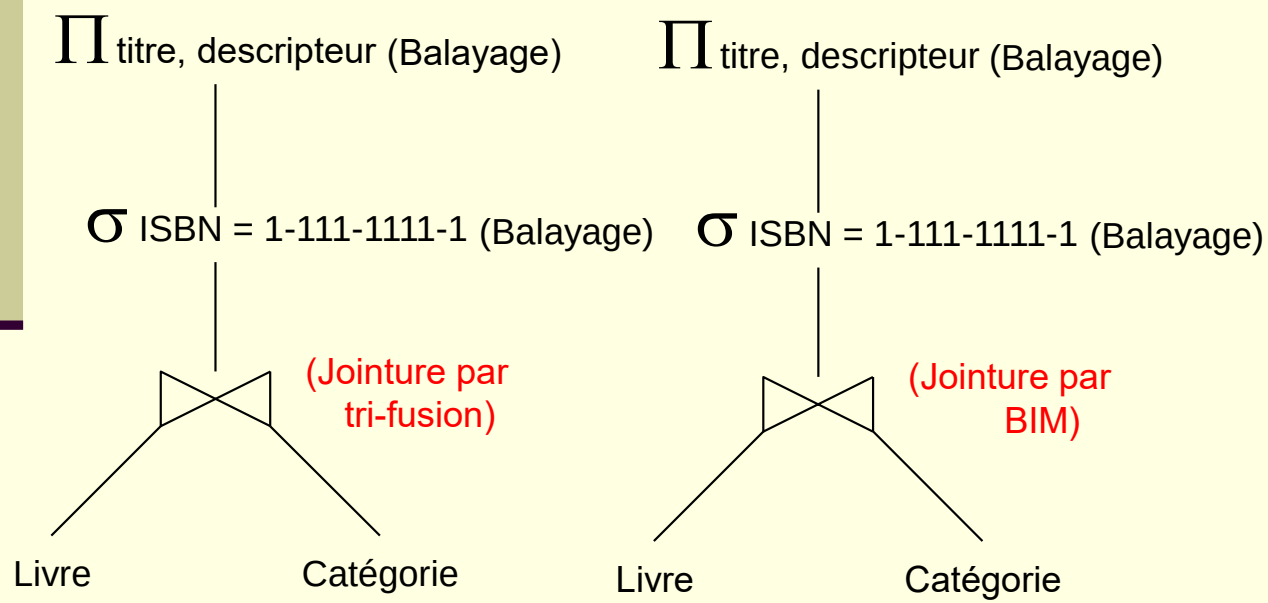
- Commutativité restreinte de la sélection et de la jointure (CSJ)
  - $\sigma_e(T_1 \bowtie T_2) = \sigma_e(T_1) \bowtie T_2$
  - Si l'expression de sélection e ne contient que des colonnes de T_1
- Commutativité restreinte de la projection et de la sélection (CPS)
  - $\pi_{listeCol}(\sigma_e(T)) = \pi_{listeCol}(\sigma_e(\pi_{(listeCol \cup \text{col de } e)} T))$
- Commutativité restreinte de la projection et de la jointure (CPJ)
  - $\pi_{listeCol}(T_1 \bowtie T_2) = \pi_{listeCol}(\pi_{(listeCol \cap col\ de\ T1)}(T_1) \bowtie \pi_{(listeCol \cap col\ de\ T2)}(T_2))$

© Copyright 2026, Moulla

Évaluation et optimisation de requêtes

Plusieurs plans d'exécution pour un arbre algébrique

- Pour chaque opération logique
  - Plusieurs choix d'opérations physiques se présentent.



© Copyright 2026, Moulla

Évaluation et optimisation de requêtes

Types de mise en œuvre:

❑ Matérialisation:

- On exécute les opérations depuis les feuilles vers la racine de l'arbre algébrique
- Chaque opération produit une table intermédiaire

Inconvénients:

- Il faut écrire les tables intermédiaires sur disque si elles sont volumineuses
- Pas d'index sur les tables intermédiaires, donc les opérations sur ces tables sont coûteuses

❑ Pipeline:

- Chaque ligne traverse toutes les opérations sans être écrite dans une table temporaire
- Aucune écriture supplémentaire

Inconvénients:

- Pas adapté à toutes les opérations
- Ex (tri): on doit trier toutes les lignes en même temps

En pratique:

- On utilise le pipeline si possible, sinon la matérialisation

Évaluation et optimisation de requêtes

Heuristiques d'optimisation

- Élaguer l'espace des solutions
  - Solutions non applicables
- Exemples d'heuristiques
  - Sélections le plus tôt possible
  - Projections le plus tôt possible
  - Les jointures plus restrictives en premier
  - Jointures supportées par index, tri-fusion ou hachage en premier

© Copyright 2026, Moulla

Évaluation et optimisation de requêtes

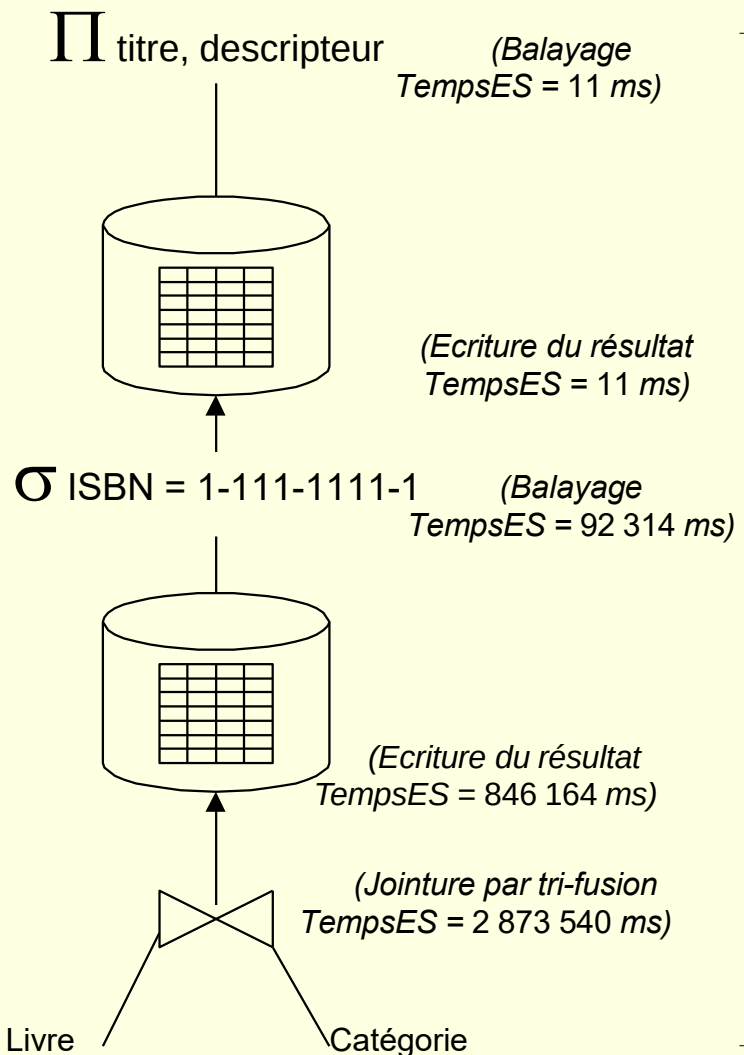
Optimisation par coût

- Minimiser le coût (ex: temps d'exécution)
- Stratégies
  - Programmation dynamique
  - Amélioration itérative
  - Algorithme génétique
  - etc.

© Copyright 2026, Moulla

Évaluation et optimisation de requêtes

Estimation du coût d'un plan d'exécution



$$TempsES(Plan\ avec\ pipeline) = TempsES\ (JTF_{Livre \bowtie Catégorie}) = 2\ 873\ 540\ ms$$

Évaluation et optimisation de requêtes

Types d'optimisation Oracle

- COST (statistique): minimise le coût estimé
  - Besoin de statistiques (ANALYSE)
  - Plus cher à calculer mais donne de meilleurs plans d'exécution
  - Mode ALL_ROWS
    - Minimise le temps total
    - Ex: on préfère JTF à BIM pour joindre deux grandes tables
  - Mode FIRST_ROWS
    - Minimise temps de réponse (obtention des premières lignes)
    - Ex: on préfère plutôt BIM pour joindre deux grandes tables
    - Pour les applications interactives où on n'a pas besoin de voir simultanément toutes les lignes d'une requête
- RULE (optimisation heuristique)
  - Utilisé seulement lorsque des statistiques ne sont pas disponibles (temps à disparaitre).

Les entrepôts de données et l'analyse de données

C'est quoi un data warehouse?

Le data Warehouse est une collection de données **orientées sujet, intégrées, non volatiles et historisées**, organisées pour servir de support d'aide à la décision.

(W. H. Inmon, 1996)

Exemples: *Qui sont mes meilleurs clients? Comment et pourquoi le chiffre d'affaire a baissé? A combien s'élèvent mes ventes journalières?*

© Copyright 2026, Moulla

Les entrepôts de données et l'analyse de données

Motivation des Entrepôts de données

Transformer les « données » des systèmes d'information opérationnels en « informations ».

Données ⟶ **Informations**

- **Comment répondre aux demandes des décideurs?**
  - En donnant un accès rapide et simple à l'information Stratégique;
  - En donnant du sens aux données.

⟹ Mettre en place un système d'information dédié aux applications décisionnelles: **data warehouse**

Les entrepôts de données et l'analyse de données

Domaines d'utilisation des DW

- **Banque**
  - Risques d'un prêt, prime plus précise
- **Santé**
  - Épidémiologie
  - Risque alimentaire
- **Commerce**
  - Ciblage de clientèle
  - Déterminer des promotions
- **Logistique**
  - Adéquation demande/production
- **Assurance**
  - Risque lié à un contrat d'assurance (voiture)
- etc.

© Copyright 2026, Moulla

Les entrepôts de données et l'analyse de données

Différences entre OLTP et OLAP

Critère	Online Transaction Processing (OLTP)	Online Analytical Processing (OLAP)
But	Contrôler et exécuter les tâches quotidiennes et fondamentales de l'entreprise	Assister dans la planification, la résolution de problème et la prise de décision
Types de données	Données opérationnelles (transactions)	Données historiques consolidées
Sources de données	BD transactionnelles	Entrepôts de données ou magasins de données
Ce que montre les données	Portrait instantané des processus d'affaires de l'entreprise	Vue multidimensionnelle de plusieurs activités d'affaires de l'entreprise
Insertions et mises-à-jour	Courtes requêtes d'insertion et de mise-à-jour lancées par les usagers finaux	Longs traitements en lot servant à rafraichir les données
Requêtes	Simples requêtes retournant quelques enregistrements (lignes) de la BD	Requêtes complexes impliquant souvent plusieurs tables et faisant l'agrégation de valeurs
Temps de réponses	Instantané	Quelques secondes à 1 minute max.

Les entrepôts de données et l'analyse de données

Différences entre OLTP et OLAP (suite)

Critère	Online Transaction Processing (OLTP)	Online Analytical Processing (OLAP)
Espace requis	Relativement petit si les données historiques sont archivées	Grand due aux données historiques et aux données d'agrégation
Modélisation de la BD	Un grand nombre de tables hautement normalisées	Moins de tables, tables typiquement dénormalisées, schémas en étoile ou flocon
Sauvegarde et récupération	Fait de façon régulière, critique pour l'entreprise	Fait de façon irrégulière, peut récupérer des données OLTP
Reporting	Routinier et très ciblé	Ad hoc, multidimensionnel, à plus large portée
Ressources requises	Simple DBMS relationnel	DBMS spécialisé multi-processeurs et à grande capacité
Nombre d'utilisateurs simultanés	Grand	Petit

Les entrepôts de données et l'analyse de données

Entreposage de données (data warehousing):

- « La copie **périodique** et **coordonnée** de données provenant de **diverses sources**, internes et externes à l'entreprise, dans un environnement optimisé pour l'**analyse** »
- **Orientés sujet:**
  - Les données sont organisées par sujet (ex: clients, produits, ventes, etc.).
- **Intégrés:**
  - Les données, qui proviennent de diverses sources hétérogènes, sont consolidées et intégrées dans l'entrepôt.
- **Historiques:**
  - Les données ont très souvent une composante temporelle (ex: date et heure d'une transaction).
- **Non-volatiles:**
  - Une fois insérées dans l'entrepôt, les données ne sont jamais modifiées ou effacées; elle sont conservées pour des analyses futures.

Les entrepôts de données et l'analyse de données

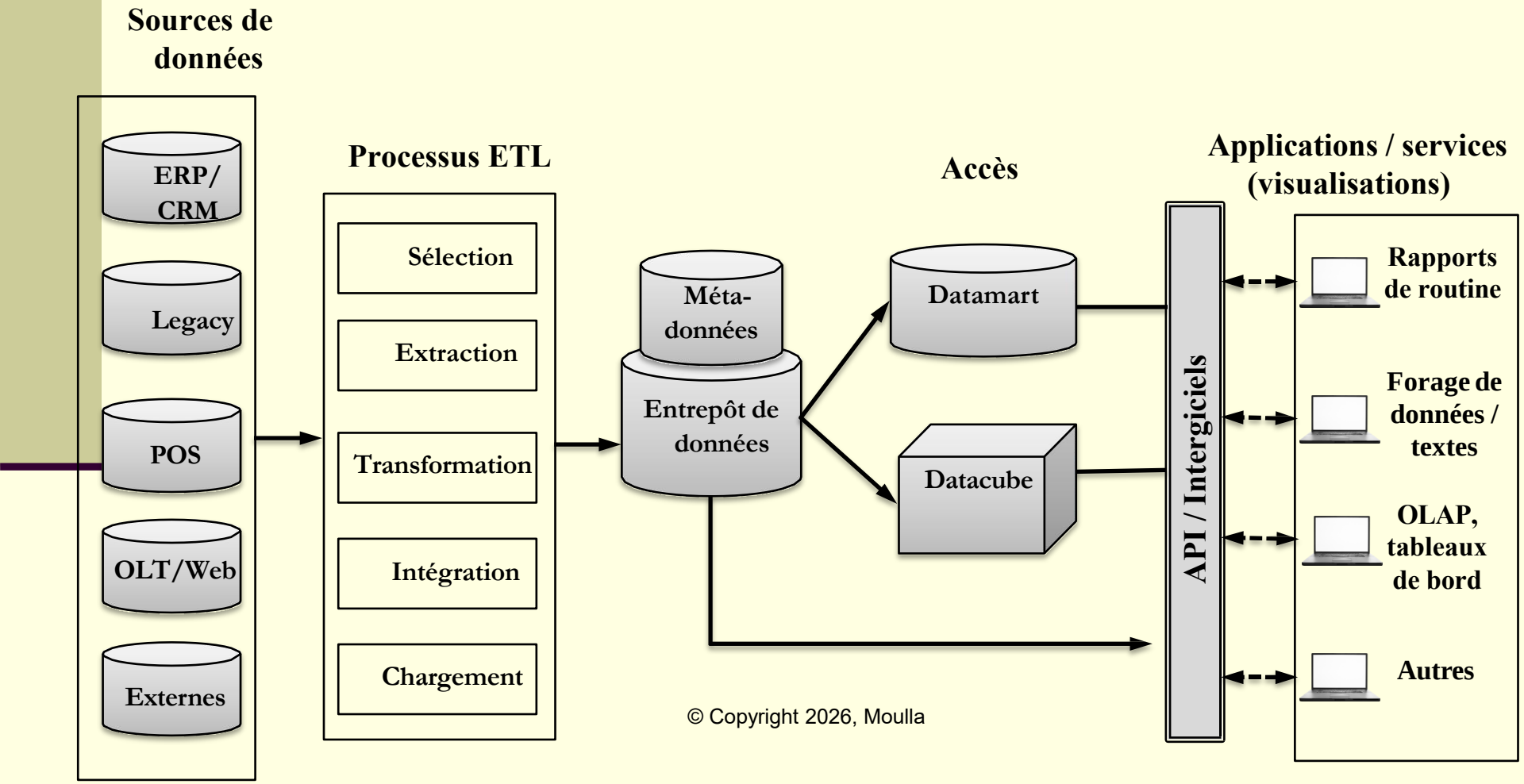
Avantages des entrepôts de données

- Permettent de mener des analyses poussées sur différents sujets d'affaires;
- Fournissent une vue consolidée des données de l'entreprise (une seule vérité);
- Procurent de l'information de qualité, plus rapidement;
- Libèrent les ressources (ex: serveurs) dédiées au traitement des transactions des tâches d'analyse;
- Simplifient l'accès aux données.

© Copyright 2026, Moulla

Les entrepôts de données et l'analyse de données

Solution d'entreposage de données



© Copyright 2026, Moulla

Les entrepôts de données et l'analyse de données

Sources de données

- **Enterprise resource planning (ERP):**
  - Gèrent les processus opérationnels d'une entreprise (ex: ressources humaines, finances, distribution, approvisionnement, etc.).
- **Customer relationship management (CRM):**
  - Gèrent les interactions d'une entreprise avec ses clients (ex: marketing, ventes, service après-vente, assistance technique, etc.).
- **Systèmes legacy:**
  - Matériels et logiciels obsolètes mais difficilement remplaçables.
- **Point of sale (POS):**
  - Matériels et logiciels utilisés dans les caisses de sorties d'un magasin.

- **Externes:**

Les entrepôts de données et l'analyse de données

Problèmes des sources de données

- Sources diverses et disparates (ex: BD, fichier texte, etc.);
- Sources sur différentes plateformes et OS;
- Applications *legacy* utilisant des technologies obsolètes;
- Historique de changement non-préservé dans les sources;
- Qualité de données douteuse et changeante dans le temps;
- Structure des systèmes sources changeante dans le temps;
- Incohérence entre les différentes sources;
- Données dans un format difficilement interprétable ou ambigu.

Les entrepôts de données et l'analyse de données

Processus Extract, Tranform, Load (ETL)

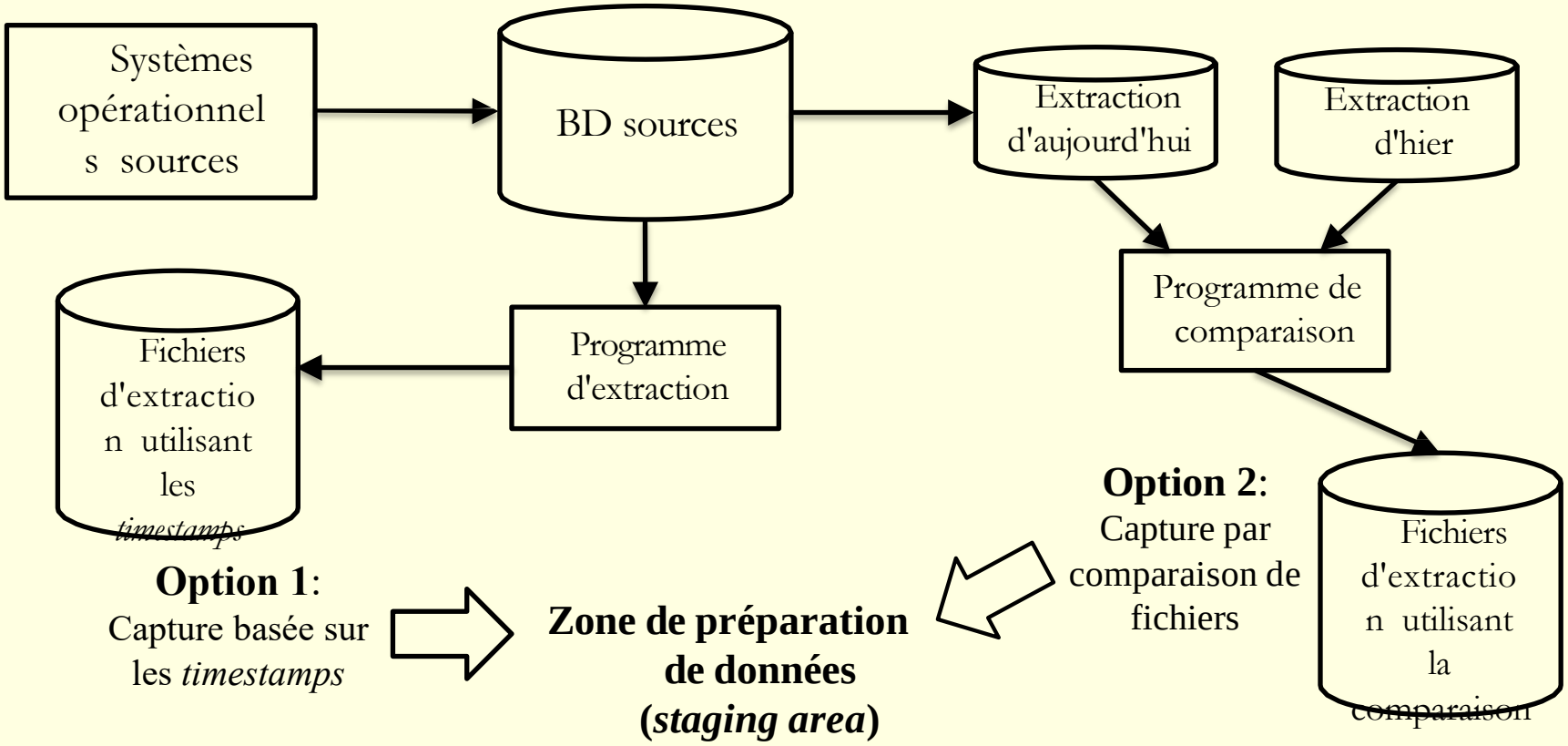
- **Extraire les données des sources hétérogènes (*extract*)**
  - Identifier les données sources utiles;
  - Déterminer les données qui ont changé
- **Consolider les données (*transform*)**
  - Données redondantes, manquantes, incohérentes, etc.
  - Découpage, fusion, conversion, aggrégation, etc.
- **Charger les données intégrées dans l'entrepôt (*load*)**
  - Mode différé (batch) ou quasi temps-réel.
- Partie la plus longue du développement (jusqu'à 70% du temps total).

© Copyright 2026, Moulla

Les entrepôts de données et l'analyse de données

Extraction des données (différée)

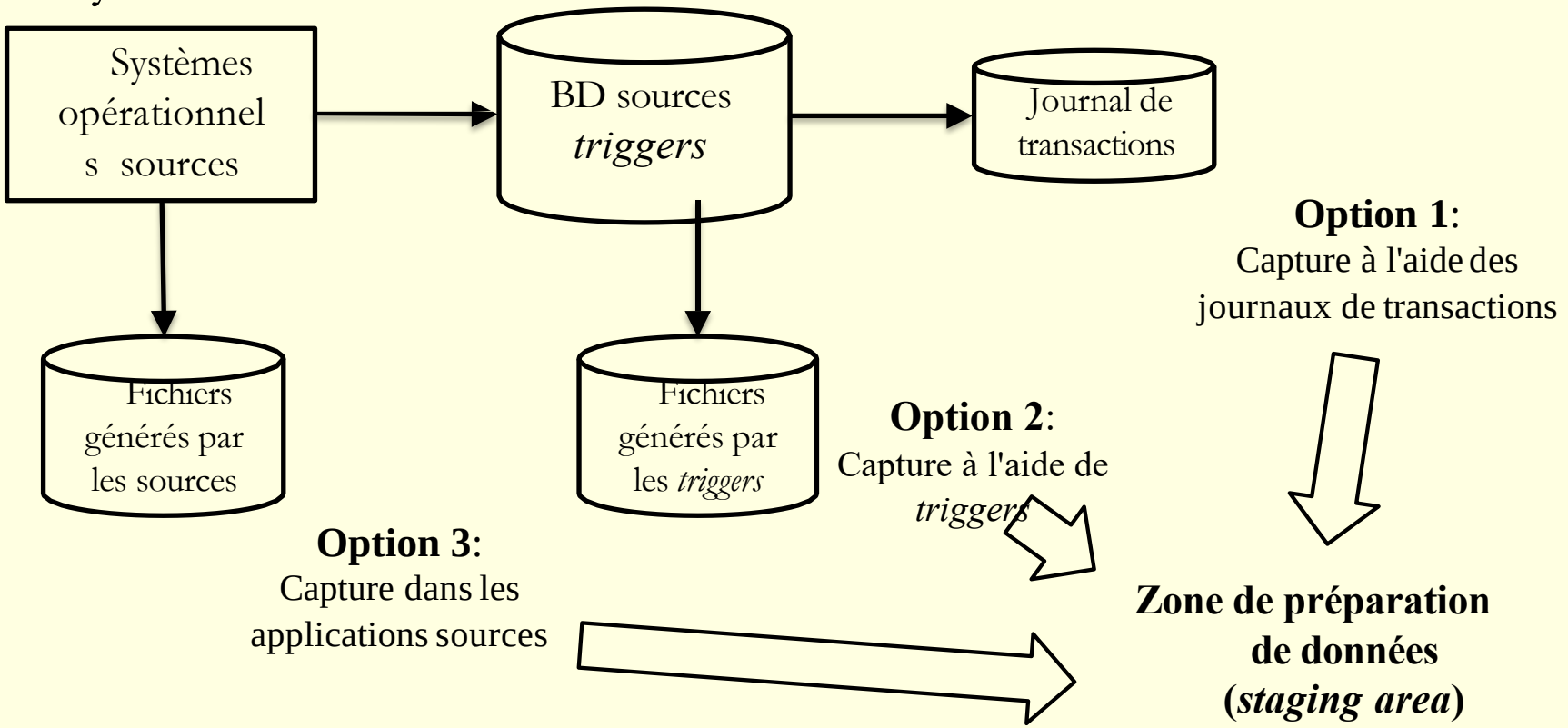
- Extrait tous les changements survenus durant une période donnée (ex: heure, jour, semaine, mois).



Les entrepôts de données et l'analyse de données

Extraction des données (temps-réel)

- S'effectue au moment où les transactions surviennent dans les systèmes sources.



Les entrepôts de données et l'analyse de données

Extraction des données

- Depuis les bases sources ou les journaux;
- Différentes techniques: requêtes et triggers;

© Copyright 2026, Moulla

Les entrepôts de données et l'analyse de données

Transformation des données

- Révision ou conversion de format :
  - Ex: Changer le type ou la longueur de champs individuels.
- Décodage de champs:
  - Ex: ['homme', 'femme'] vs ['M', 'F'] vs [1,2].
- Pré-calcul des valeurs dérivées:
  - Ex: *profit* calculé à partir de *ventes* et *coûts*.
- Découpage de champs complexes:
  - Ex: extraire les valeurs *prénom*, *secondPrénom* et *nomFamille* à partir d'une seule chaîne de caractères *nomComplet*.
- Pré-calcul des agrégations:
  - Ex: ventes par produit par semaine par région.
- Déduplication
  - Ex: Plusieurs enregistrements pour un même client

Les entrepôts de données et l'analyse de données

Chargement des données

Insertion des données dans l'entrepôt

- Faire les chargements en lot dans une période creuse (entrepôt de données non utilisé);
- Considérer la bande passante requise pour le chargement;
- Avoir un plan pour évaluer la qualité des données chargées dans l'entrepôt;
- Commencer par charger les données des tables de dimension;
- Désactiver les indexes et clés étrangères lors du chargement.

© Copyright 2026, Moulla

Les entrepôts de données et l'analyse de données

Types d'entrepôts de données

1. Magasins de données
2. Entrepôts de données d'entreprise (EDW)
   - Bus de magasins de données (datamartbus);
   - Hub-and-spokes;
   - Entrepôts de données fédérés.

© Copyright 2026, Moulla

Les entrepôts de données et l'analyse de données

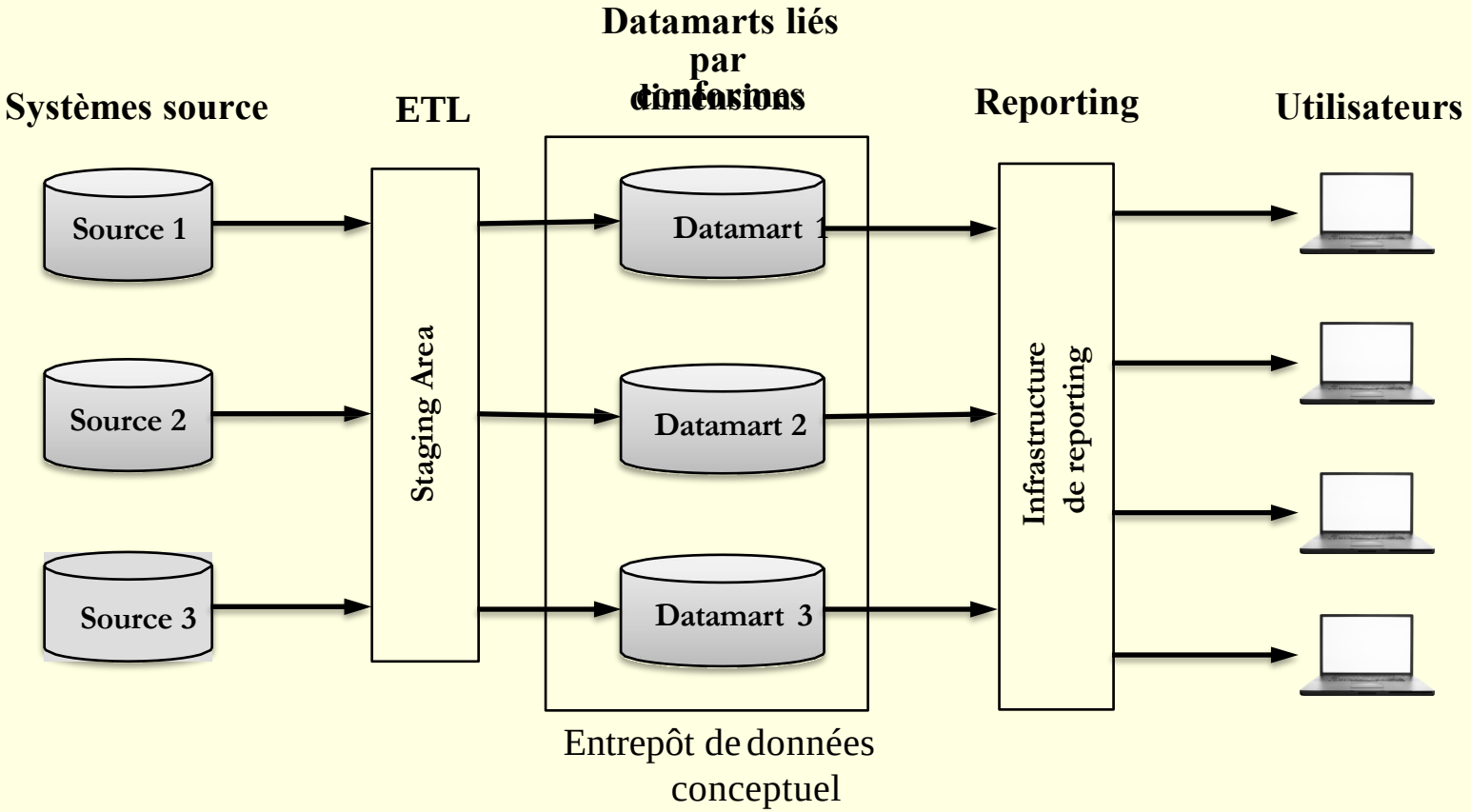
Magasins de données (Datamart)

Sous ensemble de l'entrepôt de données qui:

- Contiennent une portion du contenu de l'entrepôt de données;
- Se concentrent sur un seul sujet d'analyse (ex: les ventes OU l'inventaire, mais pas les deux);
- Servent à faire des analyses simples et spécialisées (ex: fluctuations des ventes par catégorie de produits);
- Nombre de sources limitées, provenant la plupart du temps d'un même département;
- Modélisés sous la forme d'un schéma en étoile.

Les entrepôts de données et l'analyse de données

Architecture Datamart bus



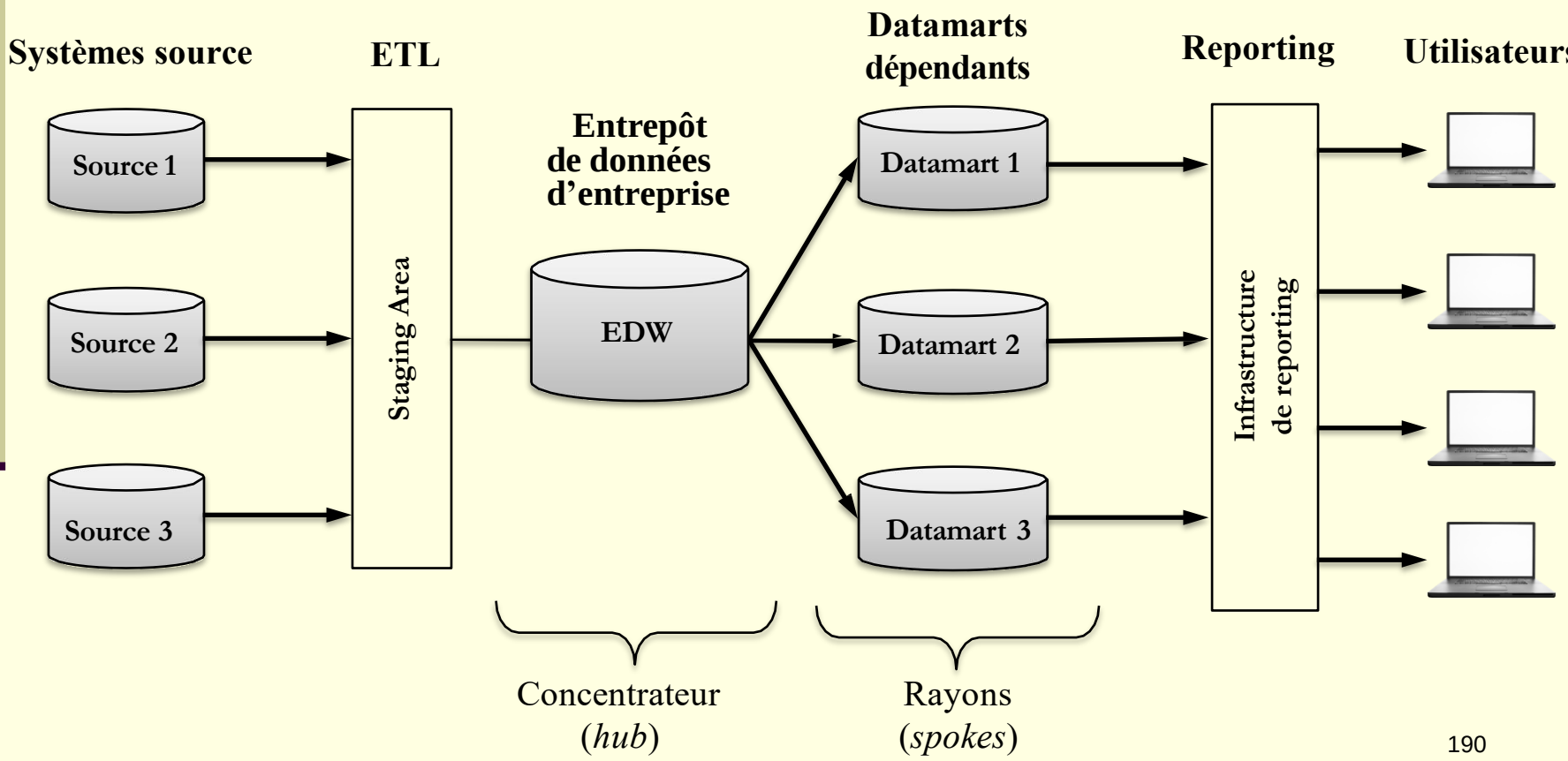
Les entrepôts de données et l'analyse de données

Architecture Datamart bus

- Approche *bottom-up*, où on construit l'entrepôt un datamart à la fois;
- Modélisation dimensionnelle (*schéma en étoile*) des datamarts, au lieu du diagramme entité-relation;
- Entrepôt de données conceptuel, formé de magasins de données inter-reliés à l'aide d'une couche d'intergiciels (*middleware*).
- Intégration des données assurée par les dimensions partagées entre les datamarts (i.e., *dimensions conformes*);
- Approche incrémentale qui donne des résultats rapidement (développement agile);

Les entrepôts de données et l'analyse de données

Architecture Hub-and-spokes



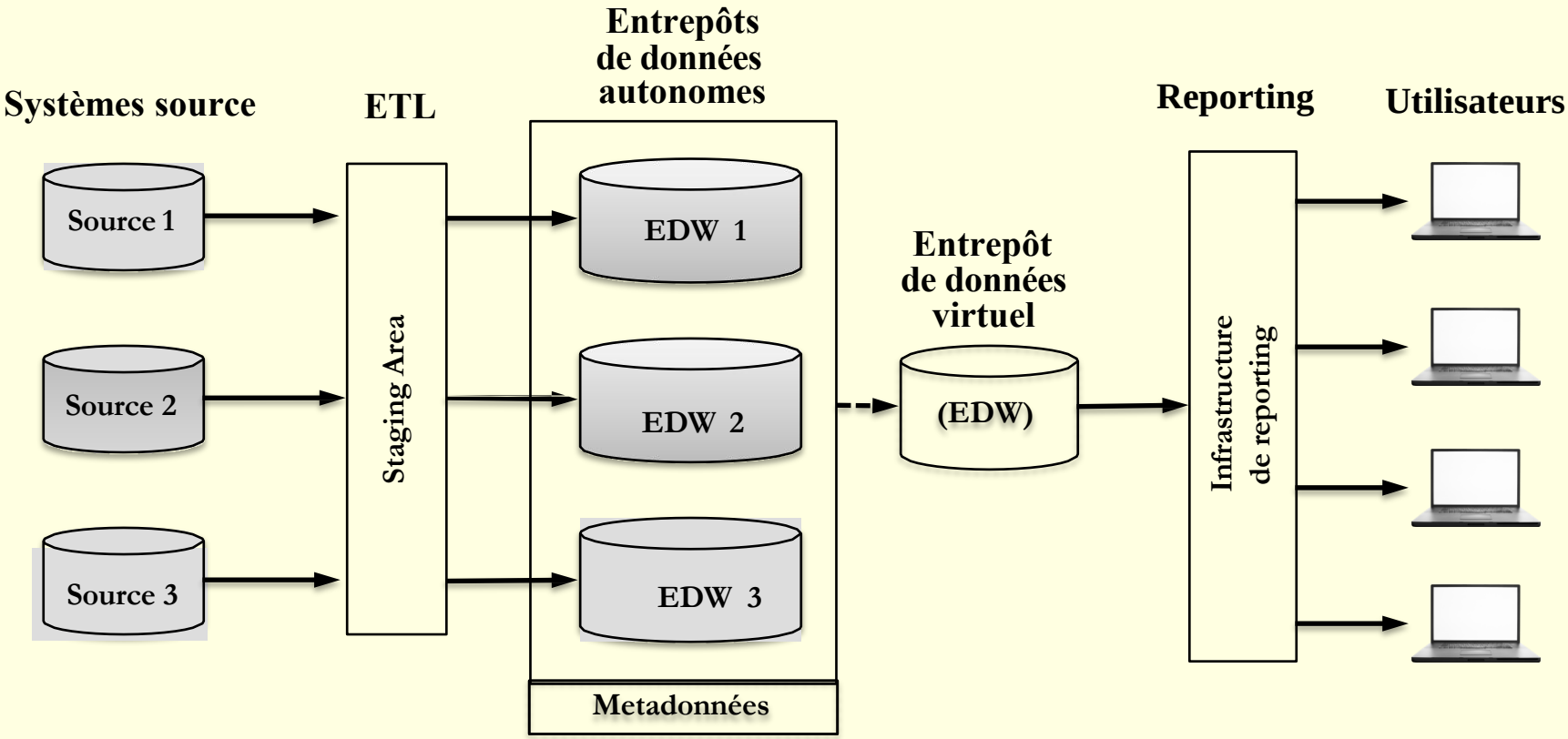
Les entrepôts de données et l'analyse de données

Architecture Hub-and-spokes

- Approche *top-down*, favorisant l'intégration et consolidation complète des données de l'entreprise;
- Entrepôt (*hub*) contient les données **atomiques** (niveau de détail le plus fin) et **normalisées** (3FN);
- Les datamarts (*spokes*) contiennent principalement des données **agrégées** (pas atomique) et suivant le **modèle dimensionnel**;
- La plupart des requêtes analytiques sont faites sur les datamarts;
- Développement plus long, dû à la complexité du processus ETL et de la modélisation;
- Meilleure qualité de données que l'architecture par bus de datamarts.

Les entrepôts de données et l'analyse de données

Architecture fédérée



Les entrepôts de données et l'analyse de données

Architecture fédérée

- Entrepôt de données distribué sur plusieurs systèmes hétérogènes;
- Données intégrées logiquement ou physiquement à l'aide de méta-données (ex: XML);
- Opère de manière transparente (l'utilisateur ne voit pas que les données sont réparties);
- Utile lorsqu'il y a déjà un entrepôt en place (ex: acquisitions ou fusions de compagnies);
- Très complexe (synchronisation, parallélisme, concurrence, etc.) et faible performance.

Les entrepôts de données et l'analyse de données

Modélisation dimensionnelle

- Représente les données sous la forme d'un **schéma en étoile**:
  - Table de faits entourée de plusieurs tables de dimension (normalement entre 8 et 15)
- Les faits (mesures) sont généralement des valeurs numériques provenant des processus d'affaires;
- Les dimensions fournissent le contexte (*qui, quoi, quand, où, pourquoi et comment*) des faits;
- Les tables ne sont pas normalisées

© Copyright 2026, Moulla

Les entrepôts de données et l'analyse de données

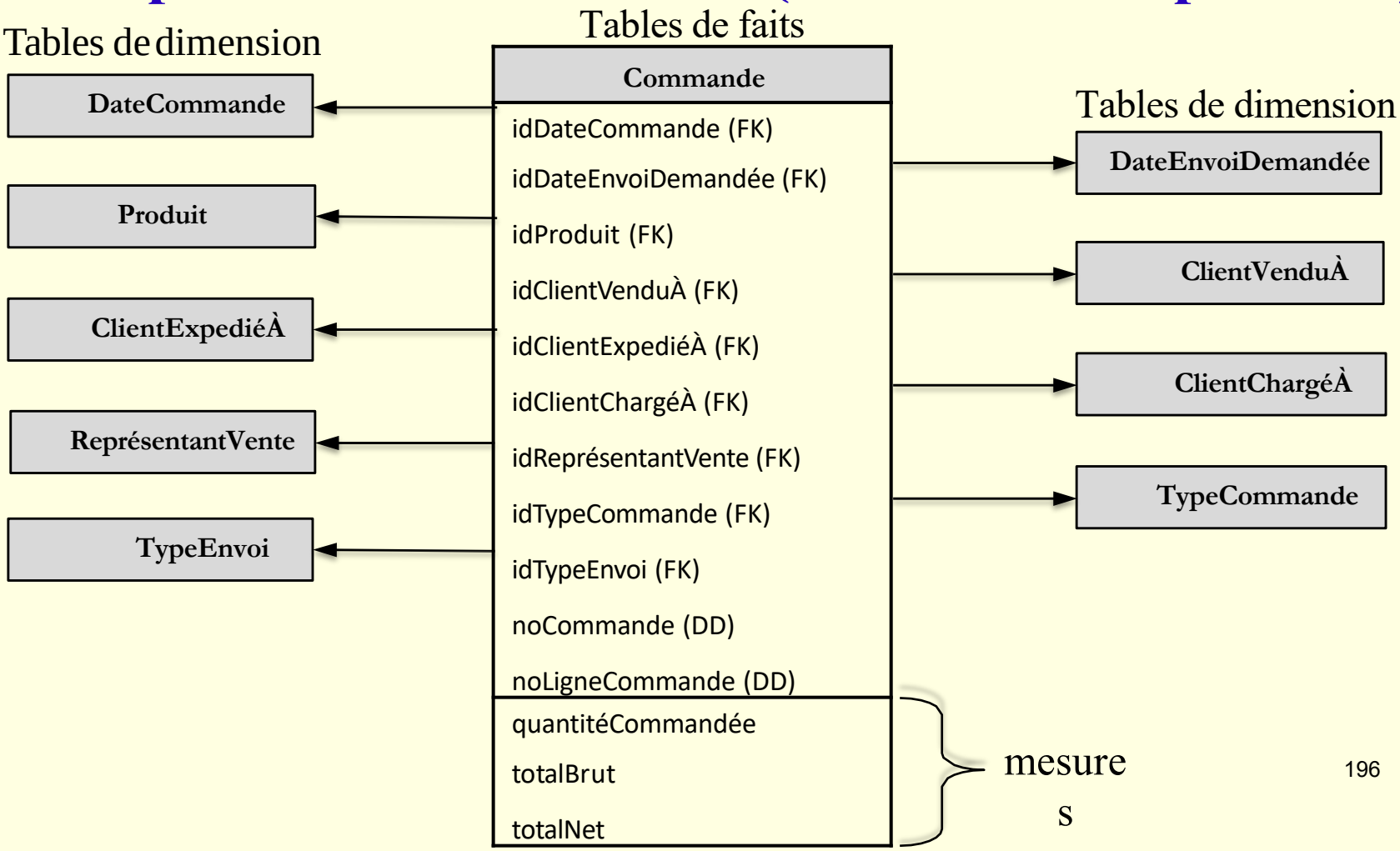
Avantages comparés au modèle entité-relation

- Compréhensibilité:
  - Données regroupées selon des catégories d'affaires qui ont un sens pour les utilisateurs d'affaires;
- Performance:
  - La dénormalisation évite les jointures coûteuses;

© Copyright 2026, Moulla

Les entrepôts de données et l'analyse de données

Exemple de schéma en étoile (commande de produits)



Les entrepôts de données et l'analyse de données

Exemple de schéma en étoile (commande de produits)

Tables de dimension:

DateCommande
idDate (PK)
date
jourDeSemaine
jourDuMois
jourDeAnnée
jourDansMoisFiscal
jourDansAnnéeFiscale
congéFérié
jourDeTravail
semaineDuMois
…

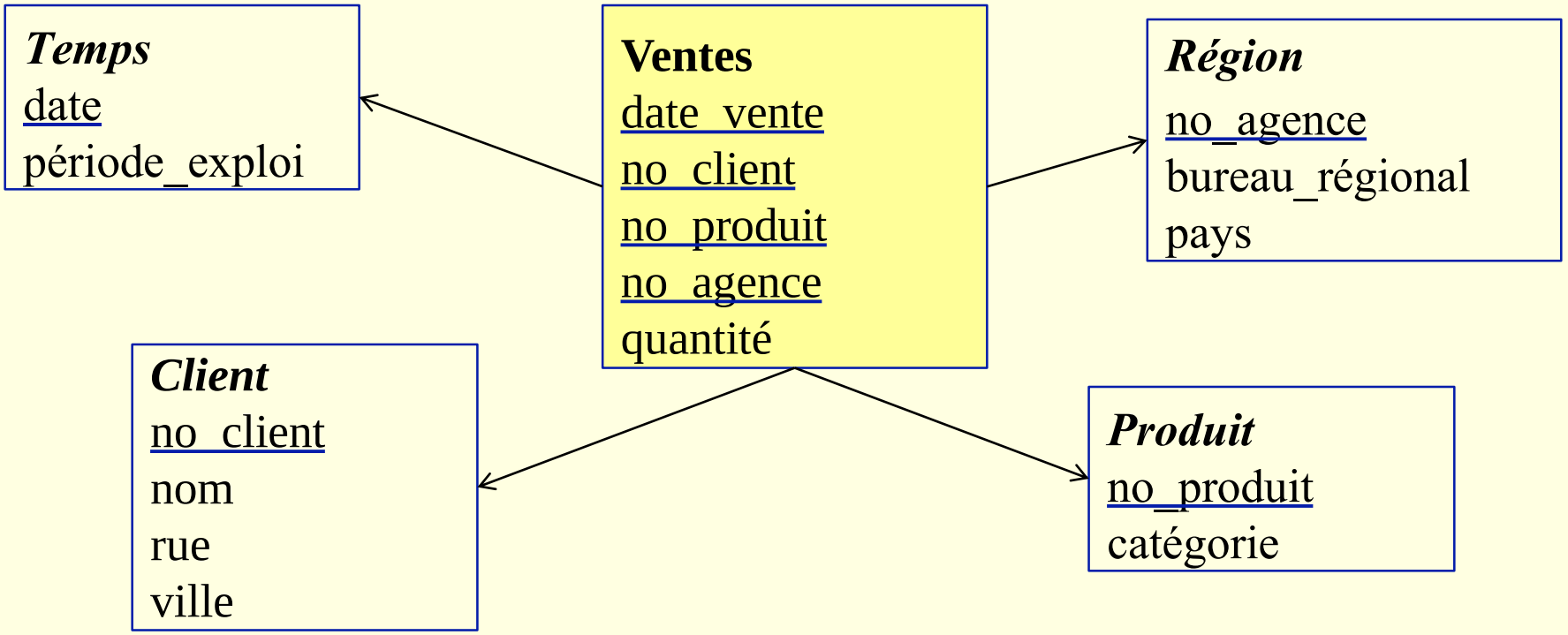
Produit
idProduit (PK)
descriptio
n SKU
marque
sousCatégorie
catégorie
département
poids
taille
couleur
…

ClientExpédiéÀ
idClient (PK)
nomFamille
prénom
sexe
dateNaissance
dateAbonnement
forfaitAbonnement
adresseRue
adresseVille
adresseProvince
…

© Copyright 2026, Moulla

Les entrepôts de données et l'analyse de données

Exemple de schéma en étoile (ventes de produits)



© Copyright 2026, Moulla

Les entrepôts de données et l'analyse de données

Tables de faits

- Correspondent à un événement d'affaires
  - Ex: achat d'un produit par un client, envoi du produit au client, commande de matériaux auprès d'un fournisseur, etc.
- Contiennent deux types de colonnes:
  - Des métriques associées à l'événement d'affaire:
    - Ex: total des ventes, nombre d'items commandés, etc.
  - Des clés étrangères vers les tables de dimension:
    - Ex: ID du client qui fait la commande, ID du produit commandé, etc.
- Contiennent typiquement un très grand nombre de lignes:
  - Jusqu'à plusieurs milliards de lignes;
  - Souvent plus de 90% des données du modèle.

Les entrepôts de données et l'analyse de données

Tables de faits

- Contient les données observables (les faits) sur le sujet étudié selon divers axes d'analyse (les dimensions).
- Fait : ce que l'on souhaite mesurer (qtés vendues, montants des ventes,…).

Clés étrangères vers les dimensions

Faits

Table des faits de Ventes

date_vente

no_client

no_produit

no_agence

Quantité vendue

Coût

Montant des ventes

Les entrepôts de données et l'analyse de données

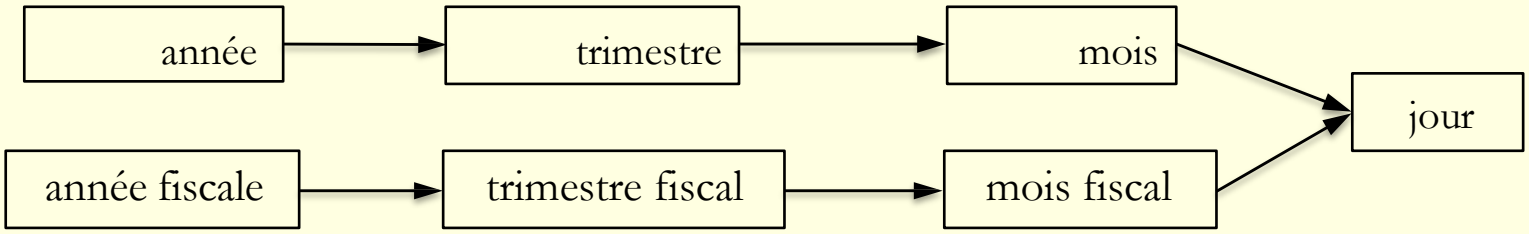
Tables de dimension

- Ensemble hautement corrélé d'attributs (*jusqu'à plusieurs dizaines*) regroupés selon les objets clés d'une entreprise:
  - Ex: produits, clients, employés, installations, etc.
- Propriétés des attributs:
  - Descriptif (ex: chaînes de caractères);
  - De qualité (ex: aucune valeur manquante, obsolète, erronée, etc.);
  - Valeurs discrètes (ex: jour, âge d'un client);
- Rôles des attributs:
  - Filtrer/agréger les données (ex: ville, catégorie produit, etc.);
  - Étiqueter les résultats (ex: champs descripteurs).

Les entrepôts de données et l'analyse de données

Hiérarchies dimensionnelles

- Ensemble d'attributs d'une table de dimension ayant une relation hiérarchique (*x* est inclus dans *y*);
- Correspondent à des relations de type 1 à plusieurs;
- Définissent les chemins d'accès dans les données (*drill-down paths*);
- Peuvent être simples:
  - **Produit** : tous) catégorie) marque) produit;
  - **Lieu** : tous) pays) province) ville) code postal.
- Ou multiples:



Les entrepôts de données et l'analyse de données

Dimension temporelle

Dimension: Temps
idDate (PK)
date
jourDeSemaine
jourDuMois
jourDeAnnée
jourDansMoisFiscal
jourDansAnnéeFiscale
congéFérié
jourDeTravail
semaineDuMois
…

- Il faut mettre toutes ces valeurs même si la plupart peuvent être déduites d'une seule colonne;

© Copyright 2026, Moulla

Les entrepôts de données et l'analyse de données

Dimension temporelle

- Problème: avoir un grain trop fin dans la dimension temporelle (ex: temps du jour) peut causer l'explosion du nombre de rangées:
  - Ex: 31,000,000 secondes différentes dans une année.
- Solution: mettre le temps du jour dans une dimension séparée:
  - **Dimension Date** : année) mois) jour;
  - **Dimension TimeOfDay** : heure) minute) secondes;
  - 86,400 + 365 lignes au lieu de 31,000,000 lignes.

Où 86,400 représente le nombre de secondes dans un jour et 365 le nombre de jours dans une année.

© Copyright 2026, Moulla

Les entrepôts de données et l'analyse de données

Dimensions à évolution lente (SCD)

- Slowly Changing Dimensions (SCD);
- Même si elles sont plus statiques que les tables faits, les dimensions peuvent également changer:
  - Ex: adresse d'un client, catégorie d'un produit, etc.
- Stratégies d'historisation:
  - **SCD Type 1**: Écraser l'ancienne valeur avec la nouvelle;
  - **SCD Type 2**: Ajouter une ligne dans la table de dimension pour la nouvelle valeur;
  - **SCD Type 3**: Avoir deux colonnes dans la table de dimension correspondant à l'ancienne et la nouvelle valeur;

Les entrepôts de données et l'analyse de données

Stratégie SCD Type 1

idProduit	description	code	catégorie
1001	'BébéLala'	'ABC999-Z'	'Éducation'



idProduit	description	code	catégorie
1001	'BébéLala'	'ABC999-Z'	'Stratégie'

- Impossible de faire des analyses sur l'ancienne valeur;
- À utiliser seulement lorsque l'ancienne valeur n'est pas significative pour les besoins d'affaires;
- Exige de mettre à jour les données agrégés avec l'ancienne valeur.

Les entrepôts de données et l'analyse de données

Stratégie SCD Type 2

idProduit	description	code	catégorie	dateEffective	dateExpirée
1001	'BébéLala'	'ABC999-Z'	'Formation'	'2007-10-08'	'9999-12-31'



idProduit	description	code	catégorie	dateEffective	dateExpirée
1001	'BébéLala'	'ABC999-Z'	'Formation'	'2007-10-08'	'2008-10-31'
1002	'BébéLala'	'ABC999-Z'	'Stratégie'	'2008-11-01'	'9999-12-31'

- Permet de faire des analyses historiques;
- Demande l'ajout d'une nouvelle ligne par changement;
- À utiliser lorsque l'ancienne valeur a une signification analytique ou si le changement est une information en soi.

Les entrepôts de données et l'analyse de données

Stratégie SCD Type 3

idProduit	description	code	oldCatégortie	newCatégorie	dateModif
1001	'BébéLala'	'ABC999-Z'	'Éducation'	'Éducation'	'9999-12-31'



idProduit	description	code	oldCatégortie	newCatégorie	dateModif
1001	'BébéLala'	'ABC999-Z'	'Éducation'	'Stratégie'	'2008-11-01'

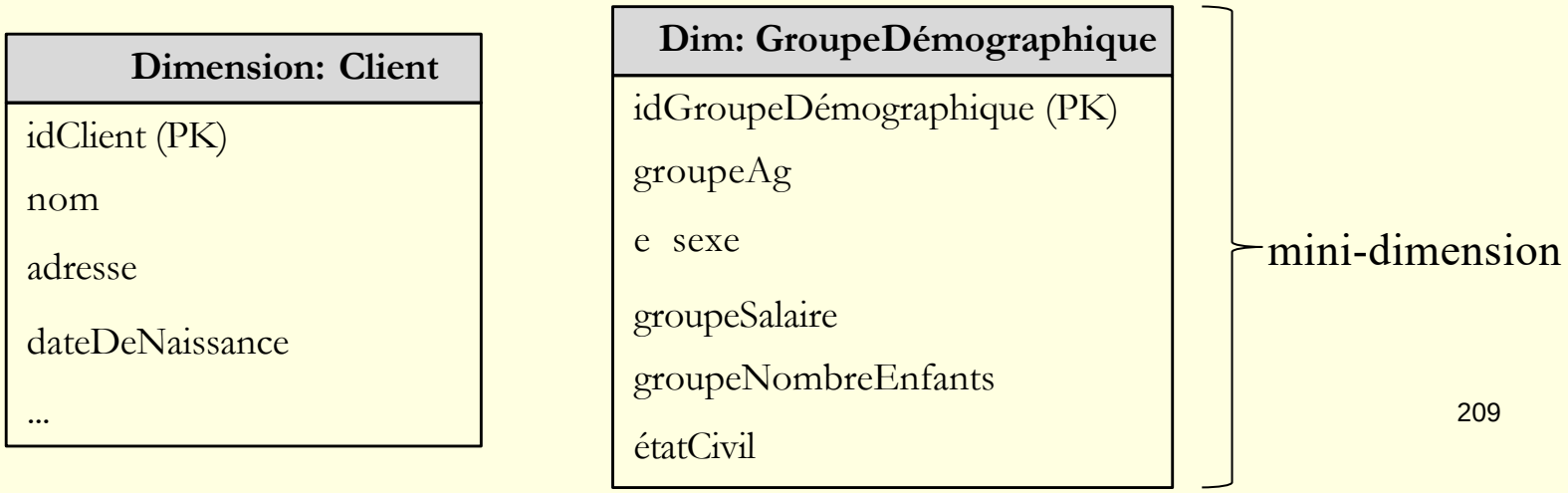
- Rarement employée;
- Profondeur de l'historique est de un seul changement;
- Utilisé lorsqu'on veut vouloir comparer les faits avec l'ancienne ou la nouvelle valeur;

© Copyright 2026, Moulla

Les entrepôts de données et l'analyse de données

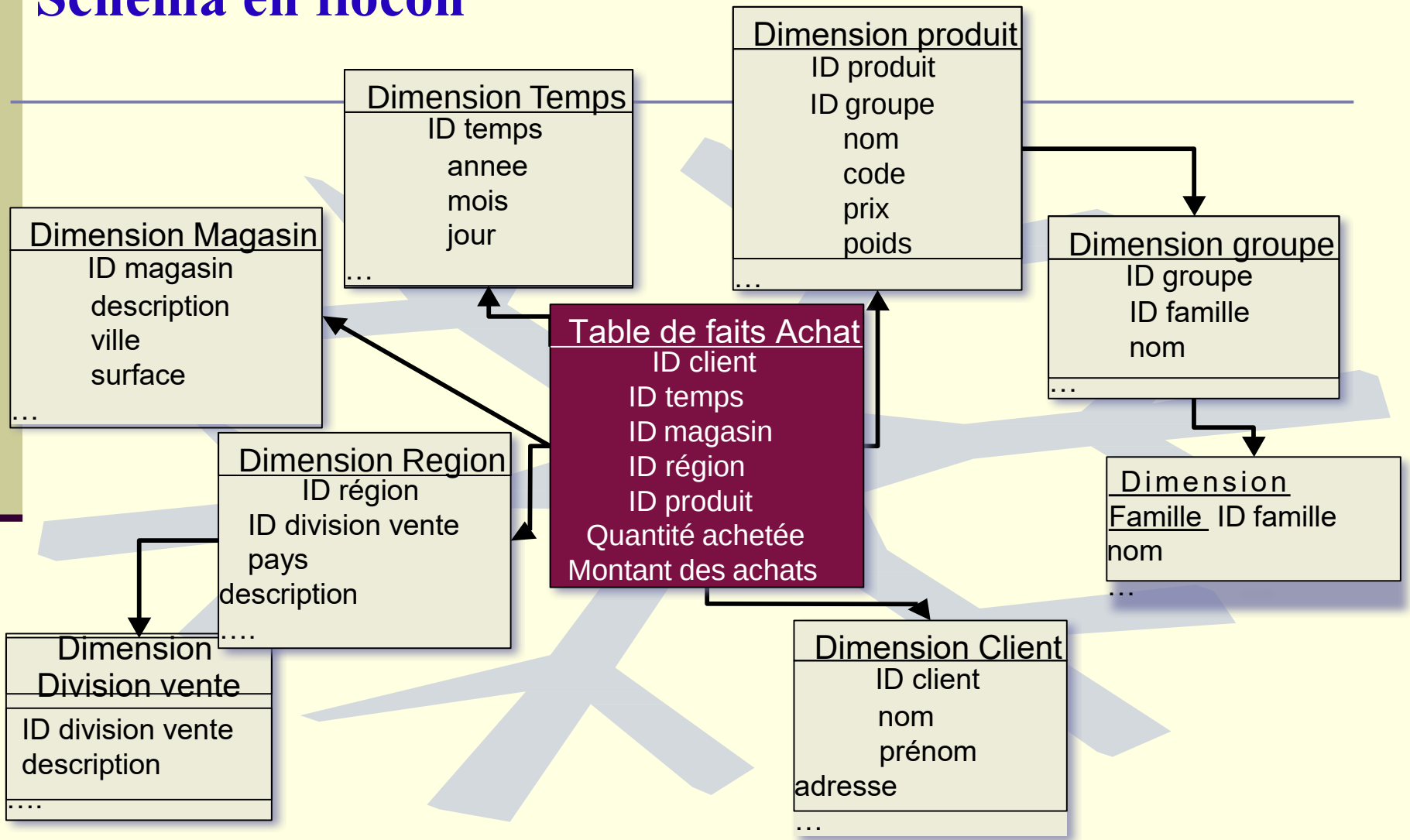
Mini-dimensions

- Sert lorsque qu'une dimension renferme des attributs qui peuvent changer souvent:
  - Ex: le profil démographique des clients (âge, revenu, etc.).
- Solution: mettre les attributs plus volatiles dans une dimension séparée (*mini-dimension*) où le grain est différent;



Les entrepôts de données et l'analyse de données

Schéma en flocon



Les entrepôts de données et l'analyse de données

Schéma en flocon

- Provient de la normalisation des tables de dimension;
- Une table de fait et des dimensions décomposées en sous hiérarchies;
- **Avantages:**
  - Petite économie d'espace;
  - Plus facile de mettre à jour les dimensions en cas de changement.
- **Inconvévients:**
  - Schéma moins intuitif aux utilisateurs d'affaires;
  - Dégradation de la performance à cause des jointures additionnelles.
- En général, on préfère ne pas normaliser les tables de dimension.

Les entrepôts de données et l'analyse de données

Stockage vertical des données dans les DW

- La plupart des RDBMS transactionnels stockent les données horizontalement:

ligne 1			ligne 2			ligne 3
col 1	col 2	col 3	col 1	col2	col 3	…

  - Facilite les requêtes retournant une ou plusieurs lignes.

- Dans les entrepôts de données, les requêtes portent souvent sur les colonnes (ex: SUM, AVG, MIN, MAX, etc.)
  - Il peut être plus efficace de stocker les données par colonnes:

col 1			col 2			col 3
ligne 1	ligne 2	…	ligne 1	ligne 2	…	…

Les entrepôts de données et l'analyse de données

Requêtes analytiques

- Exemple:
  - 300 magasins;
  - 40,000 produits;
  - 80 marques (500 produits par marque);
  - Environ une vente à chaque semaine, pour chaque produit, dans chaque magasin.
- Requête sur 1 produit, 1 magasin, 1 semaine:
  - Agrégation de **1 ligne** de la table de faits.
- Requête sur 1 produit, tous les magasins, 1 semaine:
  - Agrégation de **300 lignes** de la table de faits.
- Requête sur 1 marque, tous les magasins, 1 année:
  - Agrégation de **7,800,000 lignes** de la tables de faits.

Les entrepôts de données et l'analyse de données

Pré-agrégation des données

- Accélère les requêtes analytiques en pré-calculant l'agrégation de faits à différents niveaux des hiérarchies dimensionnelles
- Duplique l'information contenue dans la table de faits atomique (niveau le plus granulaire)
- Exemple (suite):
  - Table pré-agrégée où chaque ligne donne le total des ventes durant une semaine, pour une marquede produits dans un certain magasin;
  - Requête sur 1 marque, tous les magasins, 1 année:
    - Agrégation de **15,600 lignes** au lieu de 7.8 millions.

    (Où 15,600 représente le produit entre le nombre de magasins et le nombre de semaines dans une année).

Les entrepôts de données et l'analyse de données

Pré-agrégation des données

- Tenir compte du type et de la fréquence des requêtes faites à l'entrepôt (profilage de requêtes)
- Choisir un niveau de hiérarchie offrant un bon compromis entre l'utilité et le gain en performance.
  - Règle: Chaque nouvelle ligne doit agréger au moins 10 lignes de la table de faits atomiques.
- Approches d'agrégation:
  - BD relationnelle avec vue matérialisées (ROLAP)
  - Cube de données multidimensionnelles (MOLAP)
  - Approche hybride (HOLAP).

Les entrepôts de données et l'analyse de données

Agrégation ROLAP

- Données stockées dans une base de données relationnelles
- Un moteur OLAP (exemple moteur ROLAP = Mondrian) permet de simuler le comportement d'un SGBD multidimensionnel.
- Avantages: Plus facile et moins cher à mettre en place.
- Inconvénients: Moins performant lors des phases de calcul.

Exemple dans Oracle:

```
CREATE MATERIALIZED VIEW TransactionAgrégée
REFRESH FORCE ENABLE QUERY REWRITE
AS
SELECT idMagasin,
       P.catégorie AS catégorie,
       D.semaine AS semaine,
       SUM(T.montant) AS montantAgrégé
FROM Transaction T, Produit P, Date D
WHERE T.idProduit = P.id AND T.idDate = D.id
GROUP BY idMagasin, P.catégorie, D.semaine
```

Paramètre	Description
REFRESH FORCE	Synchronisation incrémentale lorsque possible, sinon complète
ENABLE QUERY REWRITE	Permet de réécrire la requête si cela améliore la performance

Les entrepôts de données et l'analyse de données

Cubes multi-dimensionnels (MOLAP)

- Représentent les données sous la forme d'un tableau multidimensionnel (cube) où:
  - Les coordonnées d'une case correspondent à une combinaison de valeurs des dimensions du cube;
  - Le contenu d'une case correspond aux faits (mesures) pour ces valeurs;
- Se basent beaucoup sur le pré-calcul d'agrégations selon les hiérarchies dimensionnelles.

© Copyright 2026, Moulla

Les entrepôts de données et l'analyse de données

Cubes multi-dimensionnels (MOLAP)

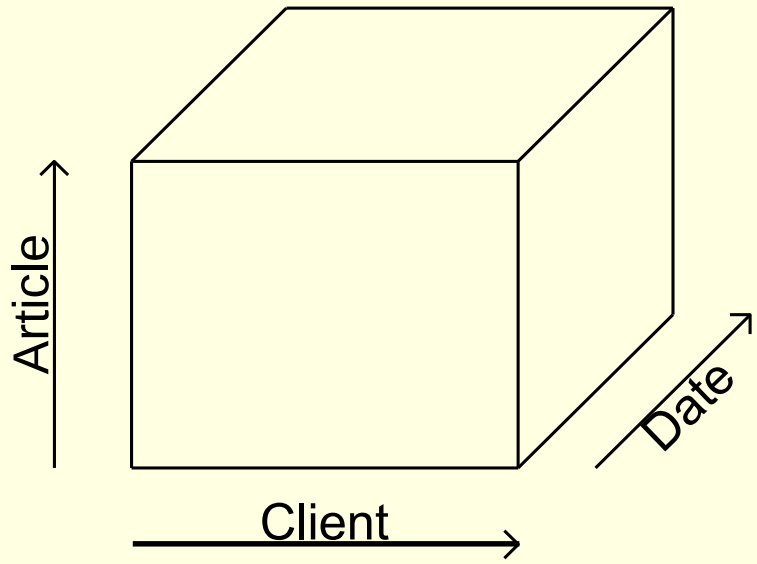
- Offrent une meilleure performance pour les opérations d'analyse multidimensionnelle que ROLAP;
- Facilitent la gestion des agrégations;
- Peuvent être limités en terme du nombre de dimensions et de valeurs possibles pour ces dimensions (explosion de la mémoire pour les agrégations);
- Les mises à jour peuvent être plus coûteuses que ROLAP;
- La technologie est plus fermée que les ROLAP (format souvent propriétaire).
- Plusieurs produits commerciaux offrent une approche hybride (HOLAP) combinant les avantages du MOLAP et du ROLAP.

exemples de moteurs MOLAP: Microsoft Analysis Services, Hyperion, …

Les entrepôts de données et l'analyse de données

Exemple de cube OLAP

Table Vente			
noClient	**noArticle**	**dateVente**	**montant**
1	10	10/01/2000	100
2	20	10/01/2000	200
3	10	10/01/2000	500
1	10	15/01/2000	300
3	40	15/01/2000	100
2	60	16/01/2000	200
4	60	20/02/2000	400
2	10	20/02/2000	200
1	40	25/02/2000	100
4	10	04/03/2000	300
1	20	04/03/2000	200
…			



Les entrepôts de données et l'analyse de données

HOLAP

- Hybride OLAP:
  - tables de faits et tables de dimensions stockées dans SGBD relationnel (données de base)
  - données agrégées stockées dans des cubes
- Solution hybride entre MOLAP et ROLAP
- Bon compromis au niveau coût et performance

© Copyright 2026, Moulla

BD parallèles et réparties

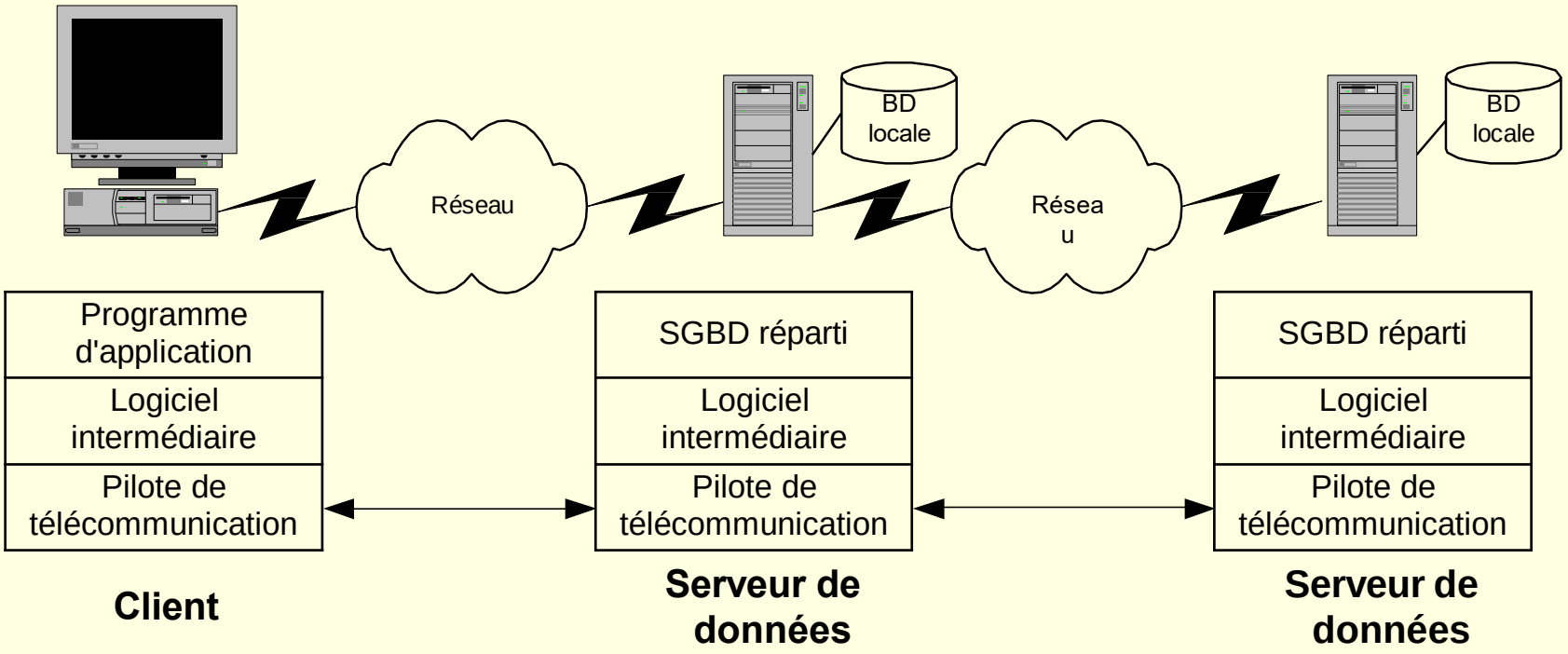
BD parallèles vs réparties

- **BD réparties**
  - Les données se trouvent sur **plusieurs sites** (noeuds)
  - Chaque site possède un certain degré d'autonomie
- **BD parallèles**
  - Exploitent le parallélisme à l'intérieur d'**un même site**
  - Peuvent utiliser plusieurs processeurs et/ou disques

© Copyright 2026, Moulla

BD parallèles et réparties

Bases de données réparties



- BD répartie dans deux sites
- Chaque site peut avoir une architecture parallèle

BD parallèles et réparties

Avantages des BD réparties

- **Performance**
  - Coûts de transfert réduits en rapprochant les données de leurs traitements
    - Ex: clients des succursales de Maroua dans une BD située à Maroua.
  - Parallélisme entre les sites (peu évident en pratique)
- **Fiabilité et disponibilité**
  - En cas de panne, les autres sites peuvent assurer la disponibilité des données.
- **Extensibilité (scalabilité)**
  - Si les besoins en stockage et calcul augmentent, on peut rajouter un nouveau noeud, au lieu de remplacer le serveur.

BD parallèles et réparties

Complexité des BD réparties

- **Transparence de la répartition**
  - Les applications ne doivent pas se soucier du fait que les données sont réparties sur plusieurs sites
  - Les nœuds peuvent avoir des schémas/SGBD différents
  - Répartition du dictionnaire de données
- **Gestion des transactions réparties**
  - Maintenir les propriété d'atomicité, isolation et durabilité des transactions est plus complexe dans un contexte réparti
- **Évaluation de requêtes réparties**
  - Les plans d'exécution doivent tenir compte
    - Localisation des données sur chaque site
    - Coût de transfert des données

BD parallèles et réparties

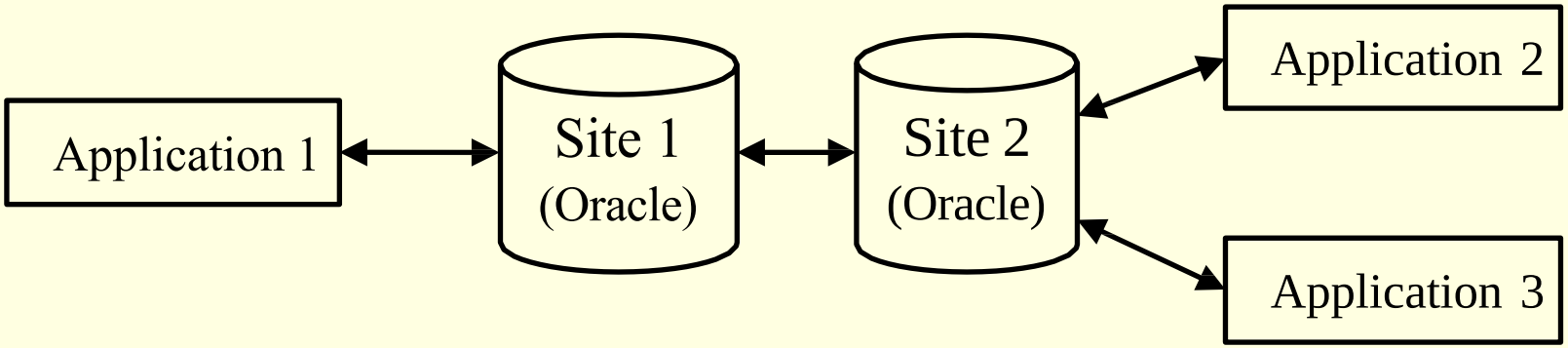
Architectures réparties

1. SGBD répartis homogènes
2. Multi-SGBD
3. SGBD fédérés

© Copyright 2026, Moulla

BD parallèles et réparties

SGBD répartis homogènes

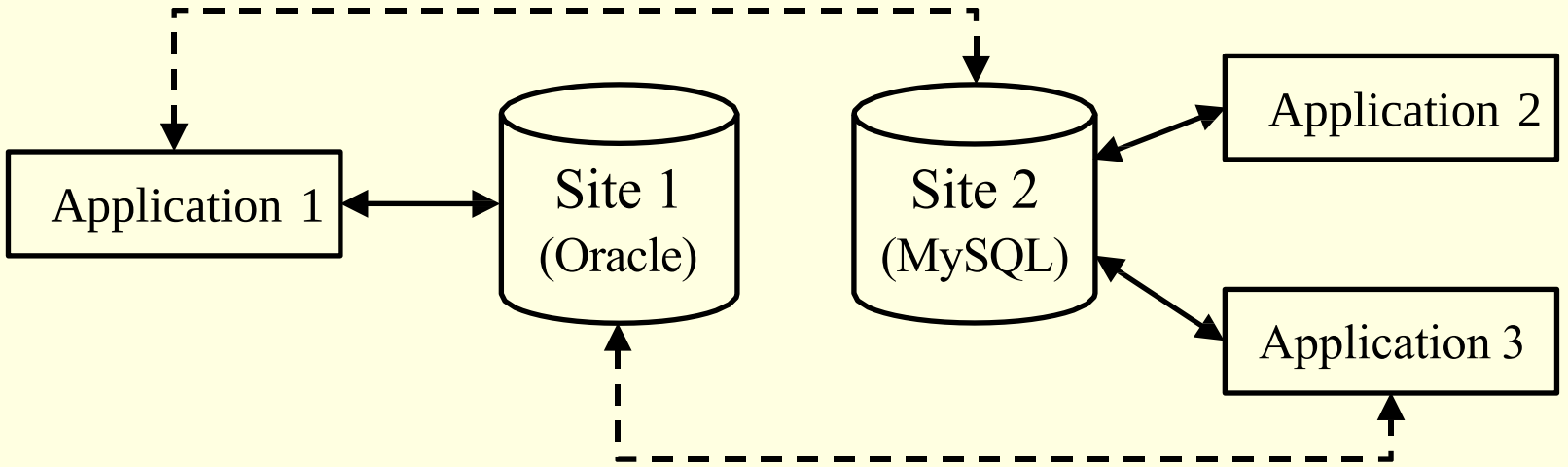


- Toutes les BD suivent un même schéma et utilisent la même technologie (ex: Oracle)
- Accès aux données et gestion des transactions réparties souvent fait de manière centralisée
- Plus grande fiabilité et performance dû à un meilleur couplage entre les sites.

© Copyright 2026, Moulla

BD parallèles et réparties

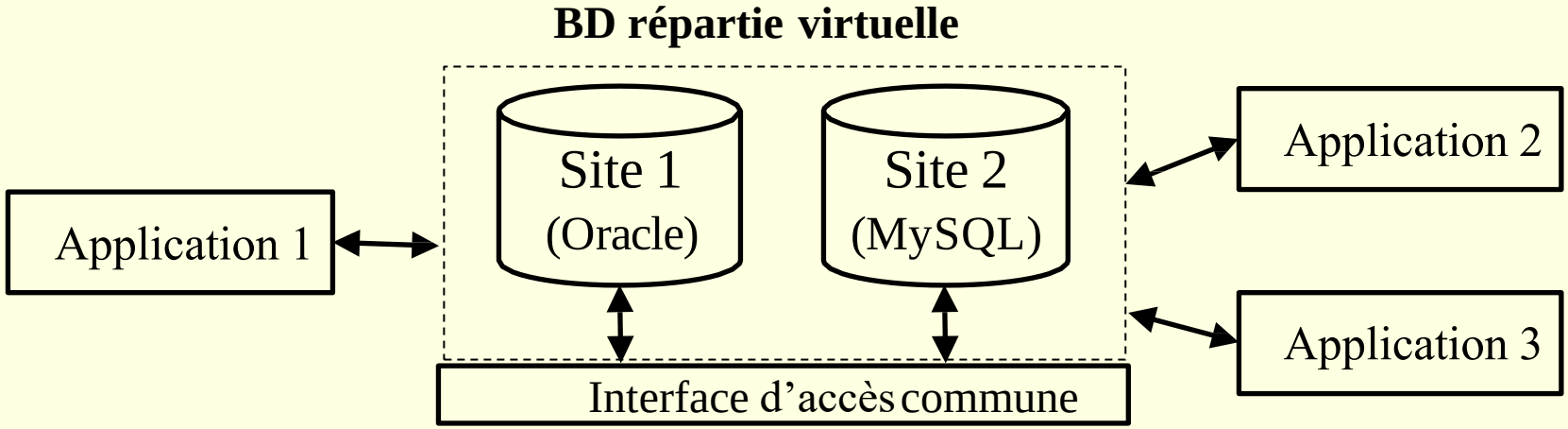
Multi-SGBD



- Chaque site est autonome et peut avoir un SGBD de type différent
- Aucune interface (ex: schéma conceptuel) commune.
- Accès aux données fait à partir de requêtes ad-hoc spécialisées.
- Peut devenir très complexe à gérer.

BD parallèles et réparties

SGBD fédérés



- Intègre plusieurs SGBD autonomes et potentiellement hétérogènes en une seule BD virtuelle.
- Interface d'accès commune pour masquer l'hétérogénéité des BD et la répartition des données.
- Mécanismes de coordination des transactions communs.

BD parallèles et réparties

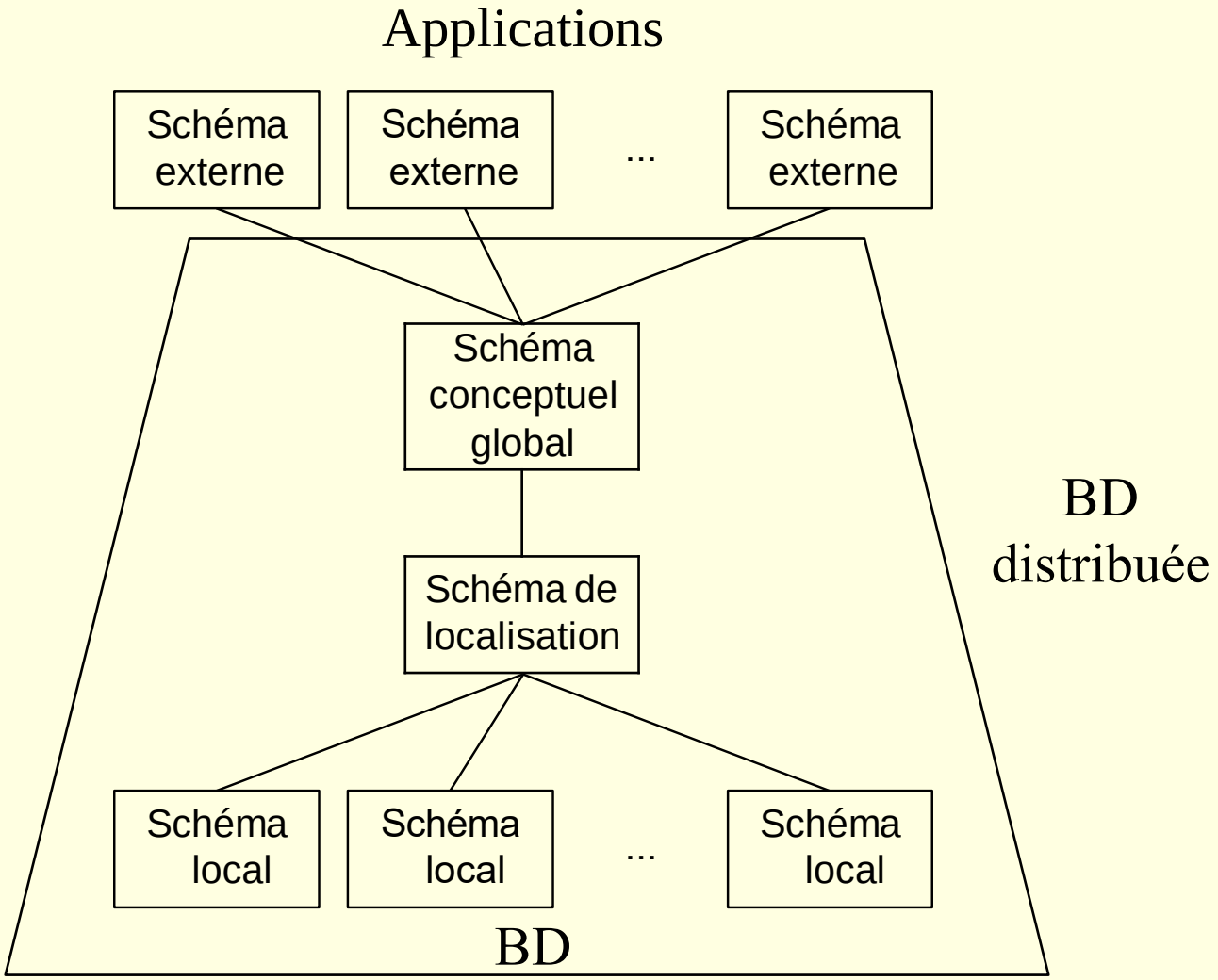
SGBD fédérés

- **Application type**:
  - Consolidation des données après la fusion/acquisition de compagnies.
- **Inconvénients**:
  - Intégration du schéma
  - Réécriture des requêtes complexes
  - Performance limitée.
- **Produits commerciaux**:
  - IBM InfoSphere Federation Server, Oracle Data Service Integrator, etc.

© Copyright 2026, Moulla

BD parallèles et réparties

Architecture des schémas



BD parallèles et réparties

Duplication répartie

- Copie partielle ou totale d'une ou plusieurs tables maîtresses situées sur des sites distants.
- Accès plus rapide aux données car moins de transferts
- Redondance assure la disponibilité des données si une des copies n'est plus accessible.
- Permet de contrôler l'accès en limitant les données distantes qui sont accessibles localement
- La synchronisation entre la table maîtresse et sa copie locale peut être synchrone ou asynchrone

© Copyright 2026, Moulla

BD parallèles et réparties

Duplication répartie (suite)

- **Duplication synchrone (*synchronous replication*)**
  - La sérialisabilité est assurée sur l'ensemble des nœuds.
  - Une transaction est confirmée seulement lorsque tous les sites ont été mis à jour.
- **Duplication asynchrone (*asynchronous replication*)**
  - Les mises-à-jour sont d'abord faites sur une copie primaire.
  - Les sites de réplication sont mis-à-jour en différé, à partir de la copie primaire, après la confirmation de la transaction.
  - Ex. d'implémentation: vues matérialisées

© Copyright 2026, Moulla

BD parallèles et réparties

Fragmentation répartie

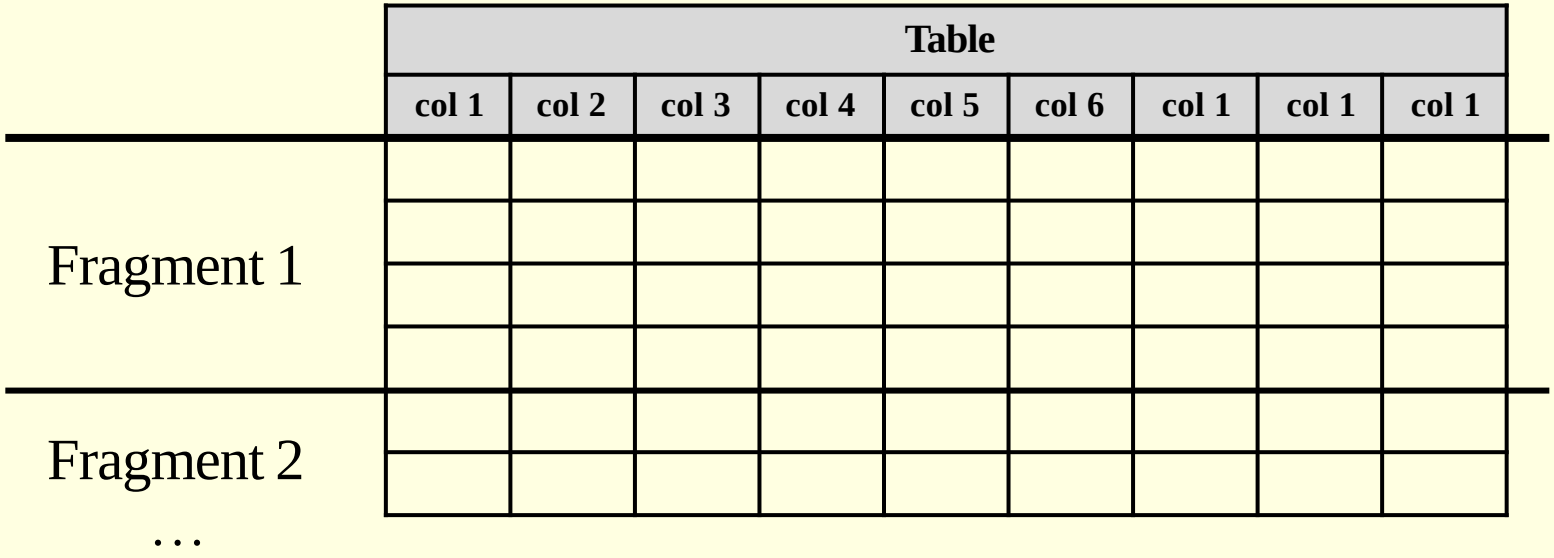
- Découpe une table en fragments répartis sur plusieurs sites
- Co-localisation des données avec leurs traitements
- Favorise l'extensibilité du système (vs données centralisées)
- Deux types de fragmentation:
  1. Fragmentation horizontale
  2. Fragmentation verticale

© Copyright 2026, Moulla

BD parallèles et réparties

Fragmentation répartie

- **Fragmentation horizontale**

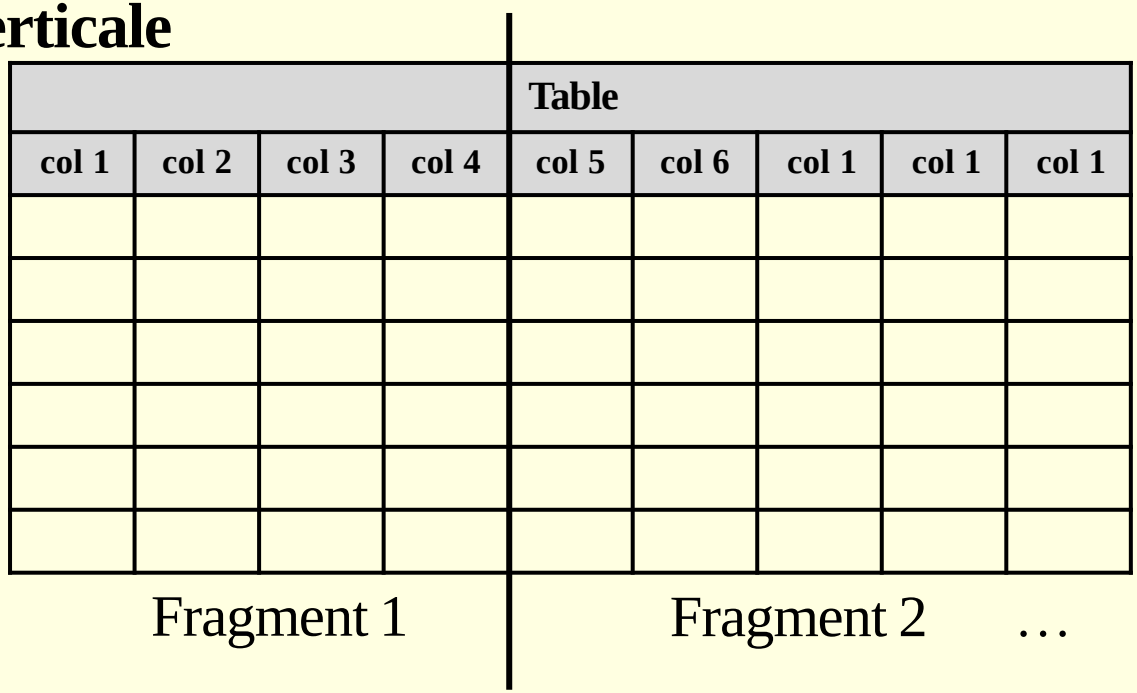


  - Chaque fragment contient un sous-ensemble de **lignes**
  - Ex: comptes des clients de Maroua sur le site de Maroua

BD parallèles et réparties

Fragmentation répartie

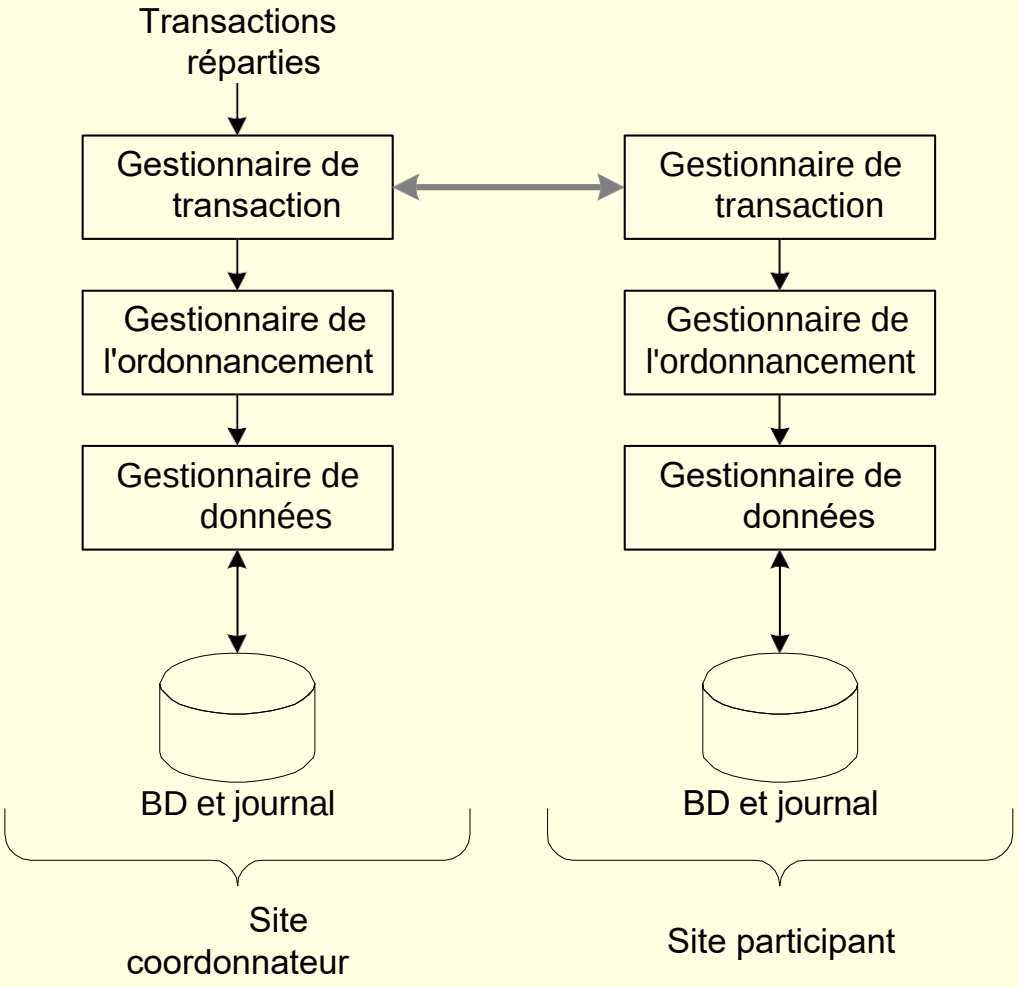
- **Fragmentation verticale**



  - Chaque fragment contient un sous-ensemble de **colonnes**
  - Ex: la colonne des salaires sur le site de la comptabilité
  - Moins utilisé que la fragmentation horizontale

BD parallèles et réparties

Transactions réparties



Site coordonnateur:

À l'origine de la transaction répartie

Site participant:

Coopère avec le gestionnaire de transaction du site coordonnateur

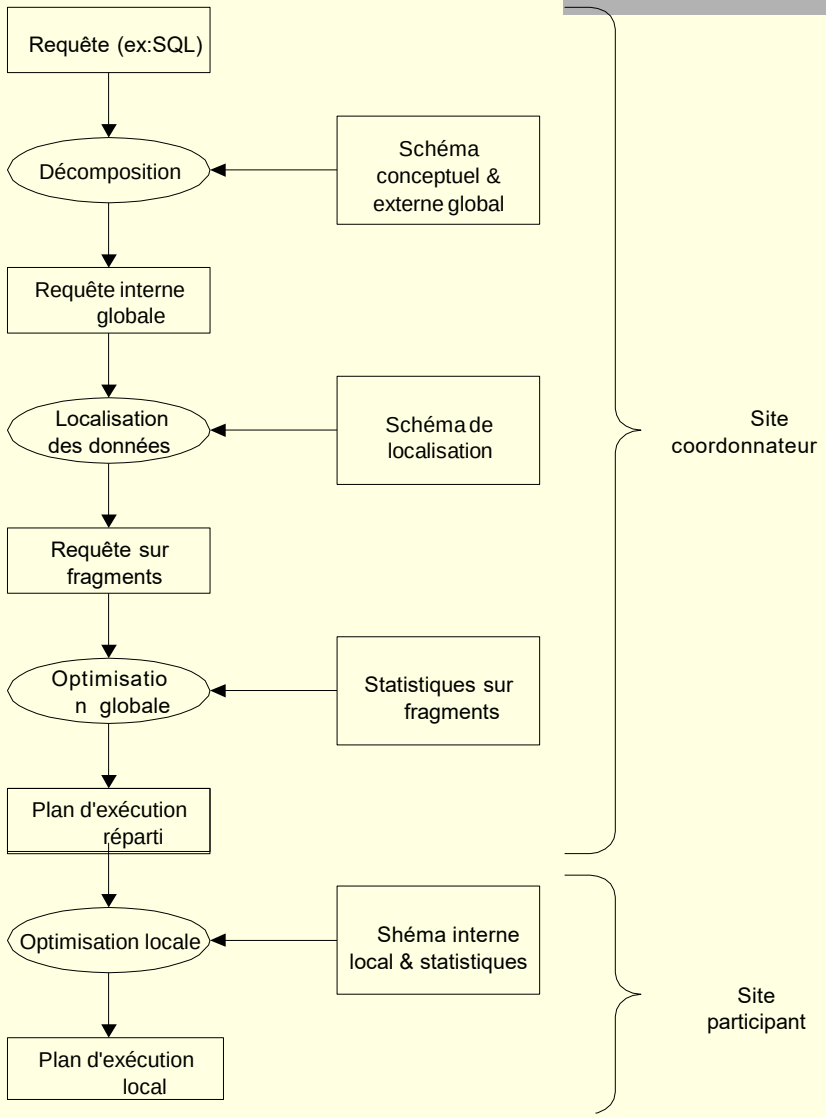
BD parallèles et réparties

Optimisation de requête répartie

- **Requêtes répartie vs locales**
  - Requête locale: le principal coût provient des écritures et lectures (E/S) en mémoire secondaire (ex: disques durs).
  - Requête répartie: le coût des E/S peut être inférieur à celui des transferts de données entre les sites.
- **Parallélisme intra-opération**
  - Parallélisation des sélections, jointures, etc.
  - Rarement avantageux dans les architectures réparties
- **Parallélisme inter-opération**
  - Calcule simultanément plusieurs opérations d'une même requête
  - Plus avantageux dans le contexte réparti

BD parallèles et réparties

Étapes d'optimisation



BD parallèles et réparties

Optimisation globale (sans parallélisme)

- Exemple: jointure entre les tables T_1, T_2 et T_3
  - Chaque site S_i contient une seule table T_i

Plan 1 :

Transférer T_1 au site 2

$T_1 \bowtie T_2 = R$ au site 2

Transférer R au site3

$R \bowtie T_3 =$ *Résultat final* au site 3

Plan 2 :

Transférer T_2 au site 1

$T_1 \bowtie T_2 = R$ au site 1

Transférer R au site3

$R \bowtie T_3 =$ *Résultat final* au site 3

Plan 3 :

Transférer T_1 au site 3

Transférer T_2 au site 3

$T_1 \bowtie T_2 \bowtie T_3 =$ *Résultat final* au site 3

Règle d'optimisation:
Transférer la plus petite table d'abord

BD parallèles et réparties

Parallélisme inter-opération et inter-site

- Opération : $T_1 \bowtie T_2 \bowtie T_3 \bowtie T_4$

Transférer T_2 au site 1
$T_1 \bowtie T_2 = R$ au site 1
En parallèle, transférer T_4 au site 3
$T_3 \bowtie T_4 = S$ au site 3
Transférer S au site 1
Ensuite, $R \bowtie S$ = *Résultat final* au site 1

© Copyright 2026, Moulla

BD parallèles et réparties

BD répartie avec Oracle (DATABASE LINKS)

- Permet à un usager local d'accéder aux tables d'une autre BD sans qu'il soit usager de cette BD.
- **Syntaxe:**

```
CREATE [SHARED] [PUBLIC]  DATABASE  LINK nomLien
CONNECT TO compteUsager   IDENTIFY BY motDePasse
 USING  nomService
```

- SHARED : permet de partager la connexion entre plusieurs usagers.
- PUBLIC : rend le lien disponible à tous les usagers locaux.
- nomService : doit être défini dans un fichier de configuration de la BD.

© Copyright 2026, Moulla

BD parallèles et réparties

BD répartie avec Oracle (DATABASE LINKS)

- **Exemple**: accès au catalogue de produits en Grande-Bretagne

```
 CREATE DATABASE  LINK site.europe.uk
CONNECT  TO Compte14 IDENTIFY BY ”abc123”
USING SID_siteUK

SELECT * FROM Compte14.Catalogue@site.europe.uk
```

© Copyright 2026, Moulla

BD parallèles et réparties

Transparence de localisation (SYNONYM)

- **Synonyme**
  - Évite aux applications de devoir connaître la localisation des données (transparence de localisation)
  - Permet de conserver les mêmes requêtes, même si le lien change

```
CREATE [PUBLIC] SYNONYM nomSynonyme
 FOR [nomSchéma].nomObjet[@lienBD]
```

- **Exemple :**

```
CREATE SYNONYM CatalogueEurope
FOR compte14.Catalogue@site.europe.uk

SELECT * FROM CatalogueEurope
WHERE ...
```

BD parallèles et réparties

Duplication avec vues matérialisées

- **Syntaxe:**

```
CREATE MATERIALIZED  VIEW nomVue
[REFRESH {FAST|COMPLETE|FORCE} [ON COMMIT|ON DEMAND]]
[FOR UPDATE]
```

- FAST: mise-à-jour incrémentale de la copie locale.
- COMPLETE: mise-à-jour complète de la copie locale à chaque rafraîchissement.
- FORCE: mise-à-jour incrémentale lorsque possible, sinon complète.
- ON COMMIT: rafraîchissement lorsqu'une transaction modifiant les tables maîtresses fait un commit.
- ON DEMAND (défaut): rafraîchissement sur demande de l'usager (l'instant peut être spécifié avec les clauses START et NEXT).
- FOR UPDATE: permet de mettre à jour la copie locale (les changements sont propagés aux tables maîtresses).

BD parallèles et réparties

Duplication avec vues matérialisées

- **Exemple:** copie locale du catalogue européen

```
CREATE MATERIALIZED VIEW CatalogueComplet
REFRESH FAST ON COMMIT
AS
(SELECT * FROM Catalogue@site.europe.uk)
UNION
(SELECT * FROM CatalogueCameroun)

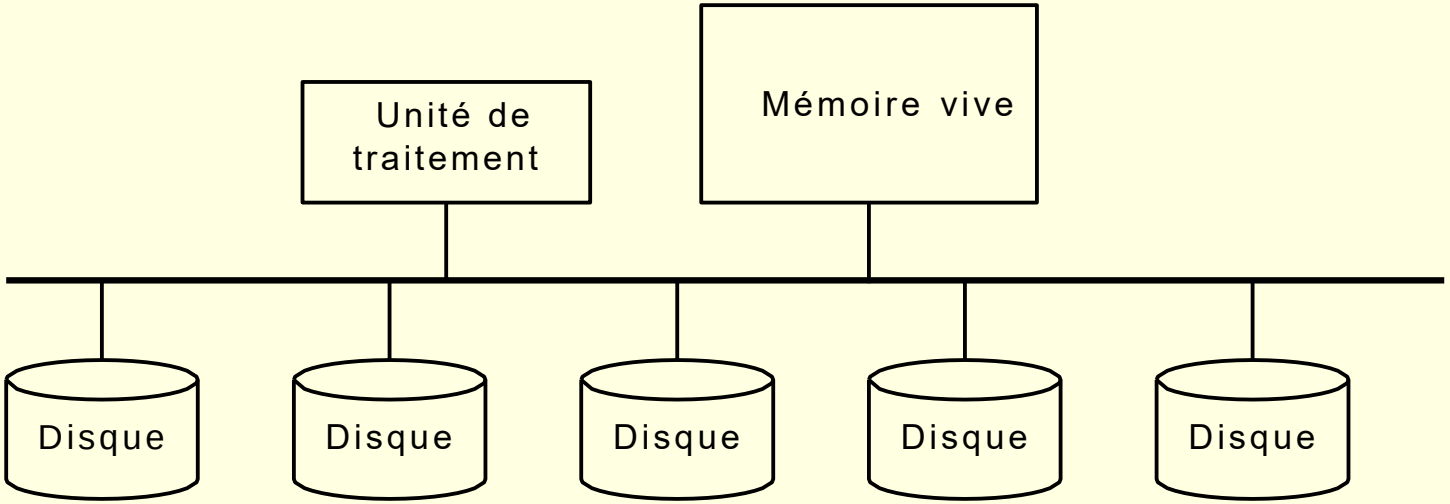
SELECT * FROM CatalogueComplet
WHERE ...
```

© Copyright 2026, Moulla

BD parallèles et réparties

Bases de données parallèles

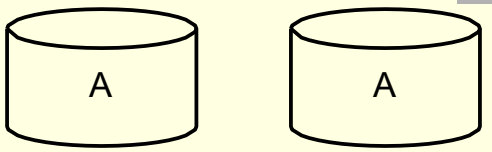
- Exploitation du parallélisme ***intrasite***
- Parallélisme de disques et d'unités de traitement



- Améliorent la fiabilité en dupliquant les données (ex: disques miroirs)
- Augmentent la performance en permettant de traitement en parallèle des requêtes

BD parallèles et réparties

Parallélisme de disque



- **Disques miroirs**:
  - Les données d'un disque maître sont dupliquées sur un autre disque
- **Code détecteur/correcteur d 'erreur**
  - Utilise un certain nombre de bits dans les données pour détecter une corruption et possiblement la corriger
  - Ex: bit de parité, code de Hamming
- **Répartition cyclique (*striping*)**
  - Améliore la performance en lecture/écriture par l'utilisation de plusieurs disques en parallèle
  - Répartit les données sur plusieurs disques, soit par bit (*bit-level striping*) ou par bloc (*bloc level striping*)

BD parallèles et réparties

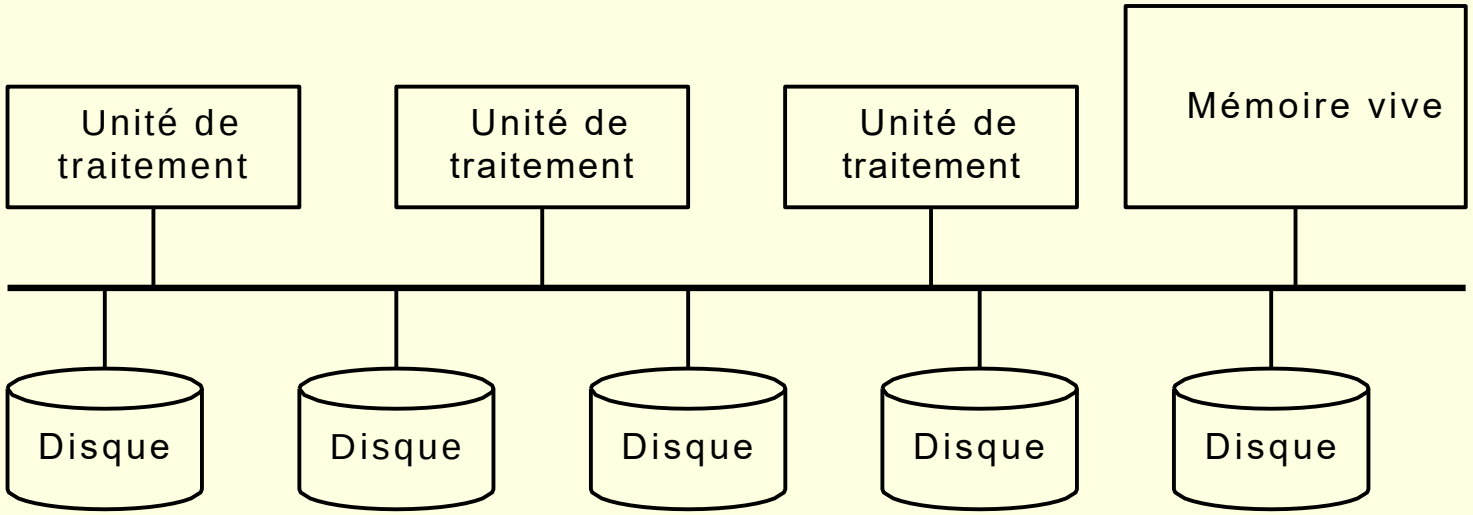
Autres formes de parallélisme

- Plusieurs processeurs
- Plusieurs unités de mémoire
- Duplication des processus SGBD
  - Processus miroirs pour fiabilité

© Copyright 2026, Moulla

BD parallèles et réparties

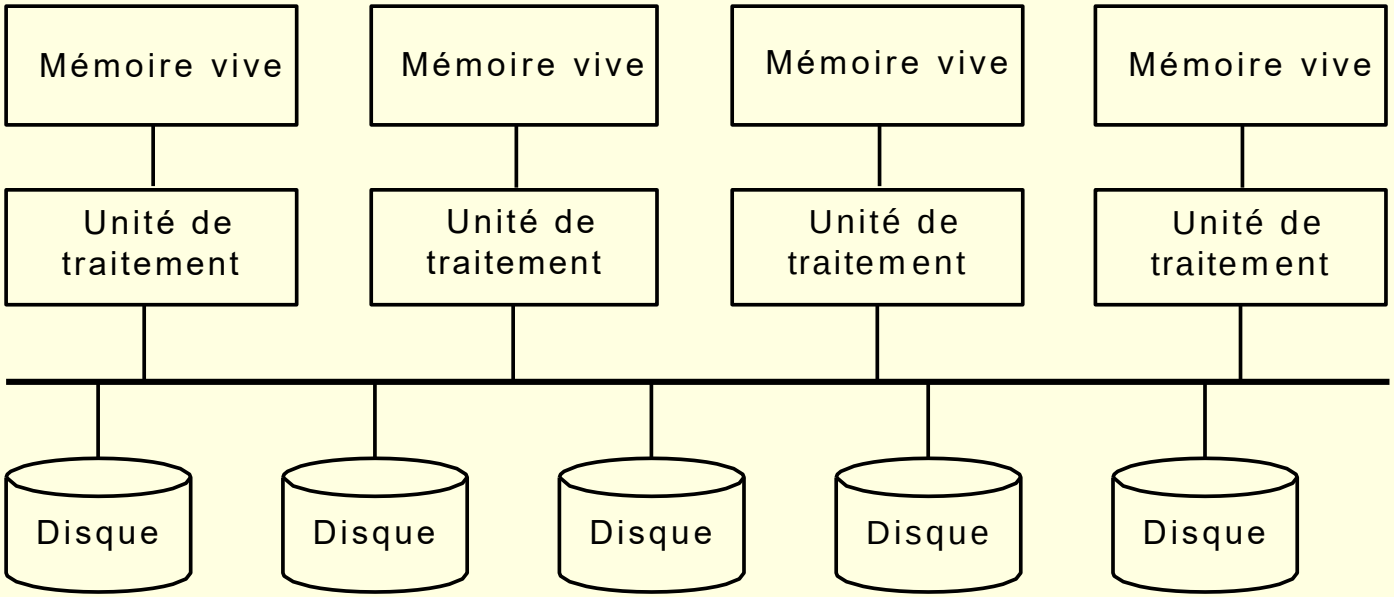
Architecture à mémoire partagée (Symmetric MultiProcessor – SMP)



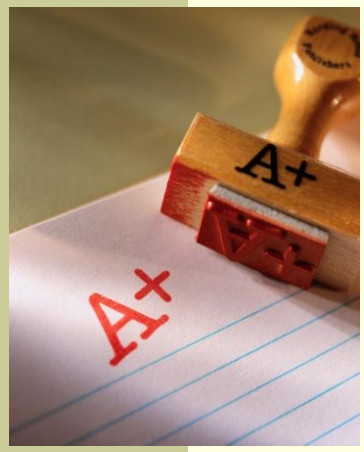
- Plusieurs processeurs qui partagent la même mémoire centrale;
- Mécanisme d'interconnexion très rapide (bus de données);
- Processus parallèles communiquent rapidement à travers la mémoire centrale partagée (goulot d'étranglement).

BD parallèles et réparties

Architecture à disques partagés

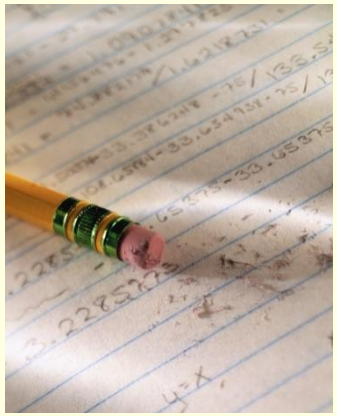


- Chaque unité de traitement possède sa propre mémoire vive;
- Les disques sont partagés entre les unités de traitement;
- Évite le goulot d'étranglement au niveau de la mémoire, mais complexifie la communication entre les processus;
- Ex: architecture en grappe (cluster)



C’est tout pour le cours de BD avancées

Maintenant, place à la partie pratique!



© Copyright 2026, Moulla